Height Balanced Trees

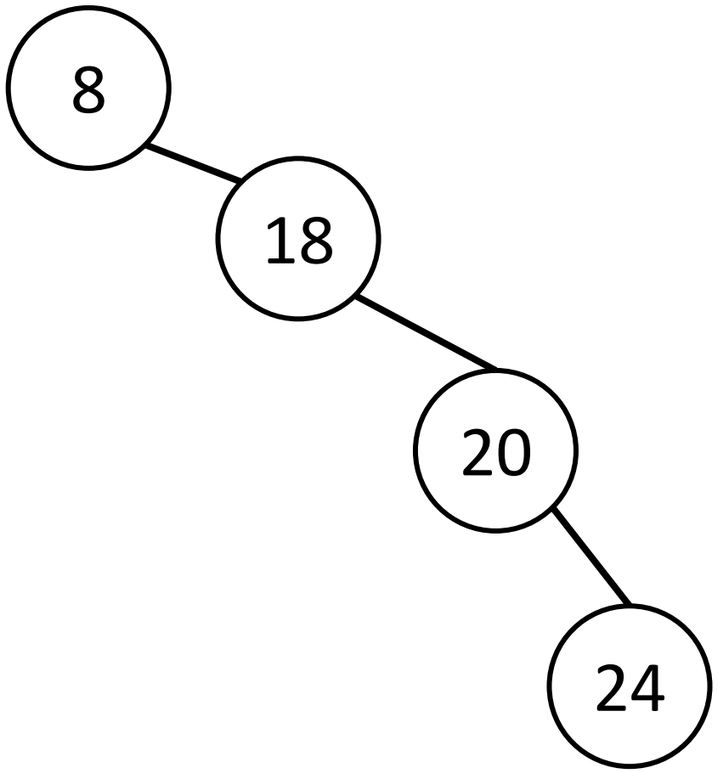
Saptarshi Pyne, Assistant Professor
Department of Computational and Data Sciences (CDS)
Indian Institute of Science Education and Research (IISER) Kolkata
Mohanpur, West Bengal, India 741246

CS3201 Programming and Data Structures II
March 13, 18, and 20, 2025

Limitation of the Binary Search Tree

In the worst case, we need to visit every single node when searching for a key.

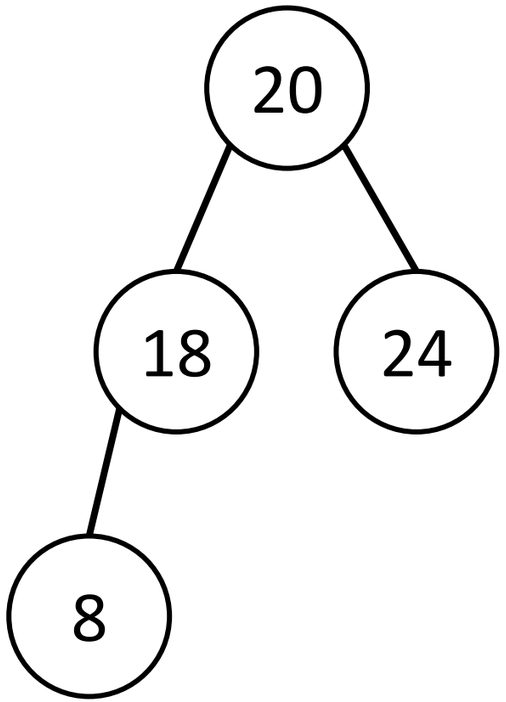
For example, if our search key is 24 and the given binary search tree is right-skewed (as shown here), we need to visit all the nodes before finding the matching node.



Resolution

We need to reorganise the binary search tree so that the difference between the heights of the left and right subtrees is small.

In other words, we need to ensure the binary search tree is always **height-balanced**.



Development of the Height-Balanced Binary Search Trees

Year	Event
Late 1950s	**The binary search tree** was independently proposed by several researchers.
1962	**The AVL tree** was proposed by two Soviet mathematicians Adel'son-Vel'skiˇı (pronounced velskii) and Landis.
1972	**The red-black tree** was proposed by a German computer scientist Rudolf Bayer. At that time, Bayer called it the 'symmetric binary B-trees'.
1978	**The red and black colour convention** was introduced by two American computer scientists Guibas and Sedgewick.

The AVL Tree

The Adelson-Velskii and Landis (AVL) Tree

An AVL tree is a binary search tree where the heights of the left and right subtrees of **any node x** differ by **at most 1**.

In other words,

$| x.left.h – x.right.h| \leq 1$

for every node x in an AVL tree T rooted at the node T.root.

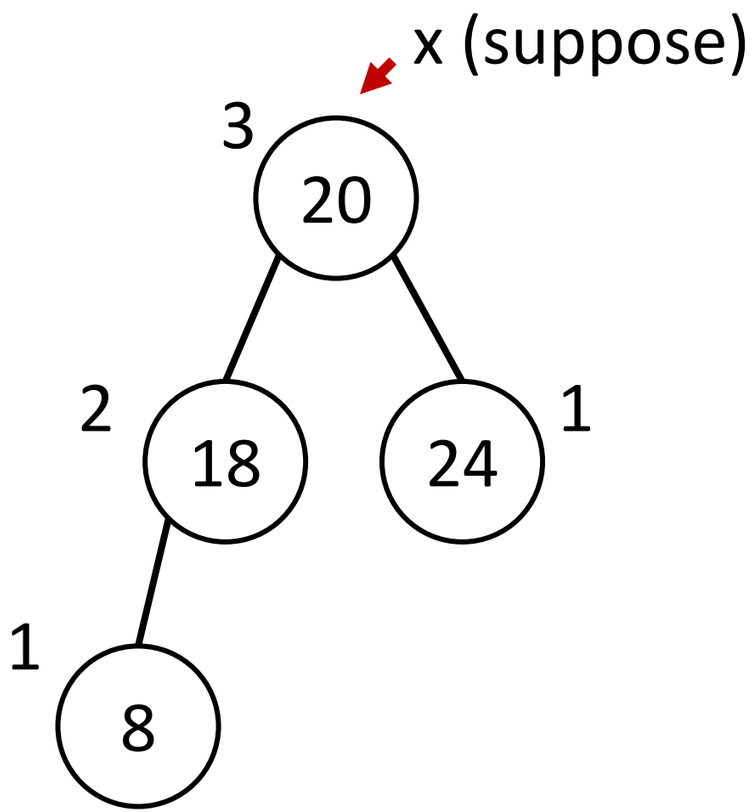
This inequality is also known as **the AVL property**.

Here, **height 'h'** is an additional attribute we need to store at each node.

The Structure of an AVL Tree Node

key	left (a pointer to the left child)	right (a pointer to the right child)	p (a pointer to the parent node. For the root node, it is NULL.)	h (height of the node. For leaf nodes, the height is 1 and it increases by 1 as we go towards the root.)	Satellite data (optional)	More attributes (optional)

How to calculate the height (h) of a node?



|x.left.h – x.right.h|
= |2 - 1|
= 1 ≤ 1.
The AVL property holds.

The Operations in an AVL Tree

The search operation is the same as that in any binary search tree.

The insertion and deletion operations must be different since we need to maintain the AVL property.

For insertion and deletion, we need two subroutines – the left and right rotations.

Left and Right Rotations

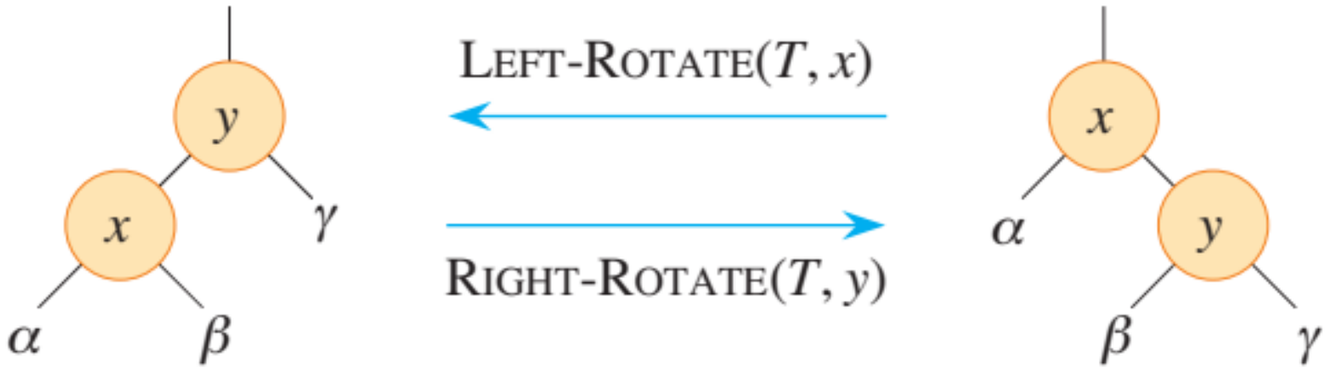


Figure 13.2 The rotation operations on a binary search tree. The operation LEFT-ROTATE(T, x) transforms the configuration of the two nodes on the right into the configuration on the left by changing a constant number of pointers. The inverse operation RIGHT-ROTATE(T, y) transforms the configuration on the left into the configuration on the right. The letters α, β, and γ represent arbitrary subtrees. A rotation operation preserves the binary-search-tree property: the keys in α precede $x.key$, which precedes the keys in β, which precede $y.key$, which precedes the keys in γ.

The Pseudocode of the Left Rotation

Given a tree T and a node x, left rotate the subtree rooted at x.

LEFT-ROTATE (T, x)

y = x.right

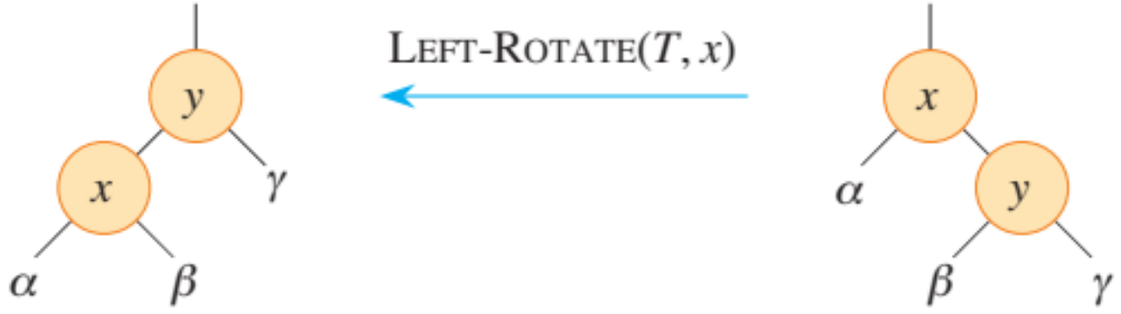
// Reposition beta

beta = y.left

x.right = beta

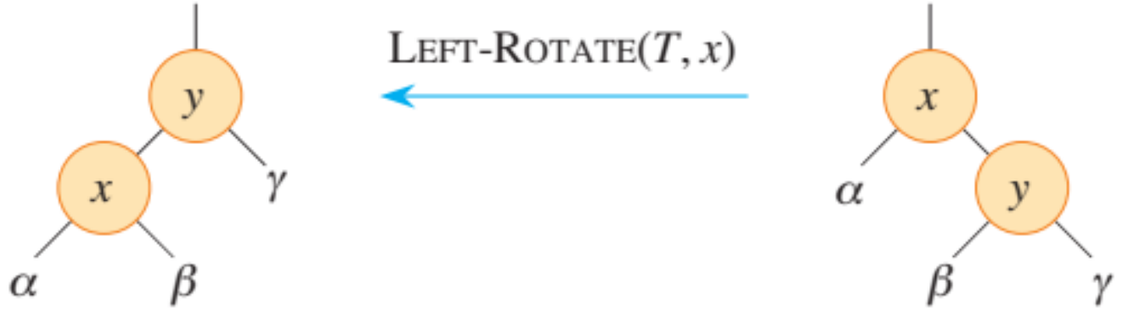
if beta != NIL

    beta.p = x



The Pseudocode of the Left Rotation (Contd.)

```
// Connect y to x's parent
y.p = x.p
if x == T.root
   T.root = y
else if x == x.p.left
   x.p.left = y
else if x == x.p.right
   x.p.right = y
```

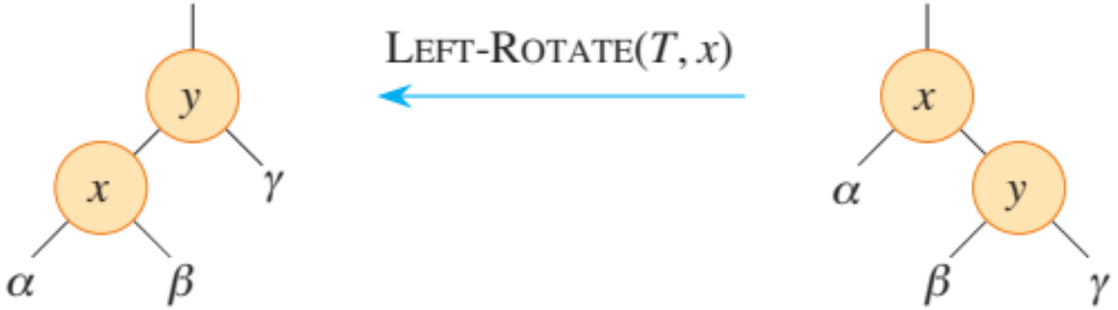


The Pseudocode of the Left Rotation (Contd.)

// Reconnect x and y

y.left = x

x.p =y

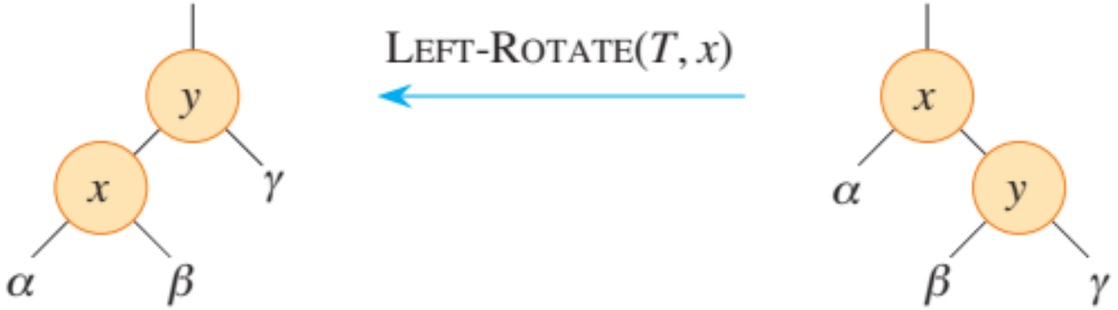


The Pseudocode of the Left Rotation (Contd.)

// Recompute the heights of x and y

x.h = 1 + max(x.left.h, x.right.h)

y.h = 1 + max(y.left.h + y.right.h)

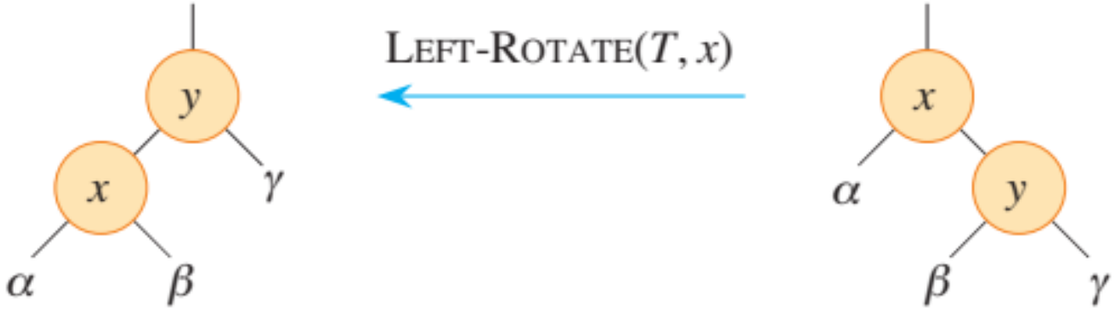


https://www.geeksforgeeks.org/dsa/insertion-in-an-avl-tree/

The Pseudocode of the Left Rotation (Contd.)

// Return the new root of the given subtree

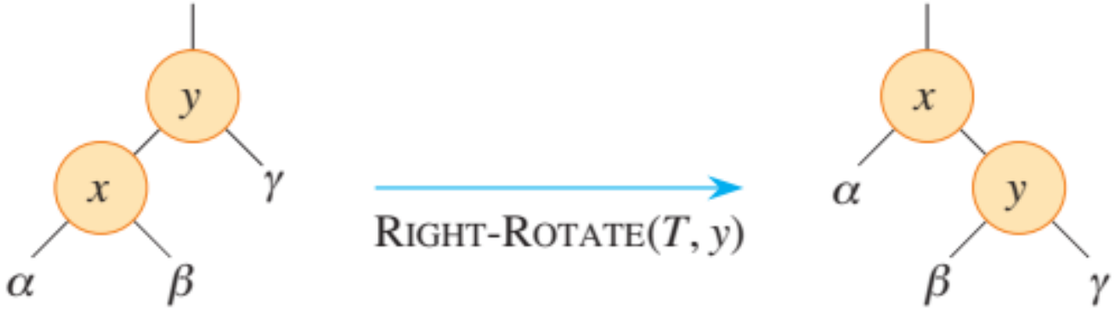
return y



https://www.geeksforgeeks.org/dsa/insertion-in-an-avl-tree/

The Pseudocode of the Right Rotation

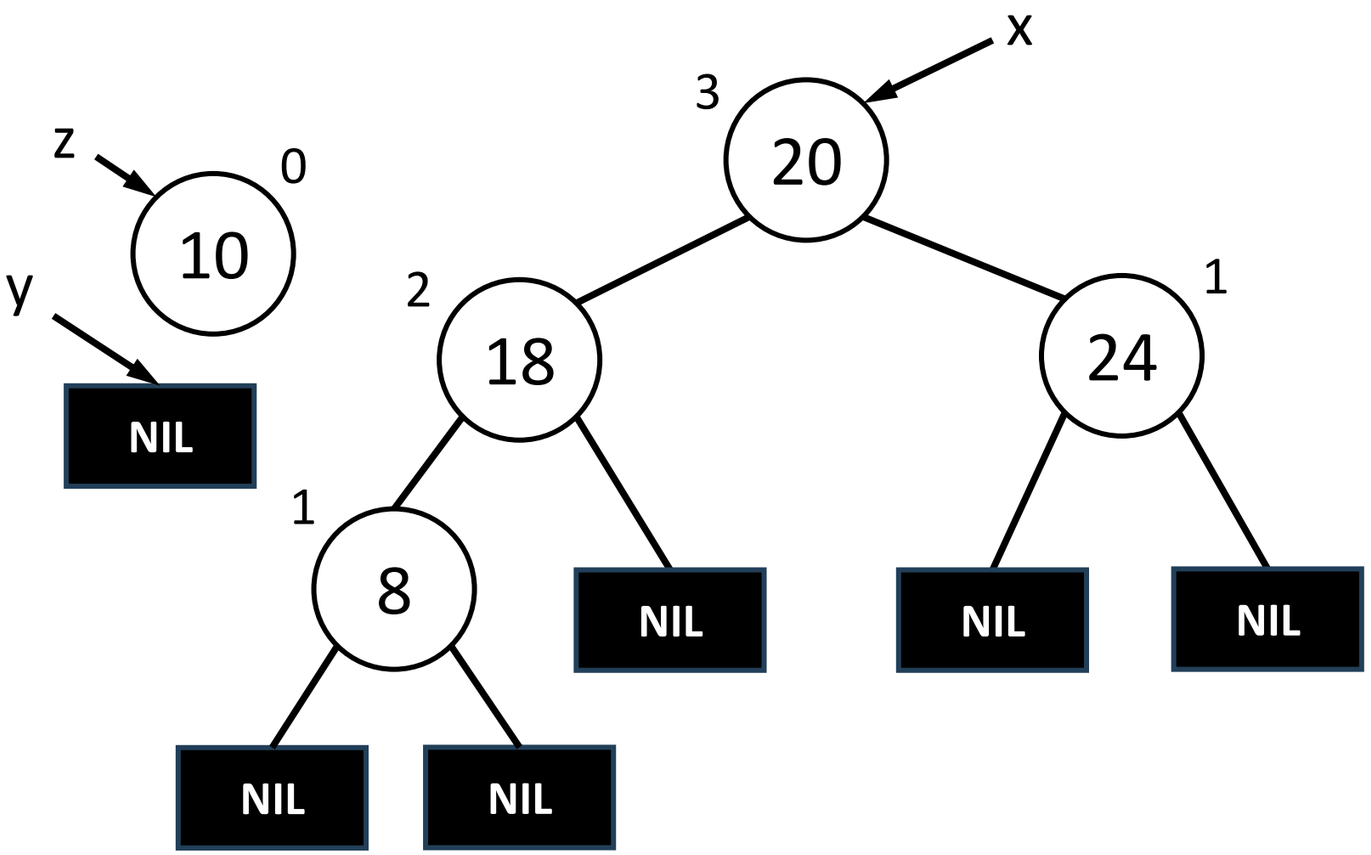
The pseudocode of the right rotation should be symmetric to that of the left rotation.



Inserting a New Node into an AVL Tree

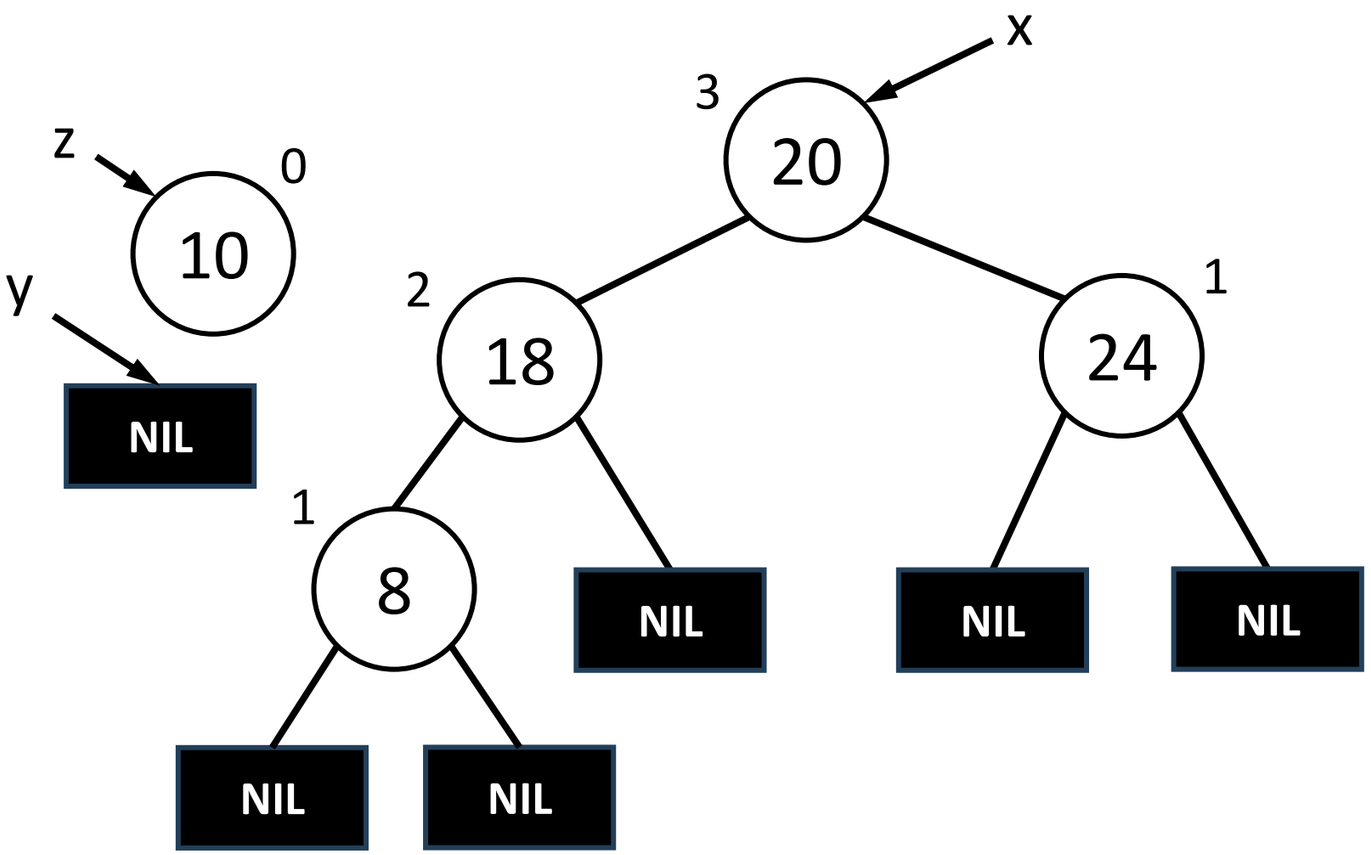
T = The given tree
z = The new node

AVL-INSERT(T, z)
y = NIL
x = T.root



Inserting a New Node into an AVL Tree (contd.)

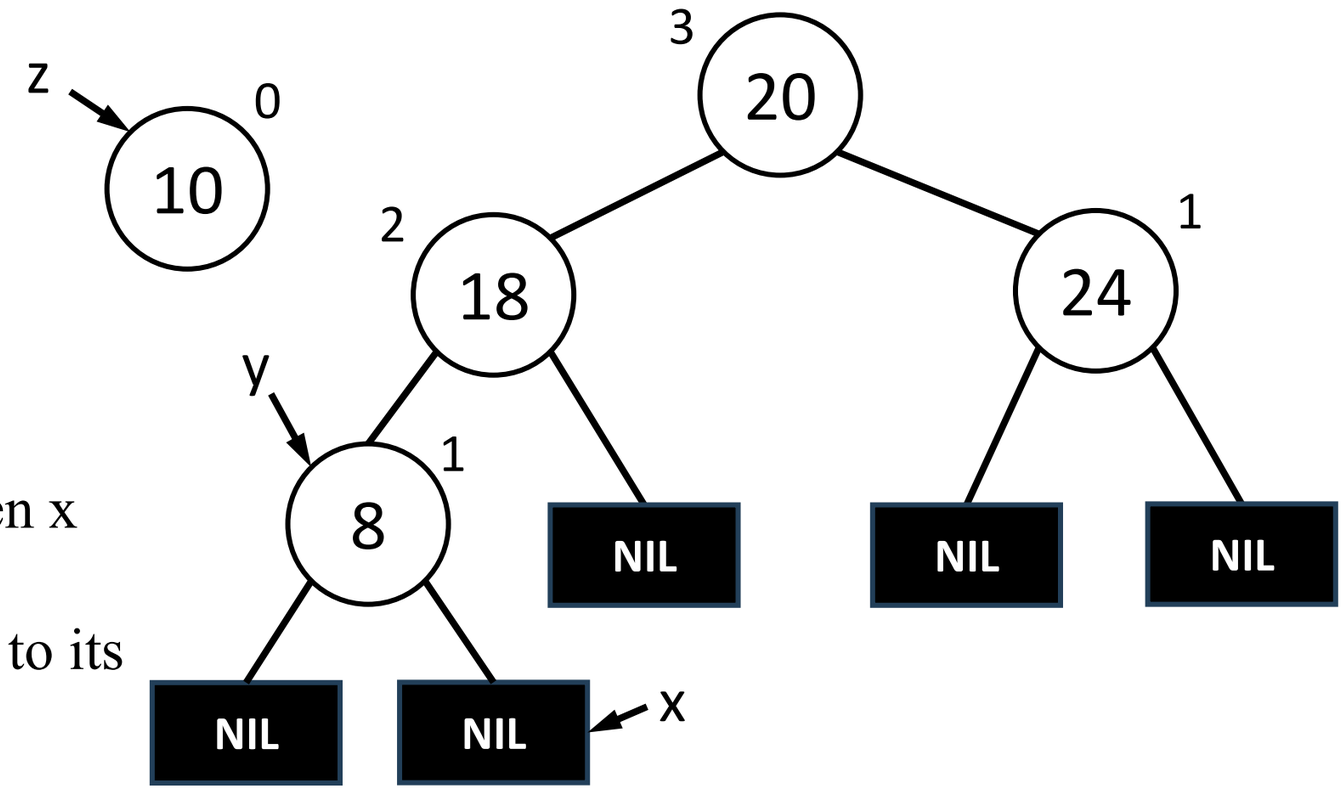
```
while x != NIL
    y = x
    if z.key < x.key
      x = x.left
    else
      x = x.right
```



Inserting a New Node into an AVL Tree (contd.)

```
while x != NIL
    y = x
    if z.key < x.key
      x = x.left
    else
      x = x.right
```

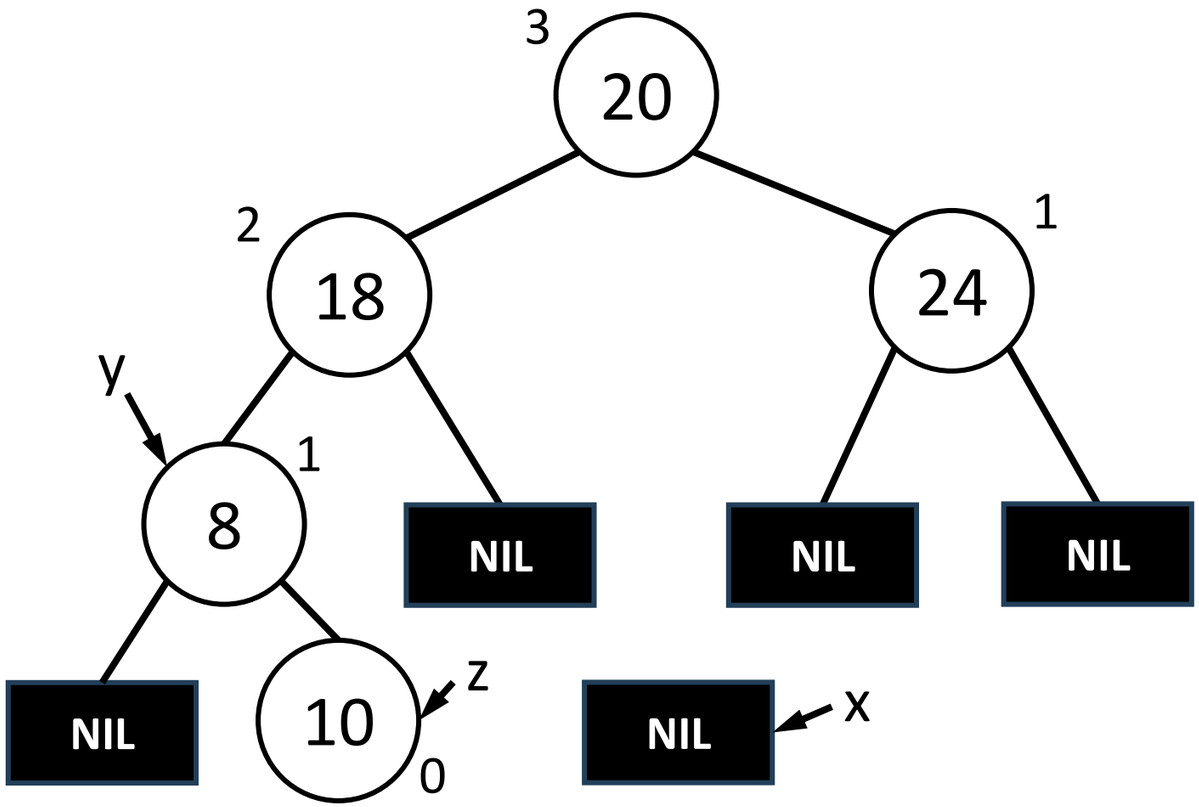
The while loop terminates only when x is pointing to the position where z should be inserted and y is pointing to its parental position.



Inserting a New Node into an AVL Tree (contd.)

```
z.p = y
if y == NIL
        T.root = z // Tree T was empty
else if z.key < y.key
        y.left = z
else if z.key > y.key
        y.right = z
```

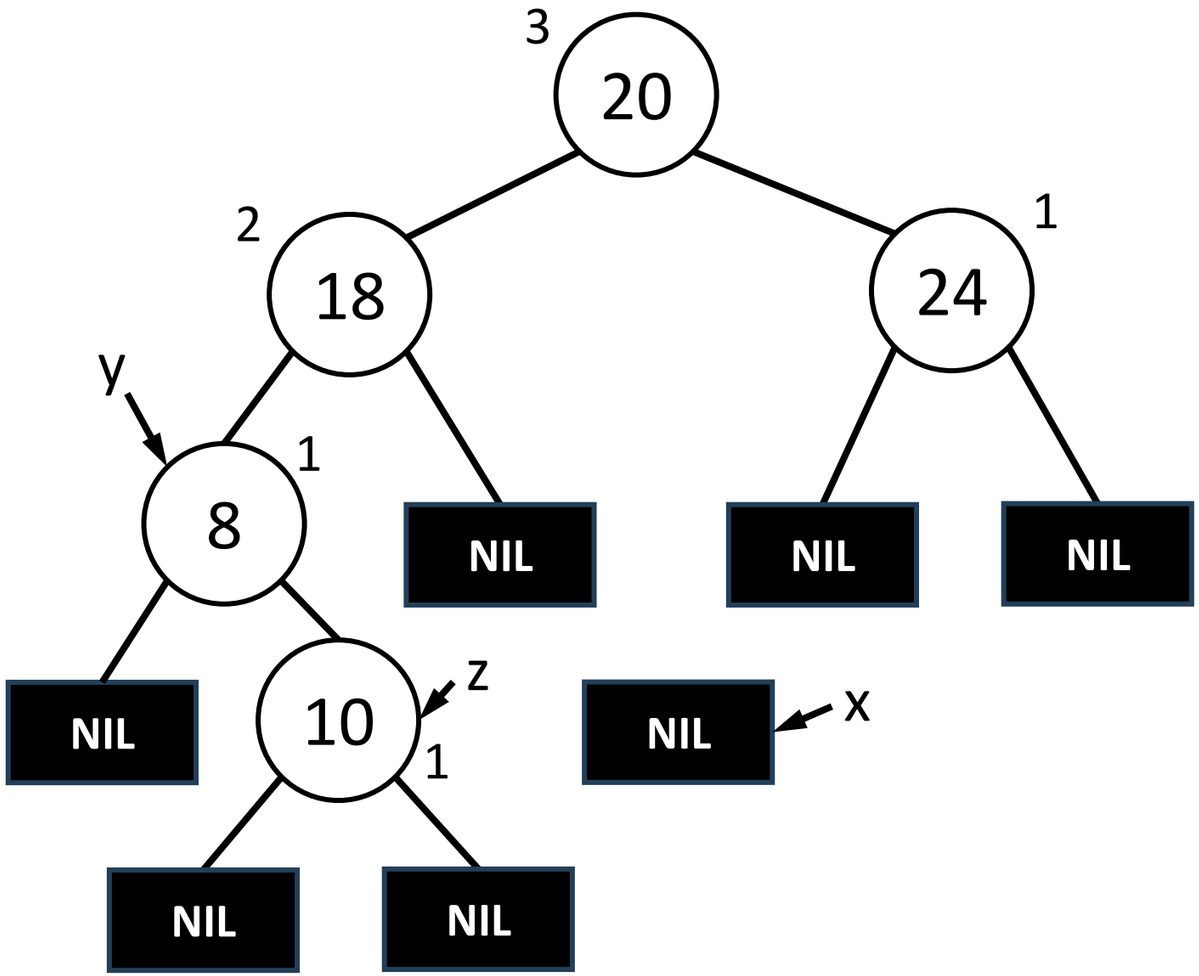
Insert z into the tree.



Inserting a New Node into an AVL Tree (contd.)

z.left = NIL
z.right = NIL
z.h = 1

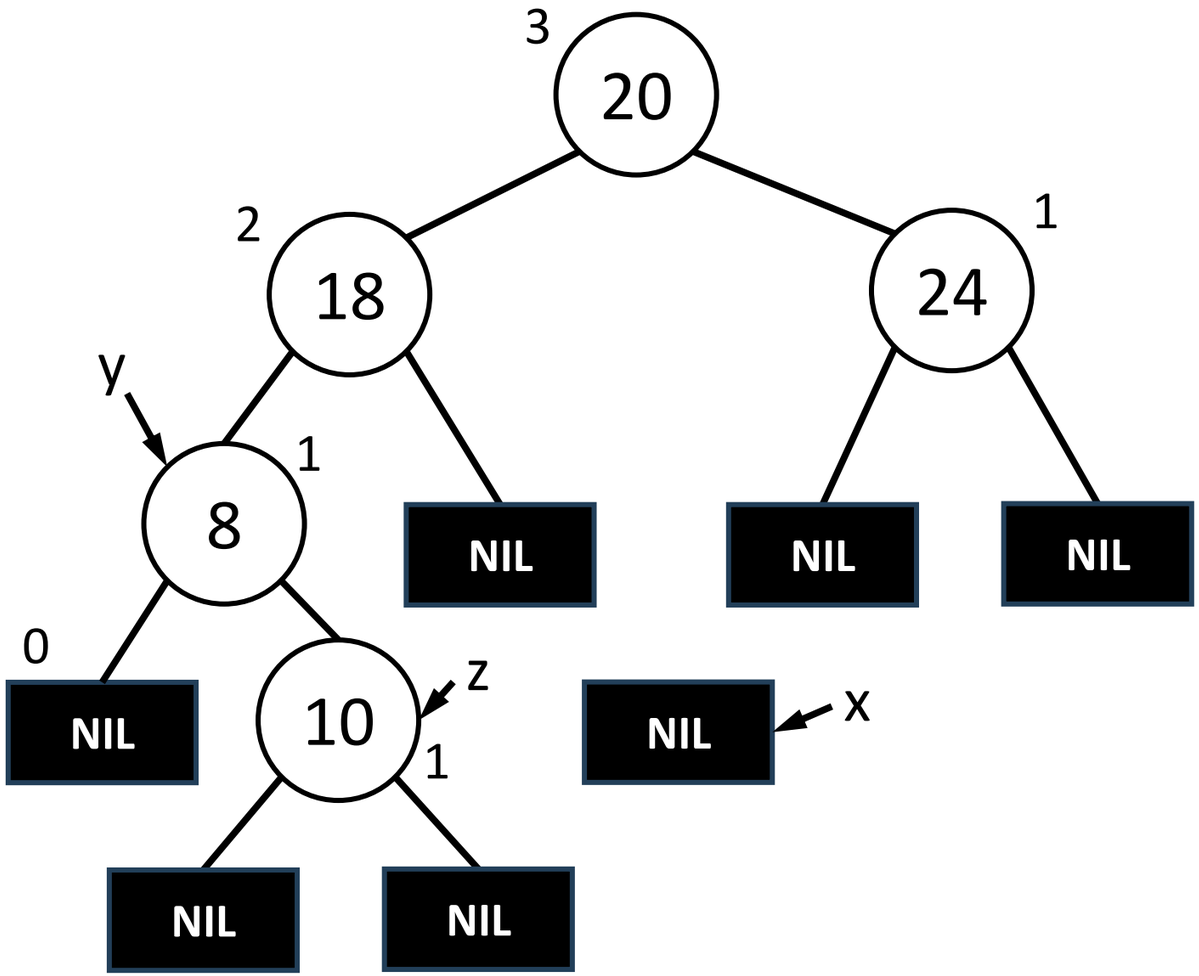
Add NIL children to z and increase z's height.



Inserting a New Node into an AVL Tree (contd.)

```
while y!= NIL
    y.h = 1 + max(y.left.h, y.right.h)
    if |y.left.h – y.right.h| == 2
        BALANCE(T, y)
    y = y.p
```

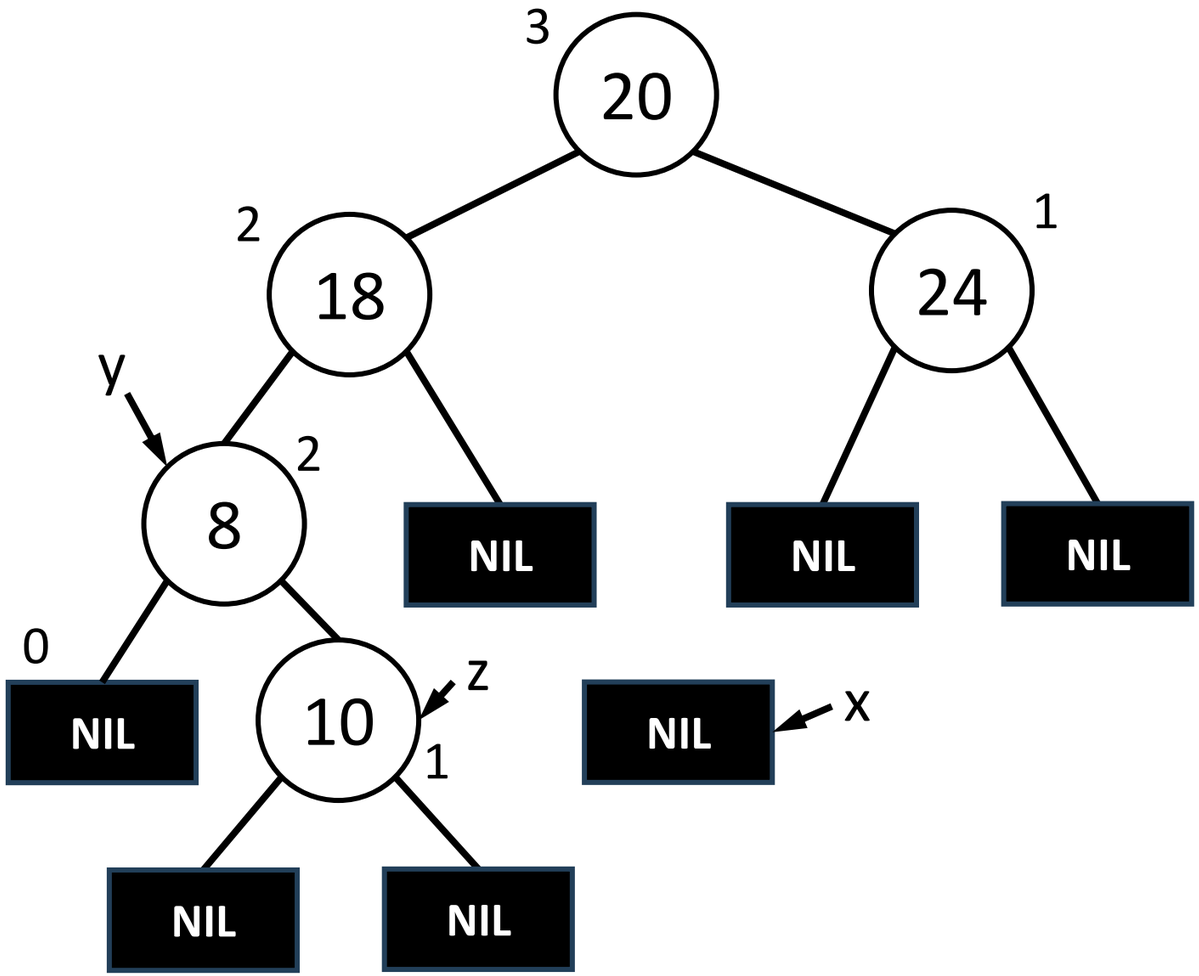
Iteratively visit all ancestors of z to check their balance.



Inserting a New Node into an AVL Tree (contd.)

```
while y!= NIL
    y.h = 1 + max(y.left.h, y.right.h)
    if |y.left.h – y.right.h| == 2
        BALANCE(T, y)
    y = y.p
```

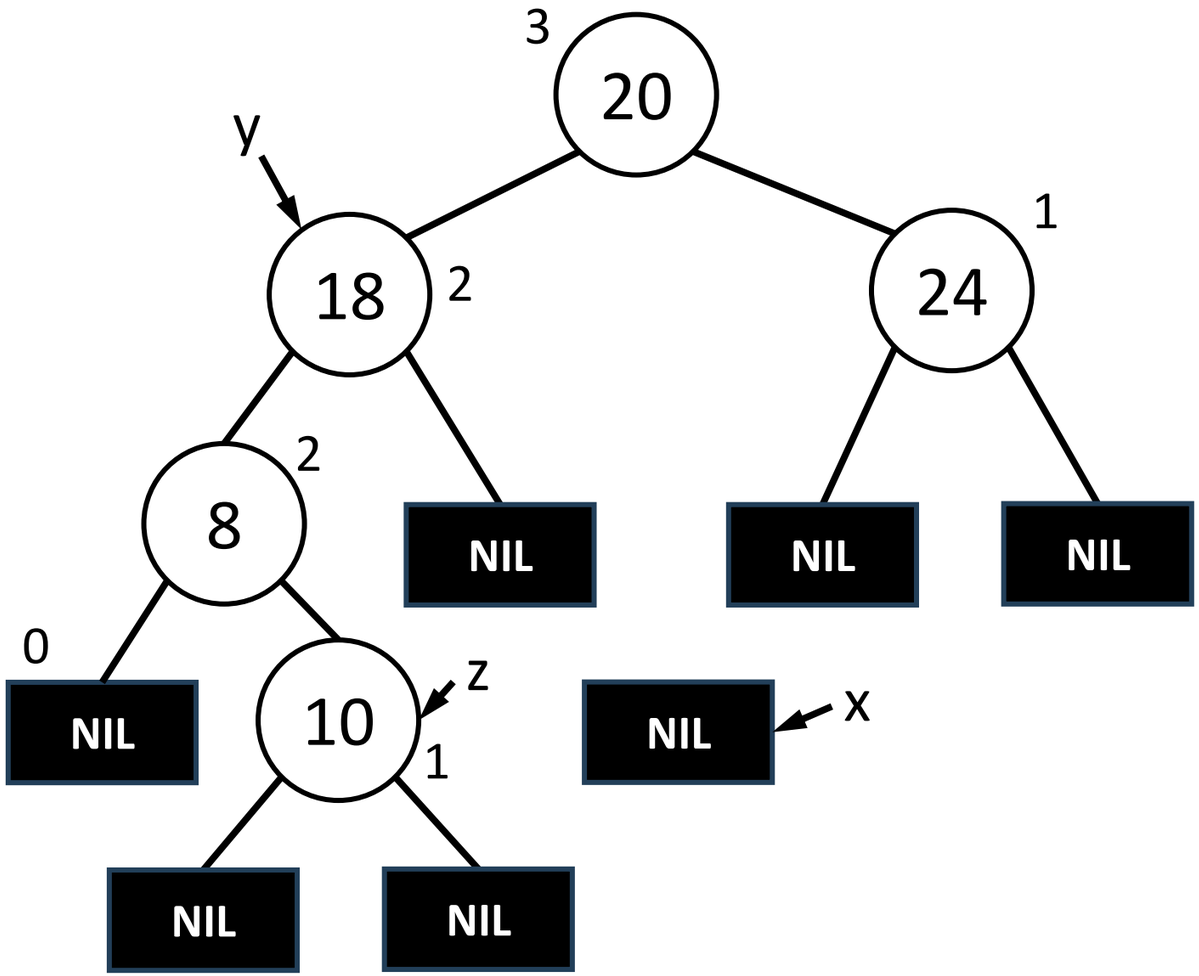
Recompute y's height.



Inserting a New Node into an AVL Tree (contd.)

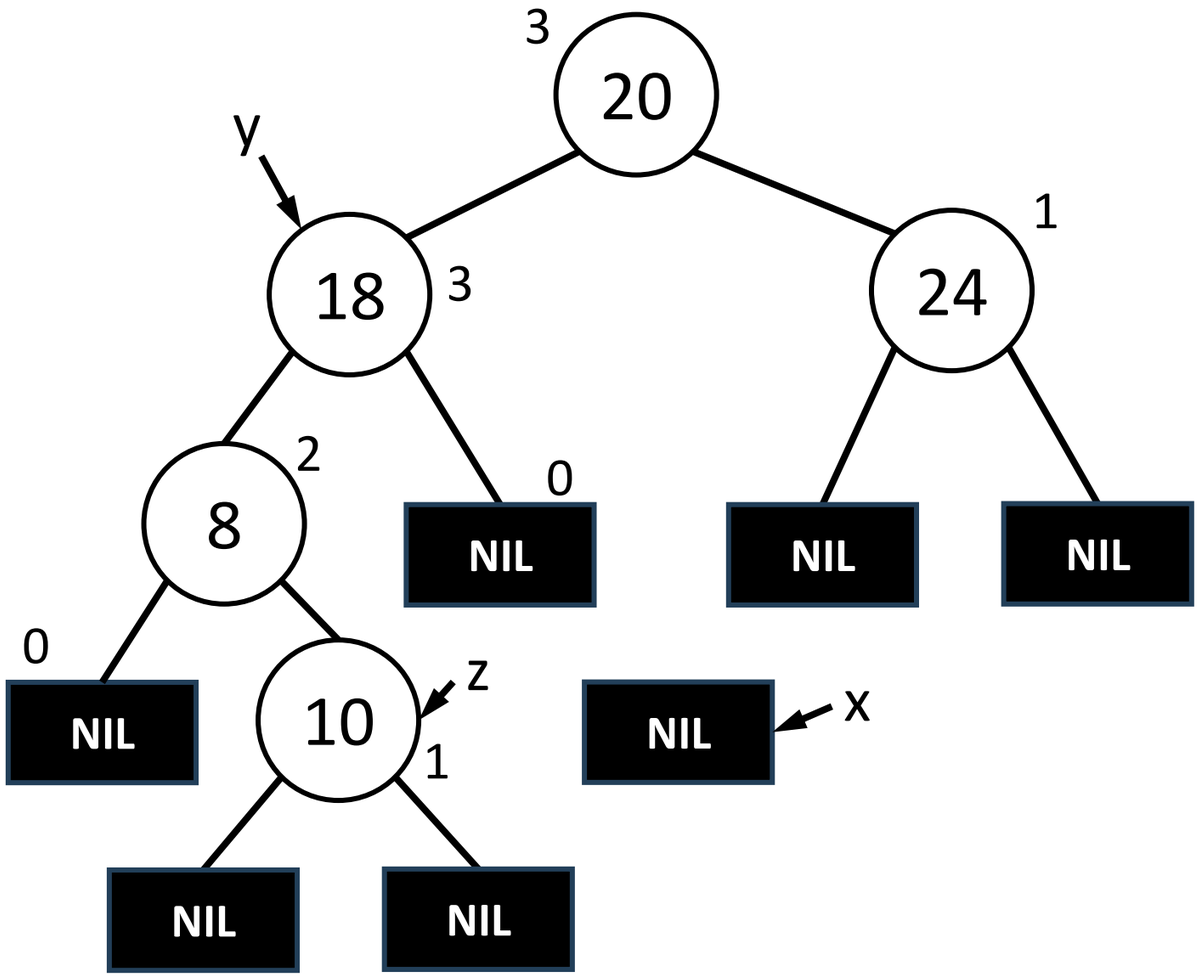
```
while y!= NIL
    y.h = 1 + max(y.left.h, y.right.h)
    if |y.left.h – y.right.h| == 2
        BALANCE(T, y)
    y = y.p
```

Since y is not unbalanced, move y to y's parent



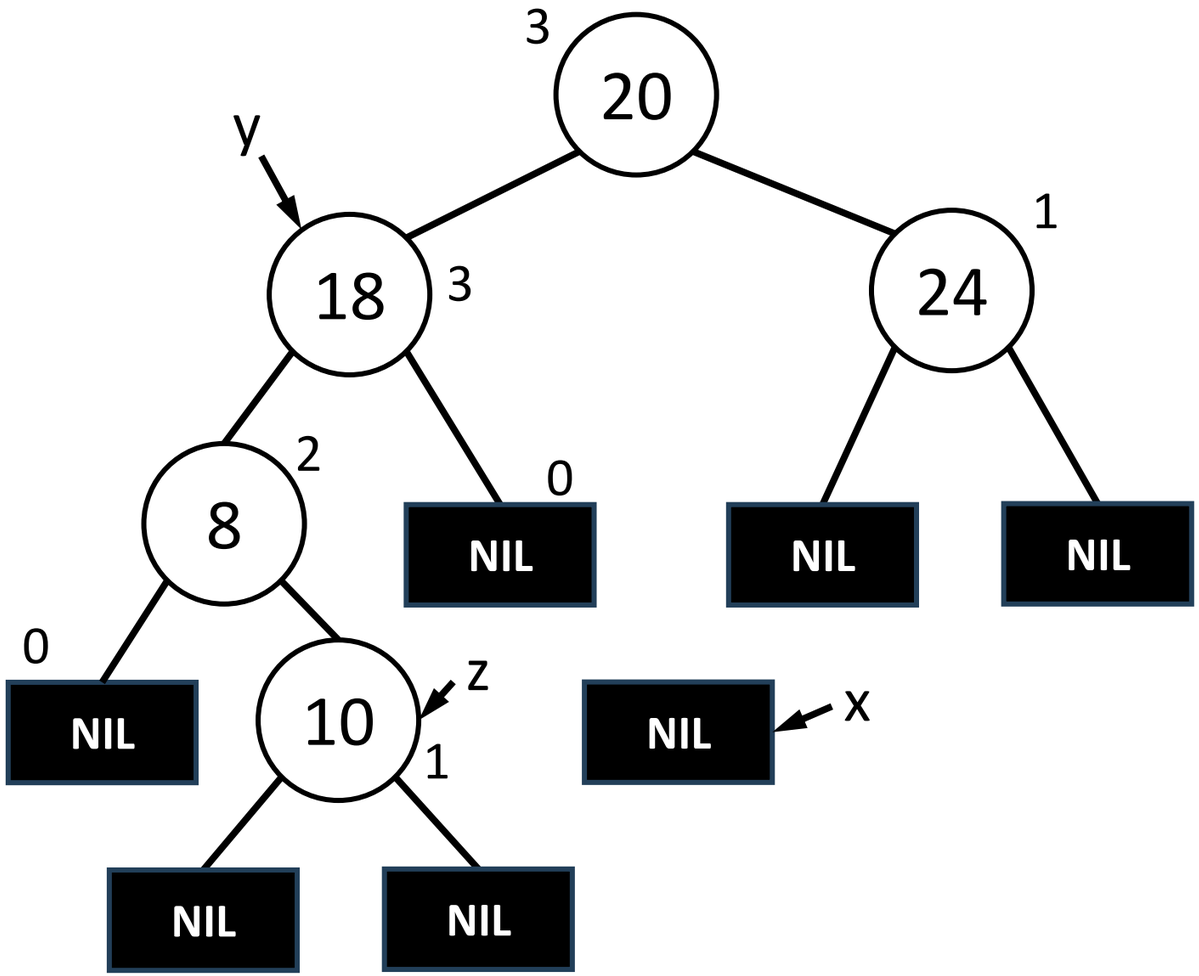
Inserting a New Node into an AVL Tree (contd.)

```
while y!= NIL
    y.h = 1 + max(y.left.h, y.right.h)
    if |y.left.h – y.right.h| == 2
        BALANCE(T, y)
    y = y.p
```



Inserting a New Node into an AVL Tree (contd.)

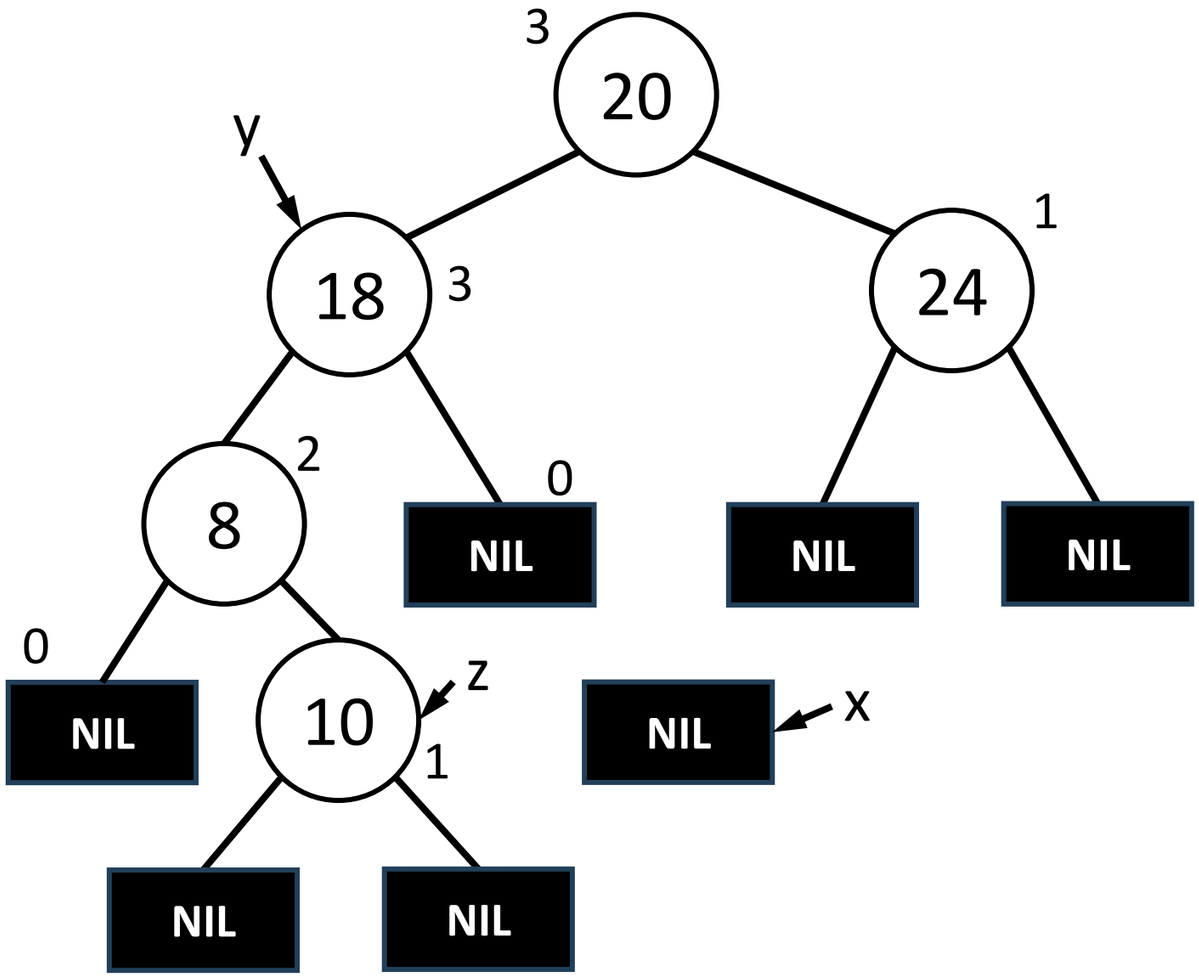
```
while y!= NIL
    y.h = 1 + max(y.left.h, y.right.h)
    if |y.left.h – y.right.h| == 2
        BALANCE(T, y, z)
    y = y.p
```



Inserting a New Node into an AVL Tree (contd.)

```
while y!= NIL
    y.h = 1 + max(y.left.h, y.right.h)
    if |y.left.h – y.right.h| == 2
        BALANCE(T, y, z)
    y = y.p
```

y is unbalanced. We need to call the BALANCE() subroutine to rebalance y.



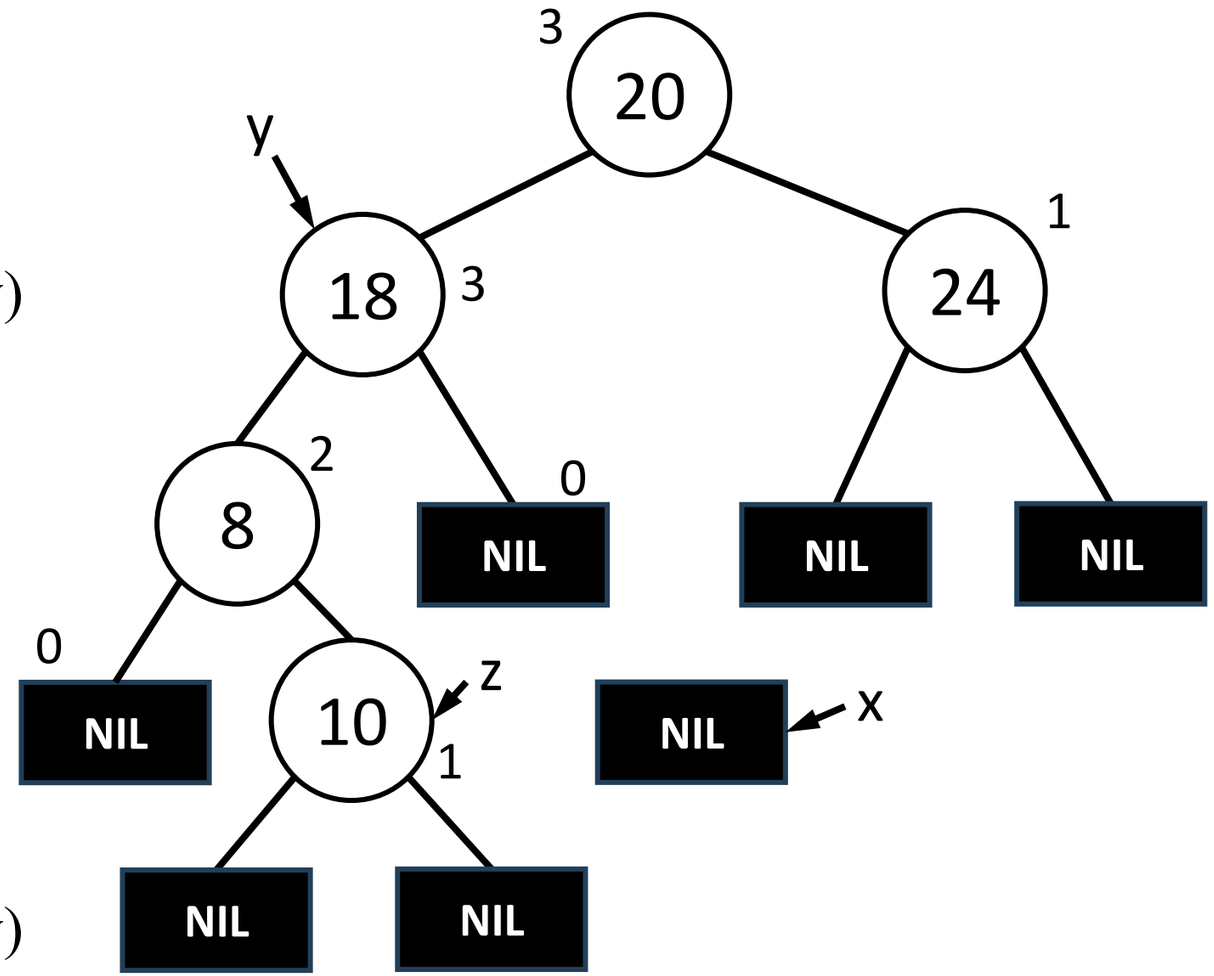
BALANCE(T, y, z) // The pseudocode of the BALANCE() subroutine

```
balanceFactor = (y.left.h – y.right.h) // Compute the balance factor of node y
\\ There could be four cases of imbalance.
\\ The left left (LL) case
if balanceFactor == 2 and (z.key < y.left.key)
    y = RIGHT-ROTATE(T, y)

\\ The right right (RR) case
if balanceFactor == -2 and (z.key > y.right.key)
    y = LEFT-ROTATE(T, y)

\\ The left right (LR) case
if balanceFactor == 2 and (z.key > y.left.key)
    y.left = LEFT-ROTATE(T, y.left)
    y = RIGHT-ROTATE(T, y)

\\ The right left (RL) case
if balanceFactor == -2 and (z.key < y.right.key)
    y.right = RIGHT-ROTATE(T, y.right)
    y = LEFT-ROTATE(T, y)
```



```
BALANCE(T, y, z) // The pseudocode of the BALANCE() subroutine
balanceFactor = (y.left.h – y.right.h) // Compute the balance factor of node y
\\ There could be four cases of imbalance.
\\ The left left (LL) case
if balanceFactor == 2 and (z.key < y.left.key)
    y = RIGHT-ROTATE(T, y)
…
```

The imbalance has been created due to the insertion of z into **y's left child's left subtree**.

BALANCE(T, y, z) // The pseudocode of the BALANCE() subroutine

```
balanceFactor = (y.left.h – y.right.h) // Compute the balance factor of node y
\\ There could be four cases of imbalance.
…
\\ The right right (RR) case
if balanceFactor == -2 and (z.key > y.right.key)
    y = LEFT-ROTATE(T, y)
…
```

The imbalance has been created due to the insertion of z into **y's right child's right subtree**.

```
BALANCE(T, y, z) // The pseudocode of the BALANCE() subroutine
balanceFactor = (y.left.h – y.right.h) // Compute the balance factor of node y
\\ There could be four cases of imbalance.
…
\\ The left right (LR) case
if balanceFactor == 2 and (z.key > y.left.key)
    y.left = LEFT-ROTATE(T, y.left)
    y = RIGHT-ROTATE(T, y)
…
```

The imbalance has been created due to the insertion of z into **y's left child's right subtree**.

BALANCE(T, y, z) // The pseudocode of the BALANCE() subroutine

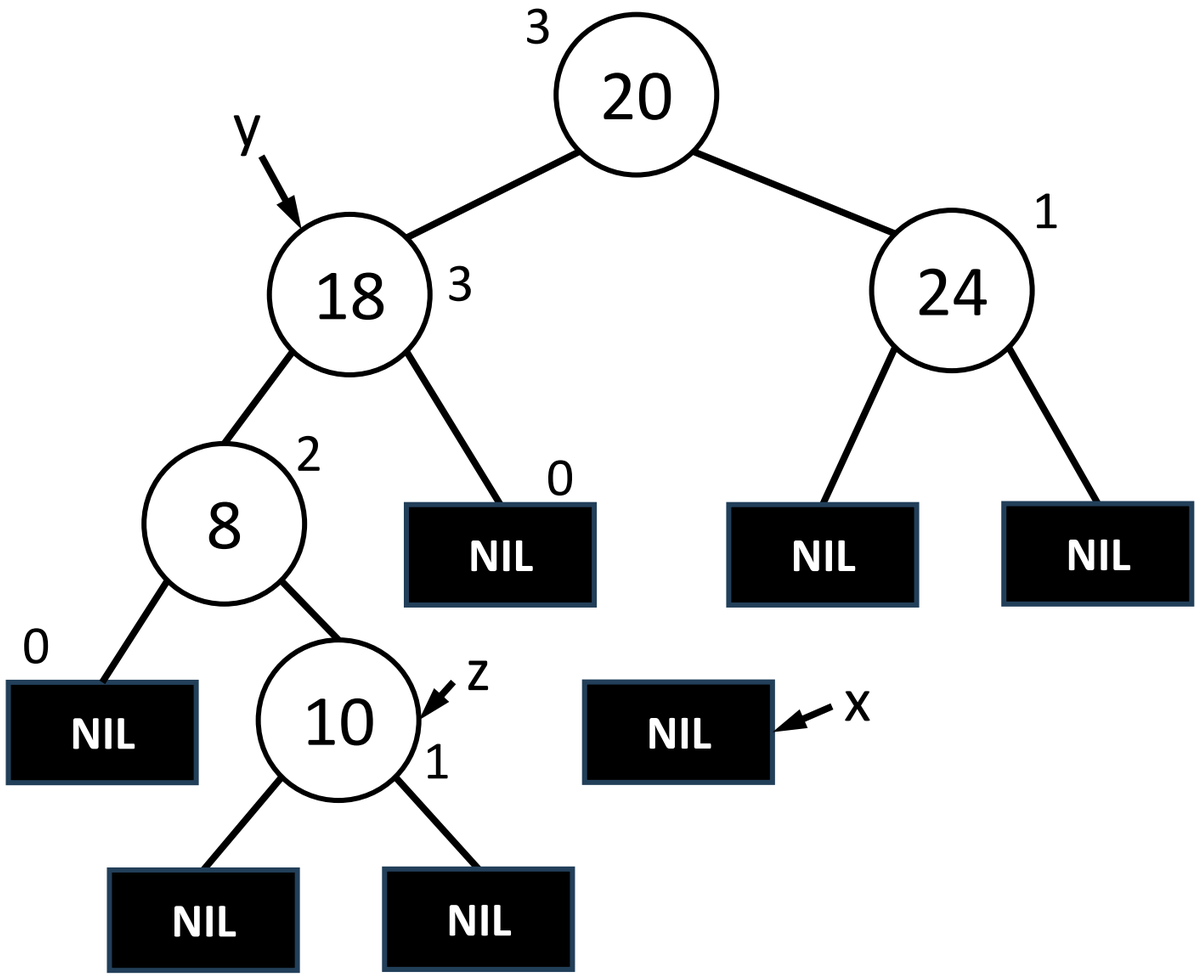
```
balanceFactor = (y.left.h – y.right.h) // Compute the balance factor of node y
\\ There could be four cases of imbalance.
…
\\ The right left (RL) case
if balanceFactor == -2 and (z.key < y.right.key)
    y.right = RIGHT-ROTATE(T, y.right)
    y = LEFT-ROTATE(T, y)
```

The imbalance has been created due to the insertion of z into **y's right child's left subtree**.

BALANCE(T, y, z) // The pseudocode of the BALANCE() subroutine

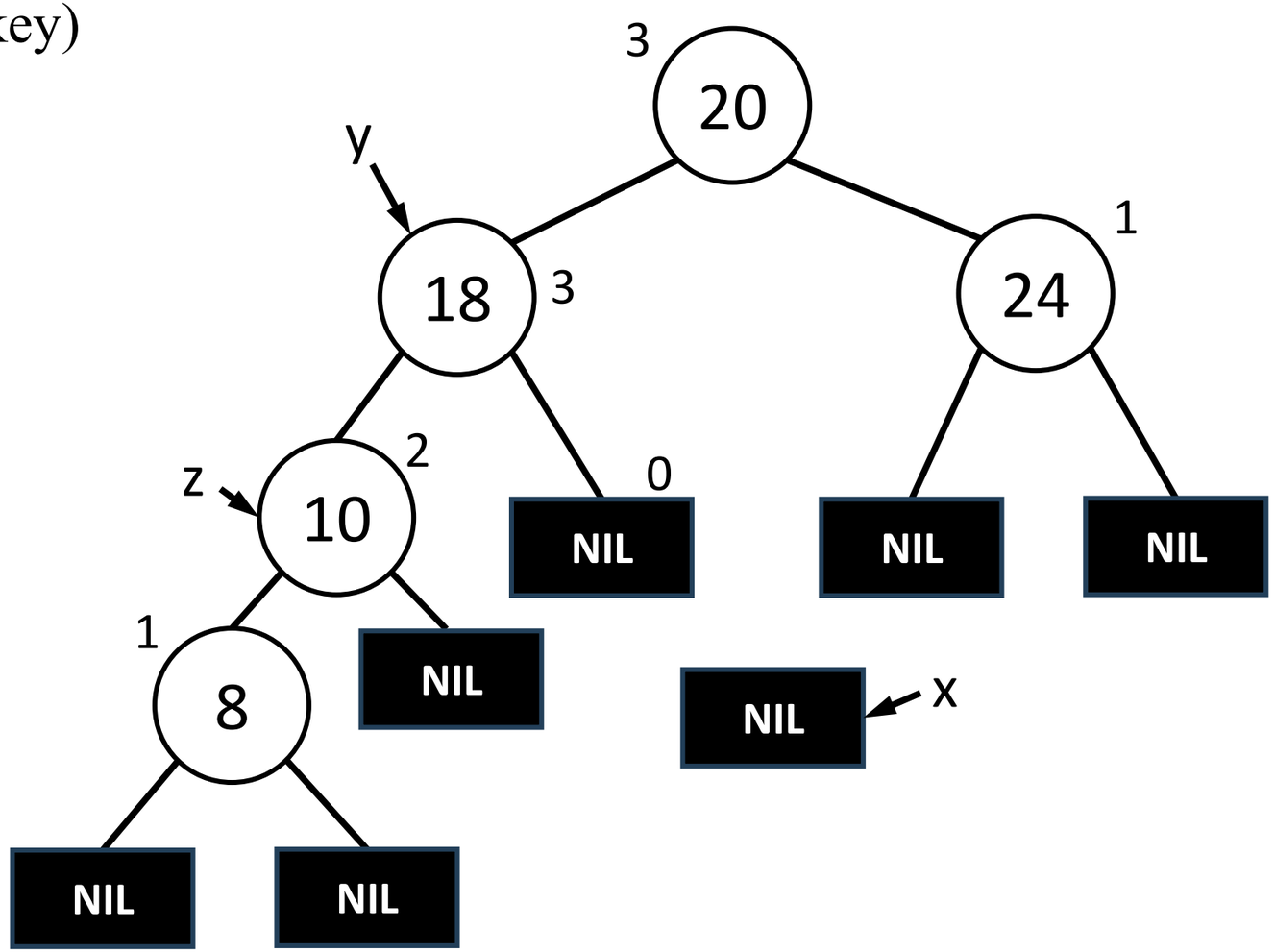
```
balanceFactor = (y.left.h – y.right.h) // Compute the balance factor of node y
\\ There could be four cases of imbalance.
…
\\ The left right (LR) case
if balanceFactor == 2 and (z.key > y.left.key)
    y.left = LEFT-ROTATE(T, y.left)
    y = RIGHT-ROTATE(T, y)
…
```

In our example, the imbalance has been created due to the insertion of z into **y's left child's right subtree a.k.a. the left right (LR) case.**



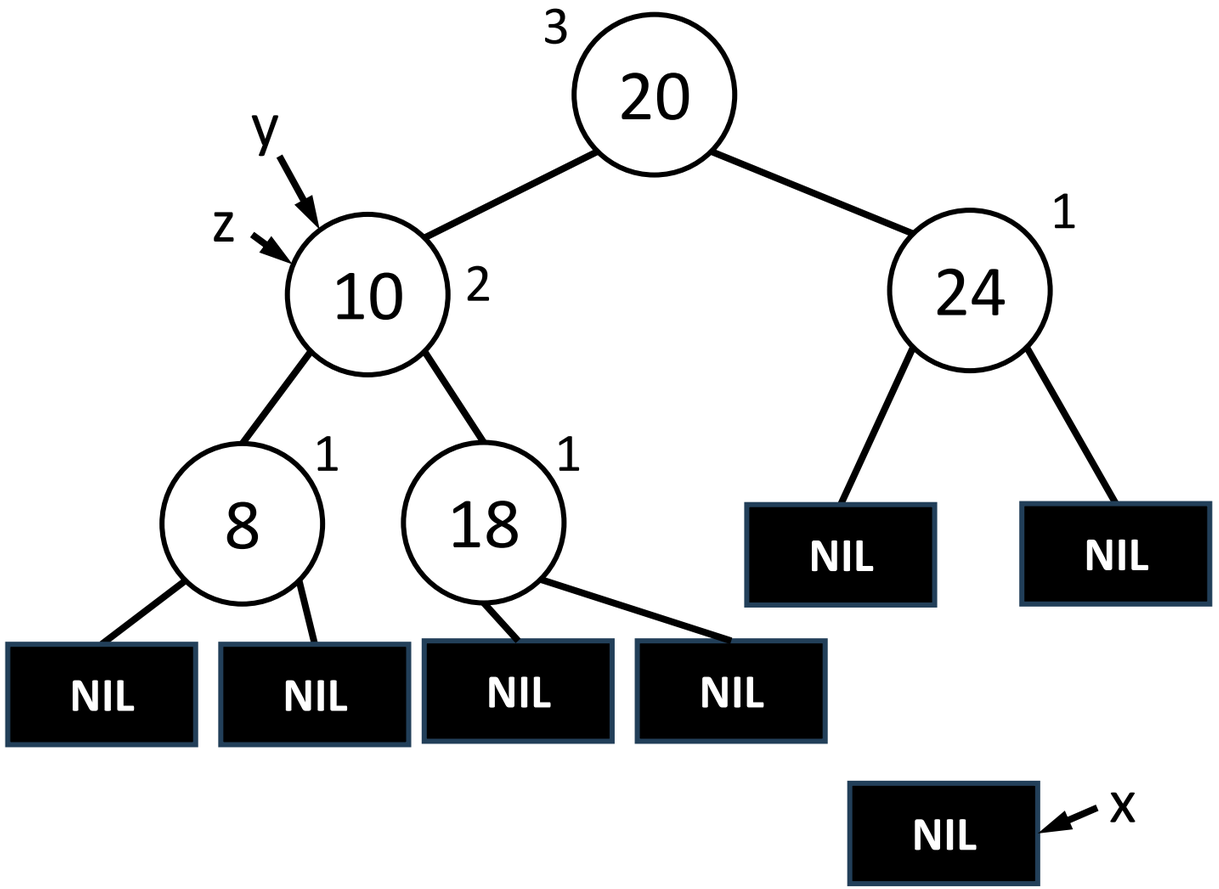
BALANCE(T, y, z)

```
balanceFactor = (y.left.h – y.right.h)
…
\\ Left right
if balanceFactor == 2 and (z.key > y.left.key)
    y.left = LEFT-ROTATE(T, y.left)
    y = RIGHT-ROTATE(T, y)
…
```



BALANCE(T, y, z)

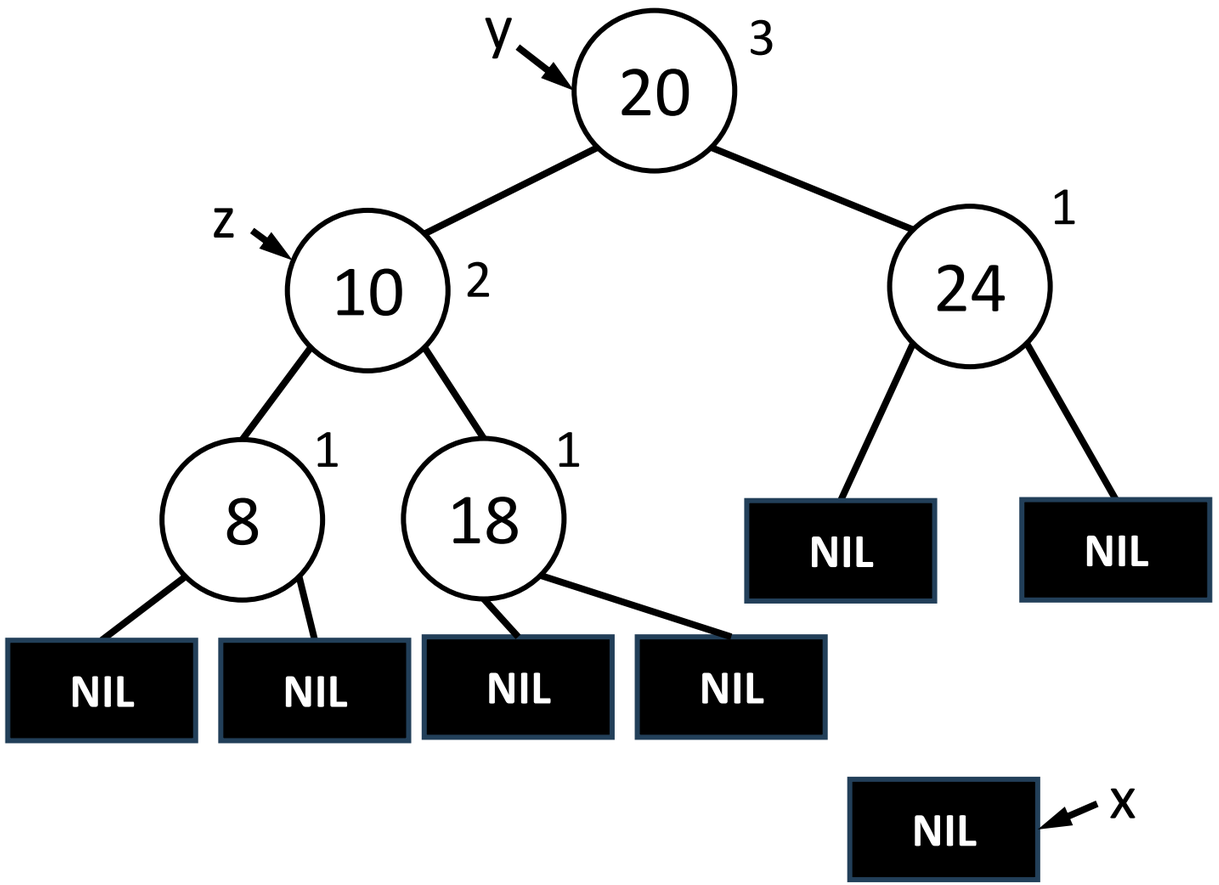
```
balanceFactor = (y.left.h – y.right.h)
…
\\ Left right
if balanceFactor == 2 and (z.key > y.left.key)
    y.left = LEFT-ROTATE(T, y.left)
    y = RIGHT-ROTATE(T, y)
…
```



Inserting a New Node into an AVL Tree (contd.)

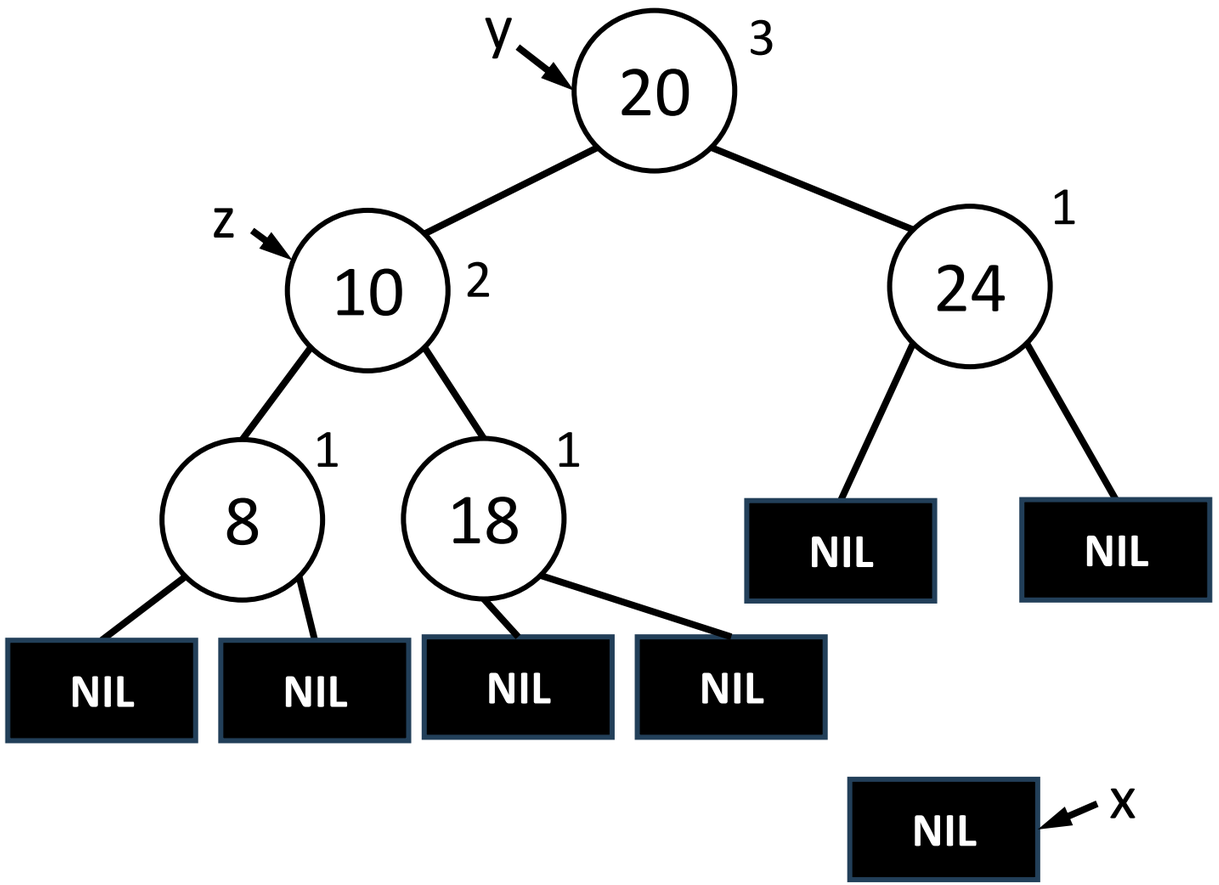
```
while y!= NIL
    y.h = 1 + max(y.left.h, y.right.h)
    if |y.left.h – y.right.h| == 2
        BALANCE(T, y, z)
    y = y.p
```

y is moved to the next ancestor to check its balance.



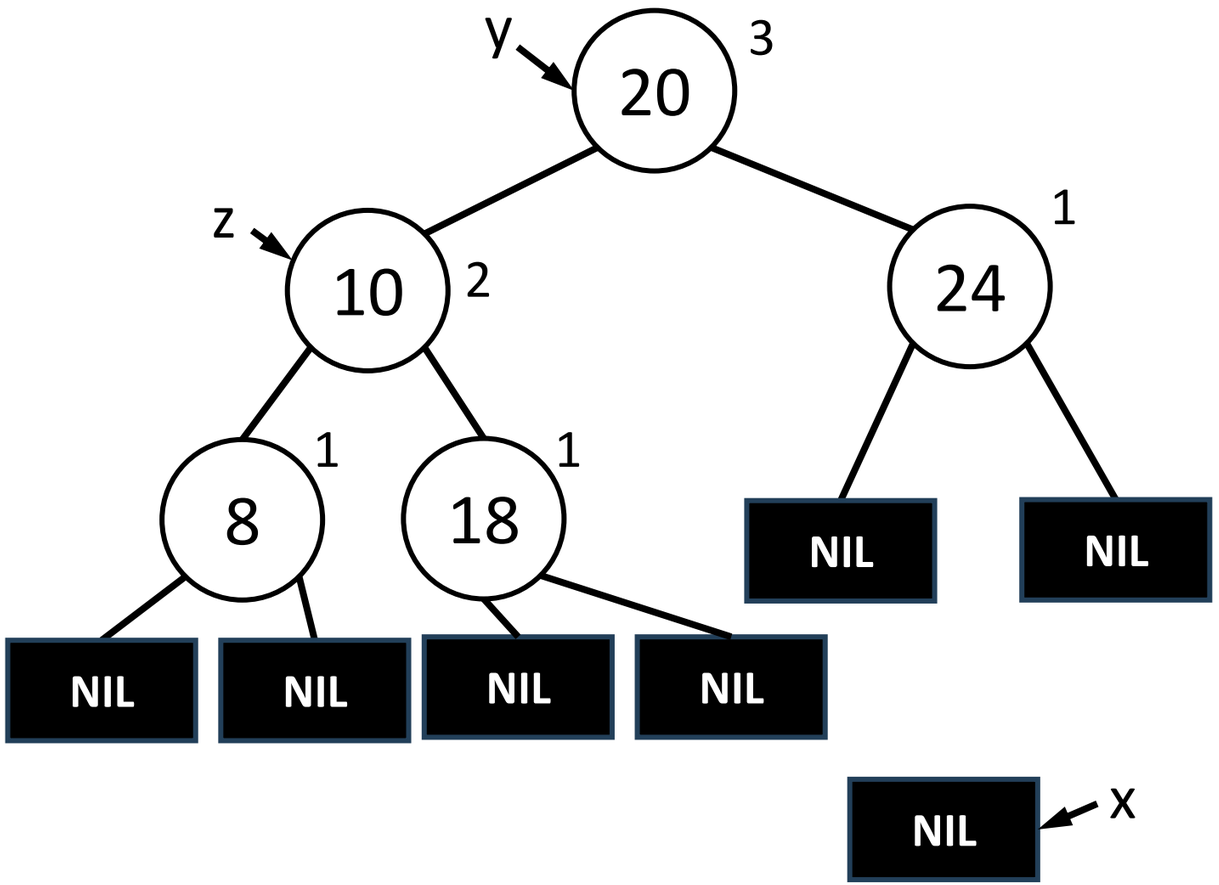
Inserting a New Node into an AVL Tree (contd.)

```
while y!= NIL
    y.h = 1 + max(y.left.h, y.right.h)
    if |y.left.h – y.right.h| == 2
        BALANCE(T, y, z)
    y = y.p
```



Inserting a New Node into an AVL Tree (contd.)

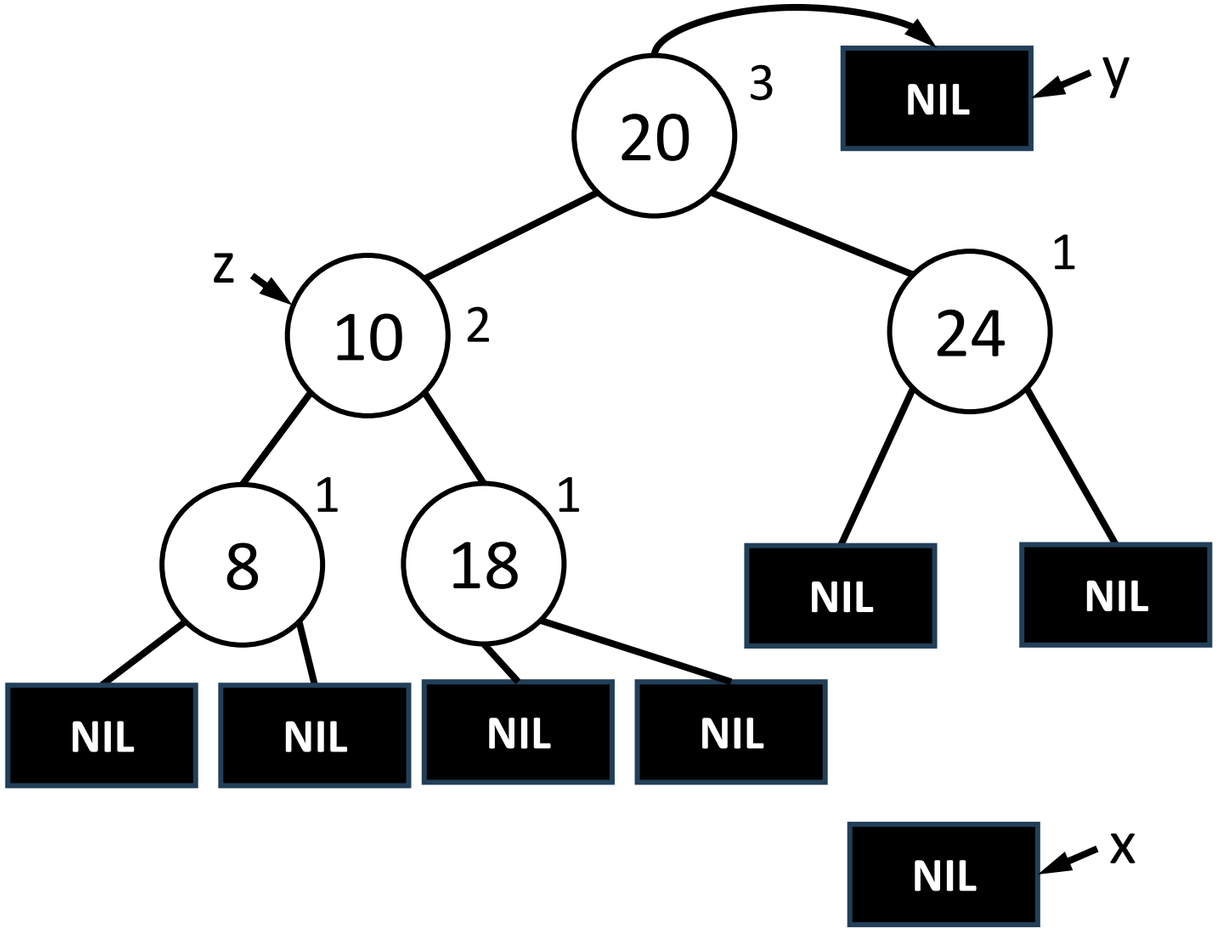
```
while y!= NIL
    y.h = 1 + max(y.left.h, y.right.h)
    if |y.left.h – y.right.h| == 2
        BALANCE(T, y, z)
    y = y.p
```



Inserting a New Node into an AVL Tree (contd.)

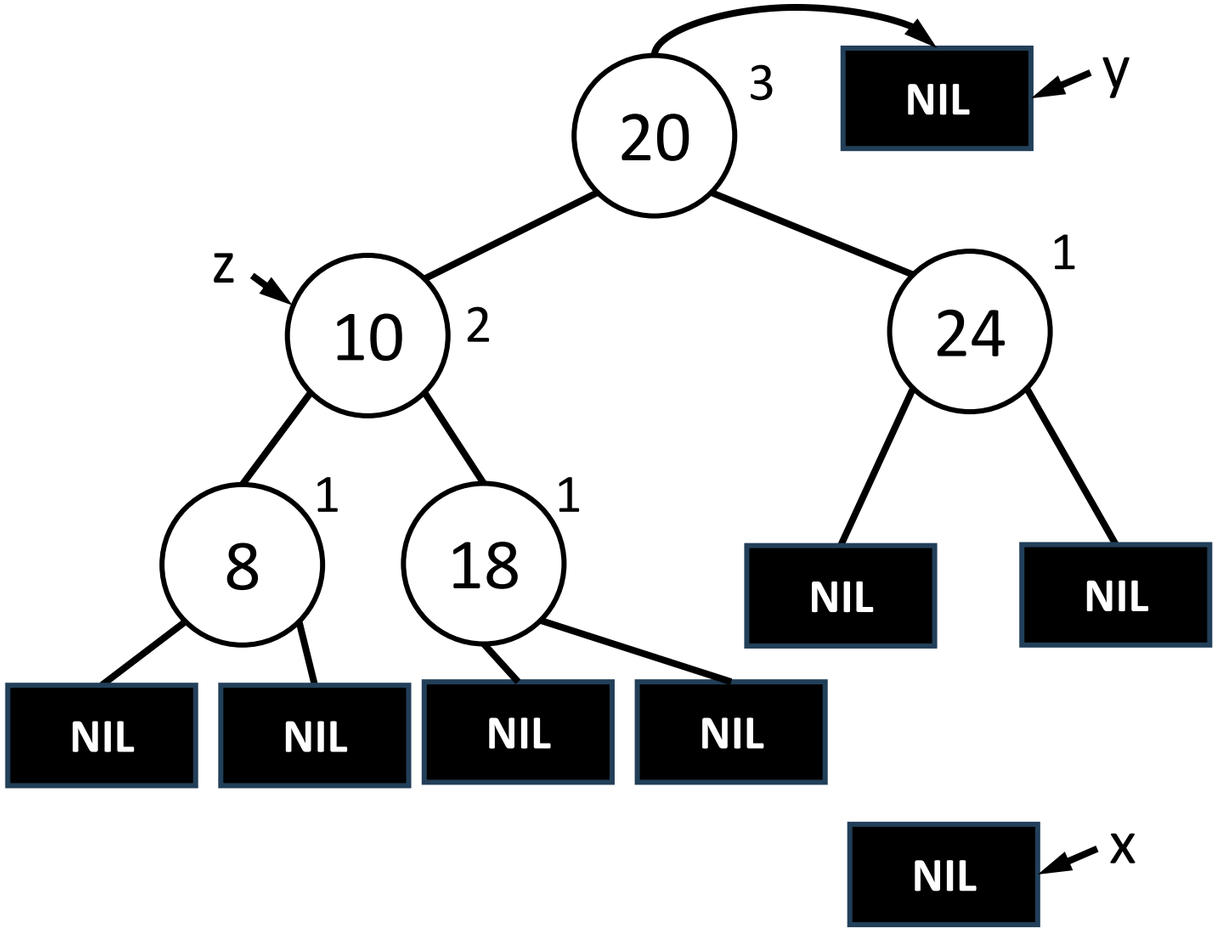
```
while y!= NIL
    y.h = 1 + max(y.left.h, y.right.h)
    if |y.left.h – y.right.h| == 2
        BALANCE(T, y, z)
    y = y.p
```

Since y is NIL, the while loop terminates and so does the insertion pseudocode.



Inserting a New Node into an AVL Tree (contd.)

The output after insertion is a valid AVL tree since
| x.left.h – x.right.h| ≤ 1 for every node x.



Inserting a New Node into an AVL Tree: Recap

Insert the new node z into tree T as we would do for any binary search tree. Add two NIL children to z and set z's height to 1.

For each ancestor y of z

- Recompute y's height.
- Check if y has become unbalanced after inserting z.
  If unbalanced, there could be **four cases** for which y has become unbalanced.
  - **The Left Left (LL) Case:** The imbalance has been created due to the insertion of z into y's left child's left subtree. **Perform a right rotation** to handle this case.
  - **The Right Right (RR) Case:** The imbalance has been created due to the insertion of z into y's right child's right subtree. **Perform a left rotation** to handle this case.
  - **The Left Right (LR) Case:** The imbalance has been created due to the insertion of z into y's left child's right subtree. **Perform a left rotation followed by a right rotation** to handle this case.
  - **The Right Left (RL) Case:** The imbalance has been created due to the insertion of z into y's right child's left subtree. **Perform a right rotation followed by a left rotation** to handle this case.

Deleting a Node from an AVL Tree

Input Parameters:

T = The given tree.

z = The node to be deleted.

Find z and delete it.

Section 6.14.4 ‘Deletion in AVL tree’, Srivastava and Srivastava, 2nd Edition

Deleting a Node from an AVL Tree (contd.)

For every node x starting from the ex-parent of z till T.root

```
    If x is balanced
        Update x.h
    Else if x is unbalanced by 1
        Update x.h
        Break
    Else if x is unbalanced by 2
        Perform rotations to restore the AVL property
```

Section 6.14.4 ‘Deletion in AVL tree’, Srivastava and Srivastava, 2nd Edition

The Red-Black Tree

The Structure of a Red-Black Tree Node

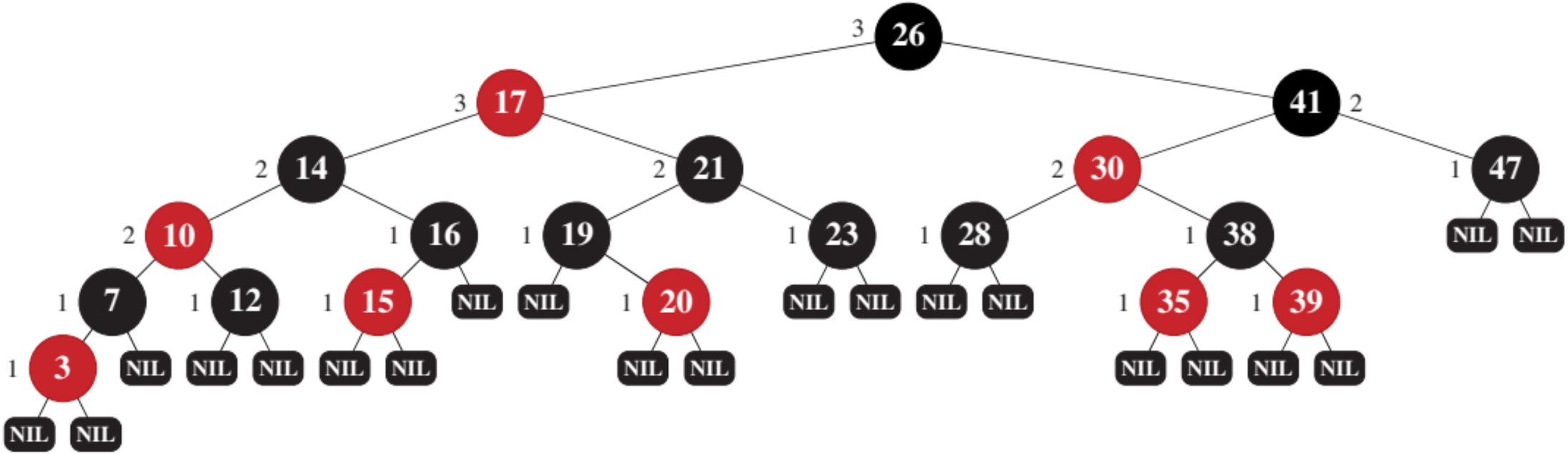
key	left (a pointer to the left child)	right (a pointer to the right child)	p (a pointer to the parent node. For the root node, it is NULL.)	color (either red or black)	Satellite data (optional)	More attributes (optional)

The Red-Black Tree

A red-black tree is a binary search tree that satisfies **the red-black properties** as listed below.

1. Every node is either red or black.
2. The root node is black.
3. Every **leaf node (NIL)** is black.
4. If a node is red, then both its children are black.
5. For each node, all **simple paths** from the node to descendant leaves contain the same number of black nodes.

An Example of a Red-Black Tree



Each non-leaf a.k.a. ‘internal’ node x is marked with its **black height:**
bh(x) = The number of black nodes on any simple path from node x (but excluding node x) to a descendant leaf.

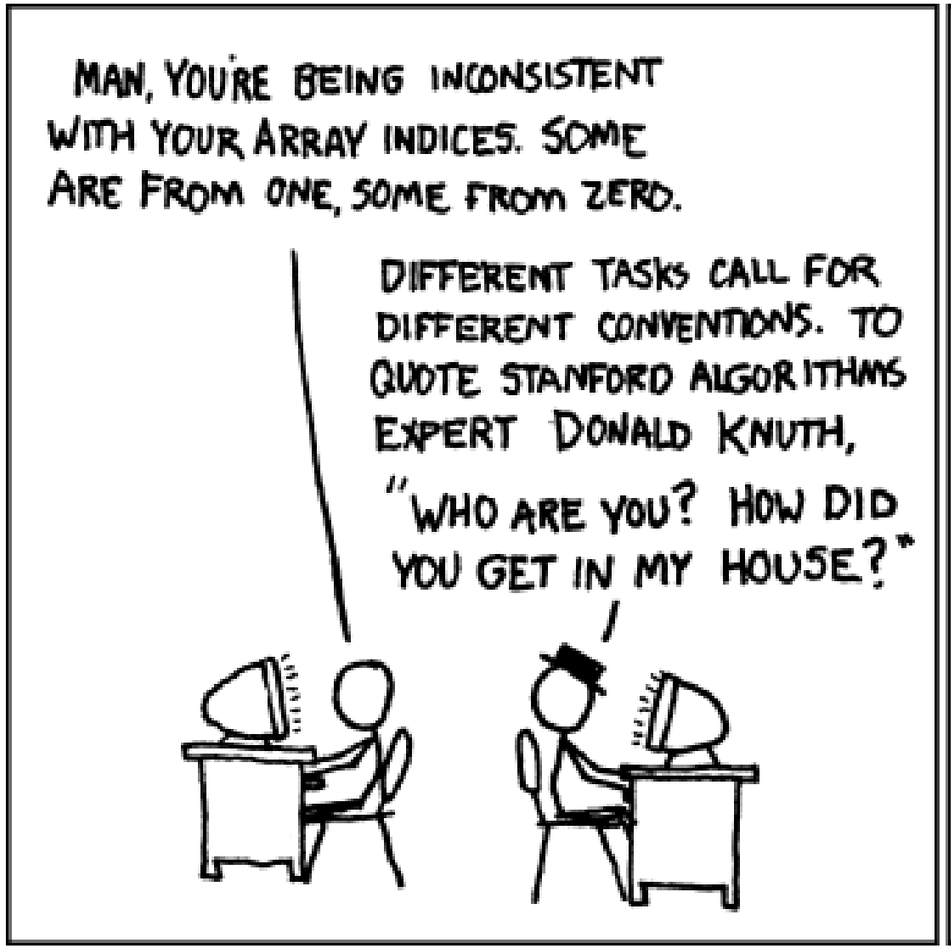
Figure 13.1(a), Chapter 13, CLRS 4th Edition

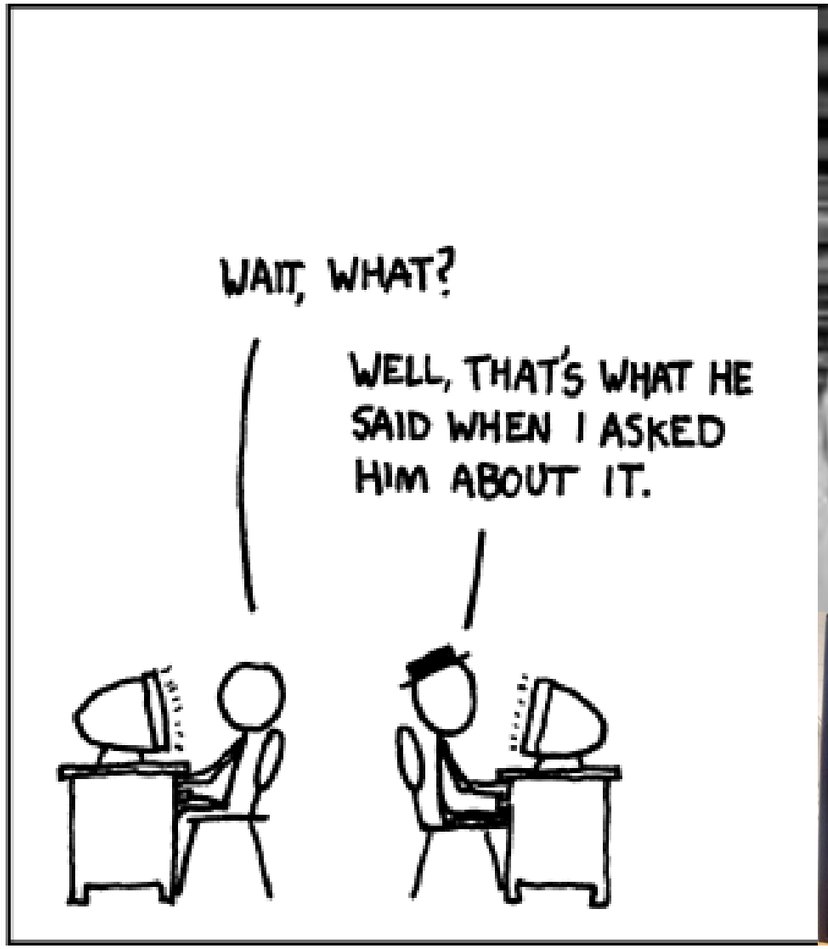
The Black Height of a Red-Black Tree T

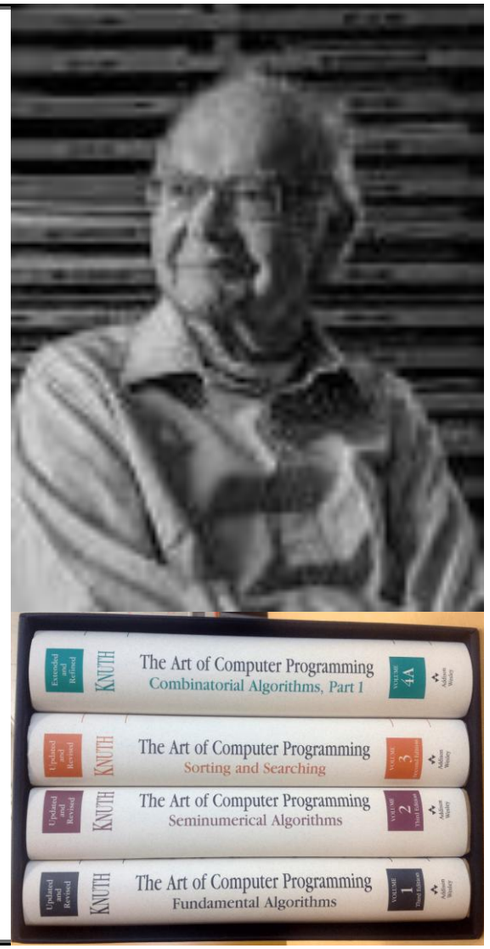
bh(T)

= bh(T.root)

Why are leaves NIL?







Comics: https://xkcd.com/163 , Donald Knuth's photo: https://www.nytimes.com/2018/12/17/science/donald-knuth-computers-algorithms-programming.html , TAOCP volume's photo: https://www.preining.info/blog/2014/05/knuths-taocp-hardcover-ebook/

What is a Simple Path?

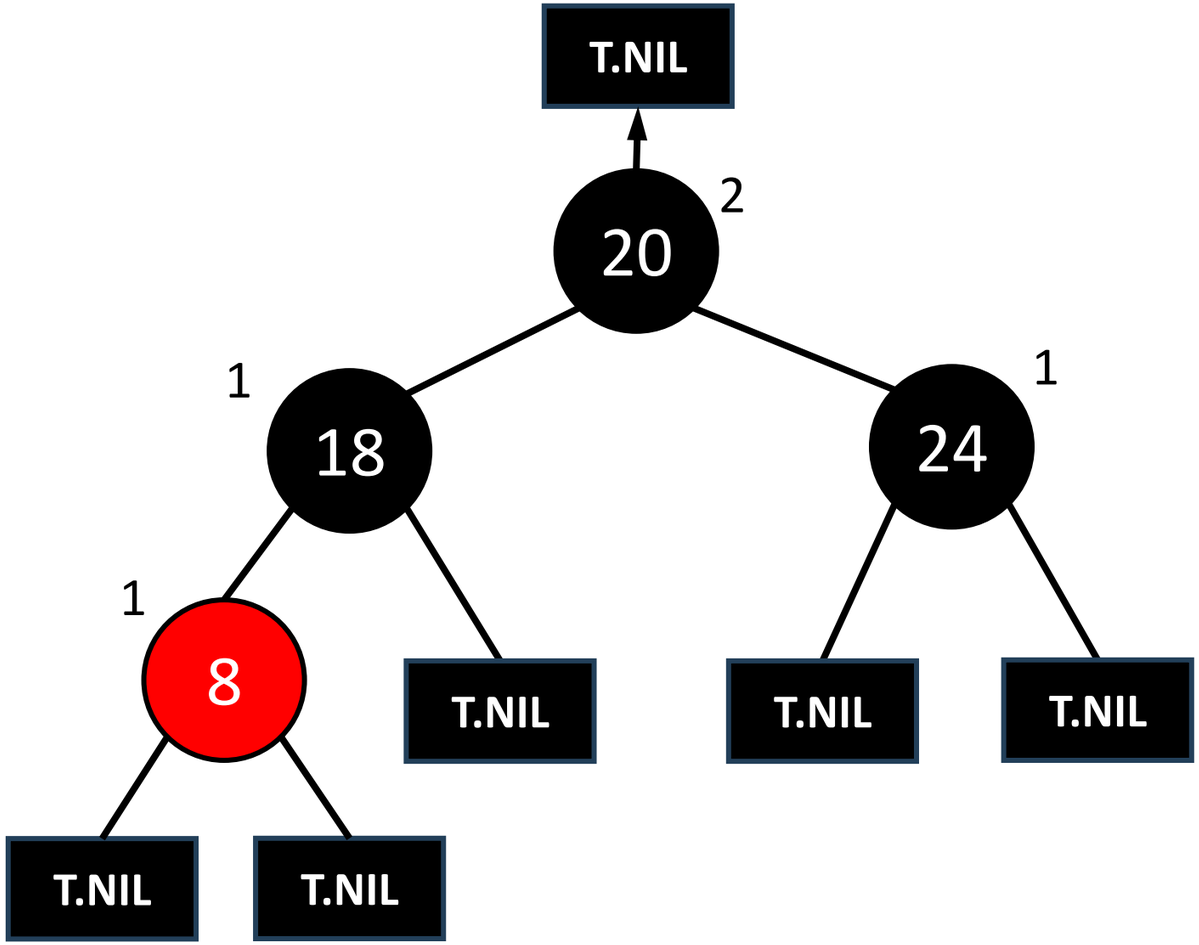
In graph theory, a path is **simple** if no vertices are repeated along the path.

If we allow vertices to repeat along a path, it is called a **walk**.

Inserting a New Node into a Red-Black Tree

T = The given tree

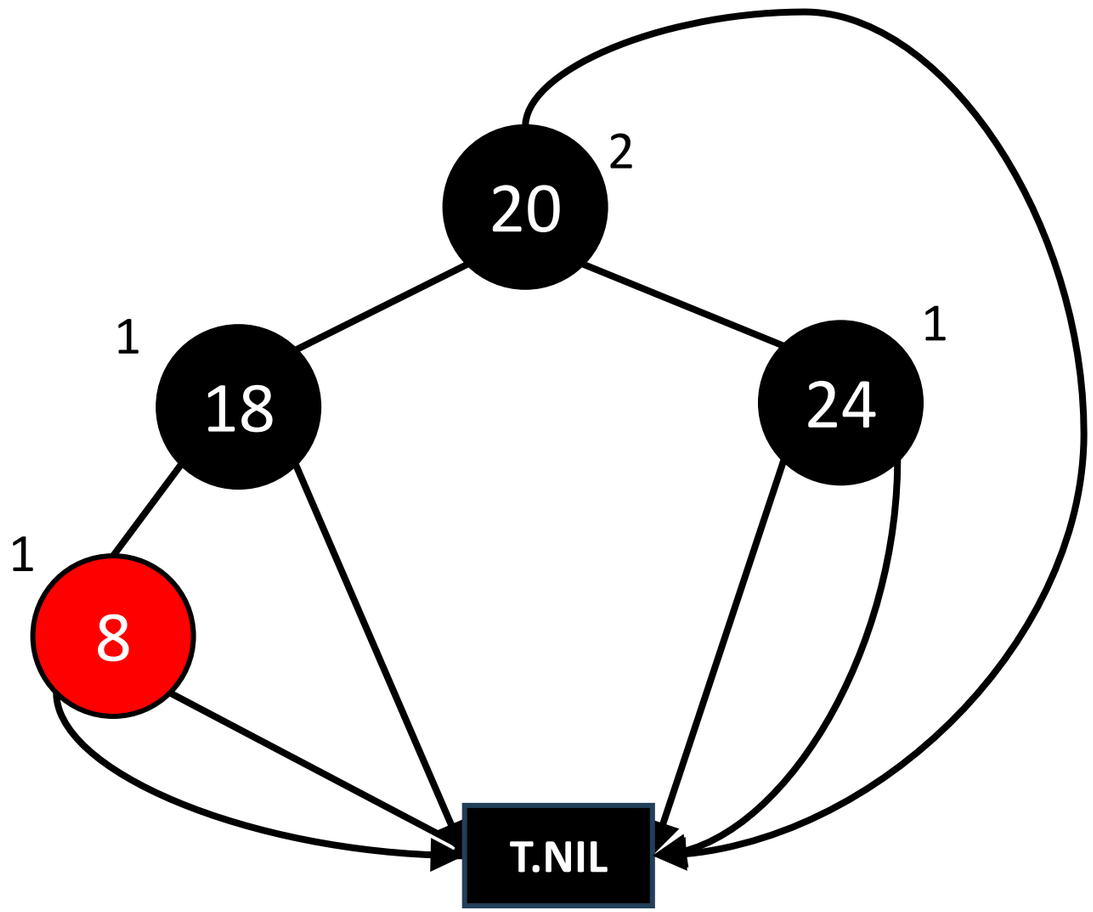
1. Every node is either red or black.
2. The root node is black.
3. Every **leaf node (NIL)** is black.
4. If a node is red, then both its children are black.
5. For each node, all **simple paths** from the node to descendant leaves contain the same number of black nodes.



Inserting a New Node into a Red-Black Tree

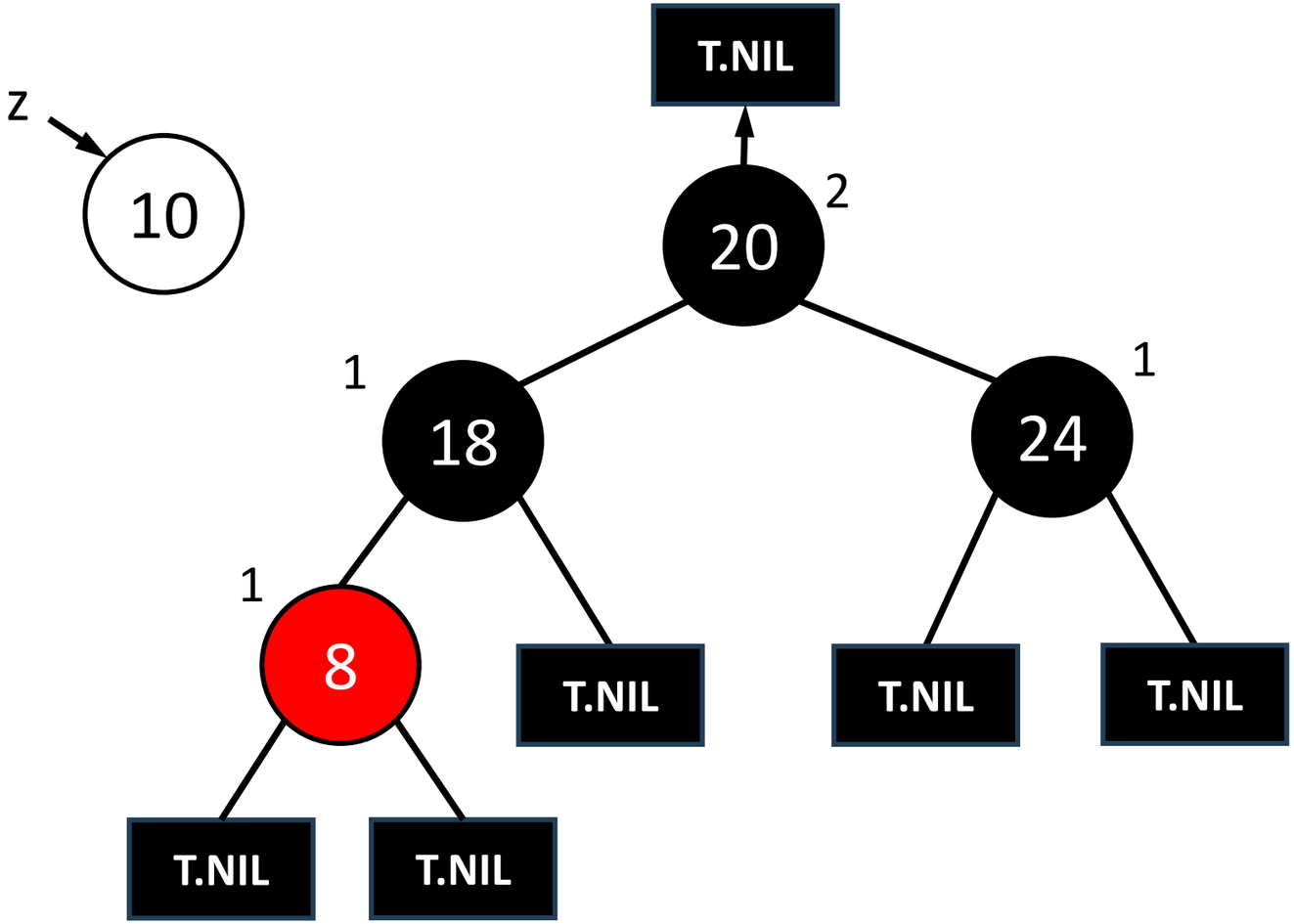
To save space, a single node can be used as T.NIL in practice.

The node's colour should be black but its other attributes can have arbitrary values since they will never be used.



Inserting a New Node into a Red-Black Tree (contd.)

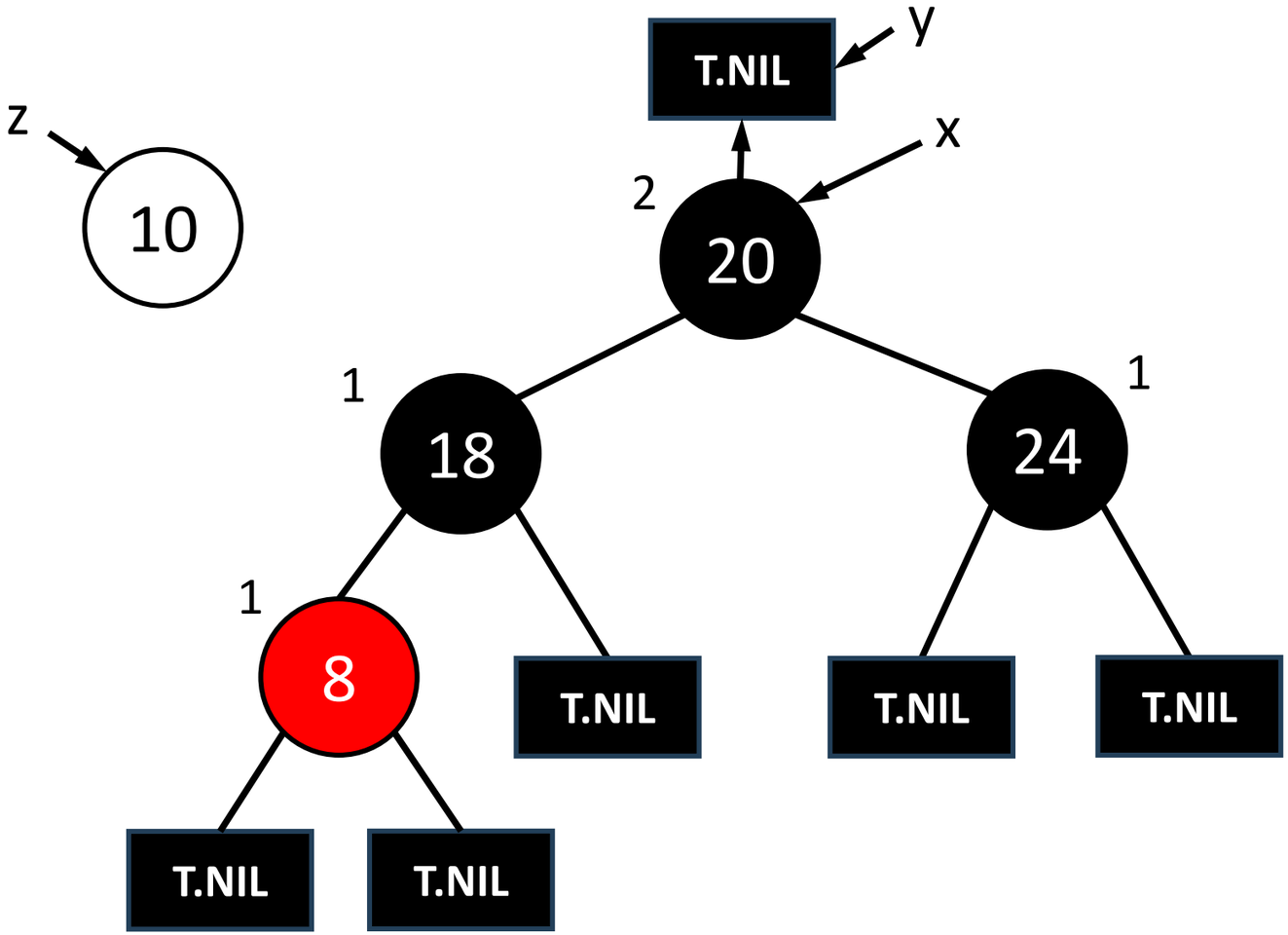
T = The given tree
z = The new node



Inserting a New Node into a Red-Black Tree (contd.)

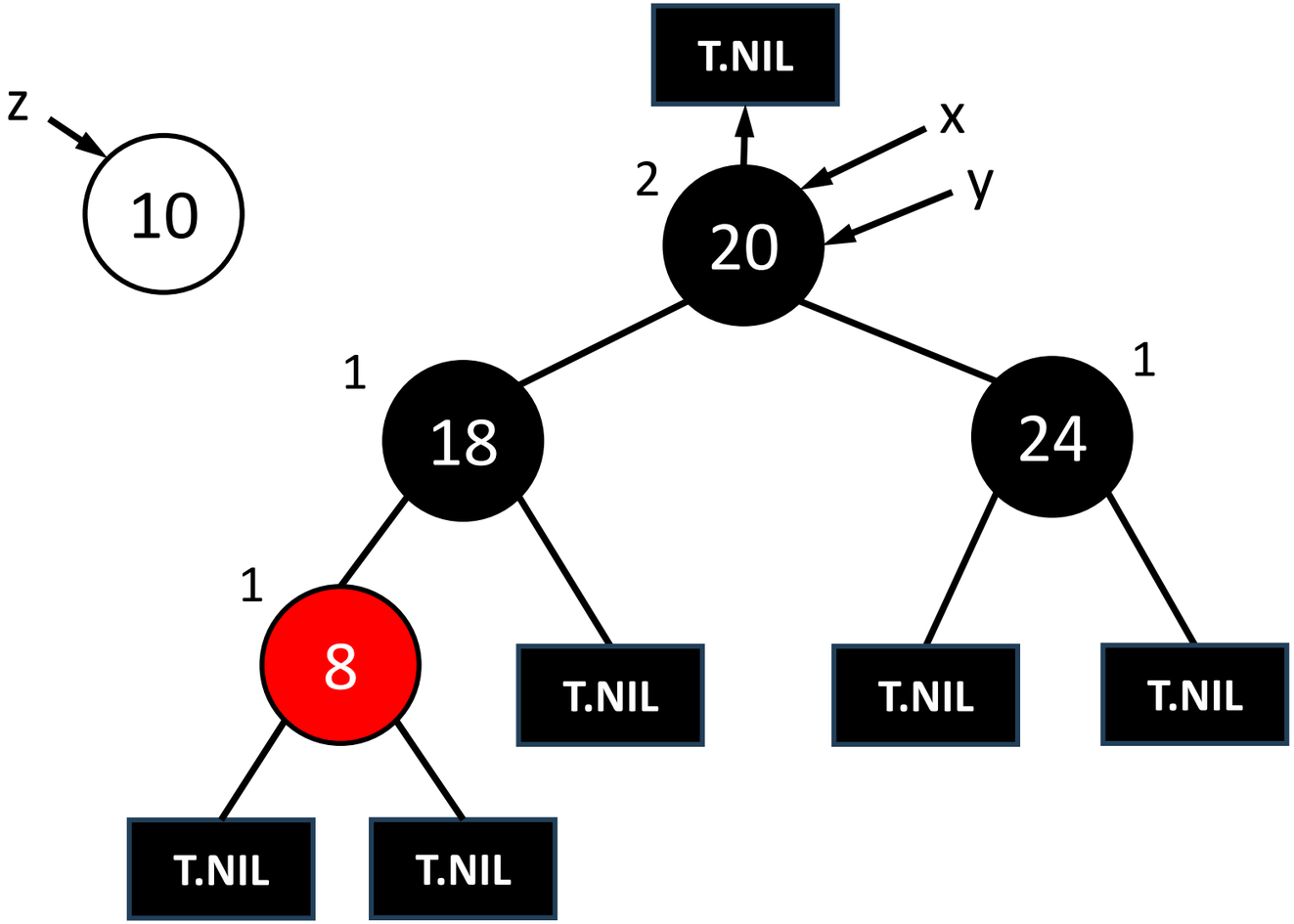
T = The given tree
z = The new node

RB-INSERT(T, z)
y = T.nil
x = T.root



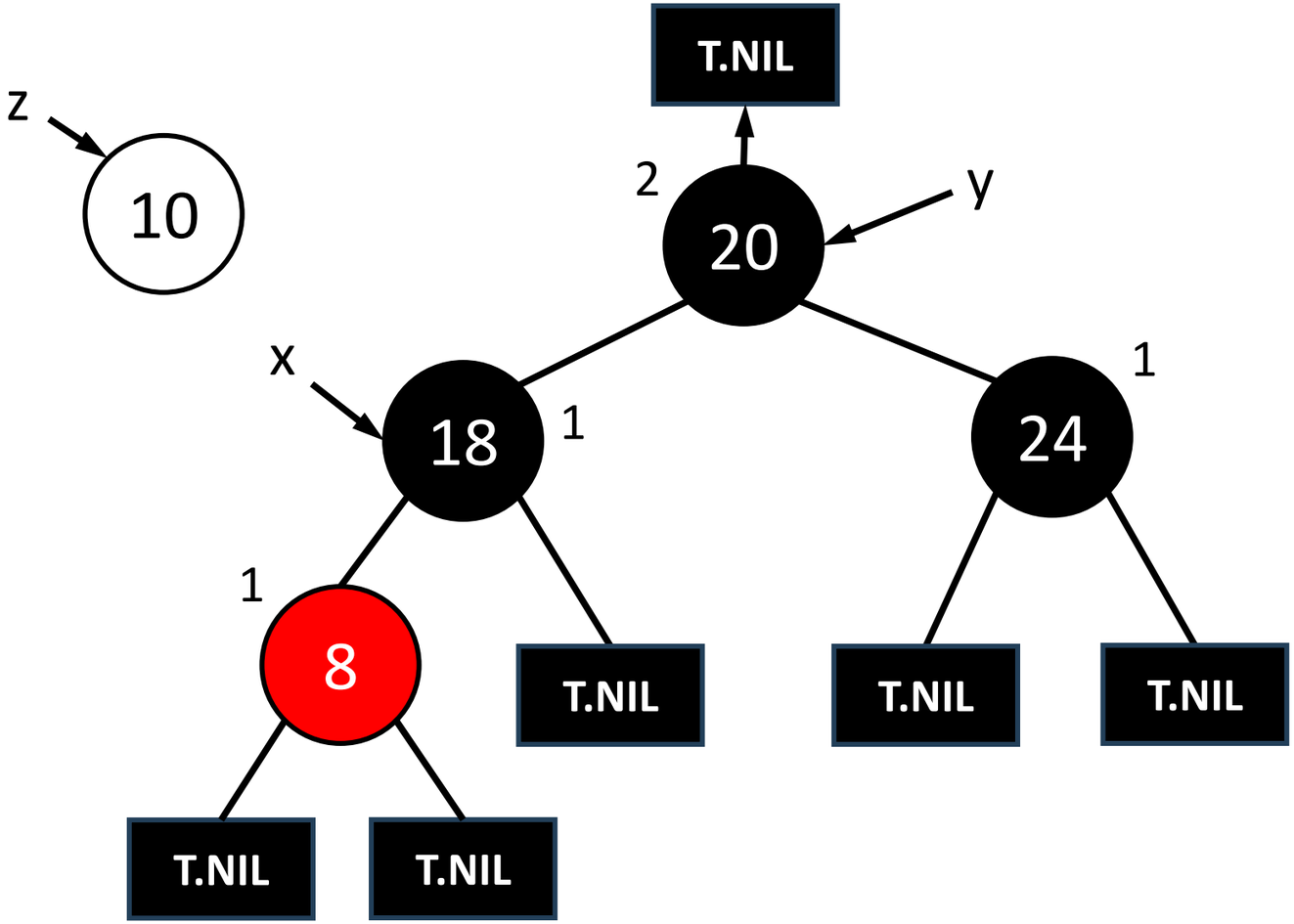
Inserting a New Node into a Red-Black Tree (contd.)

```
while x != T.nil
    y = x
    if z.key < x.key
      x = x.left
    else
      x = x.right
```



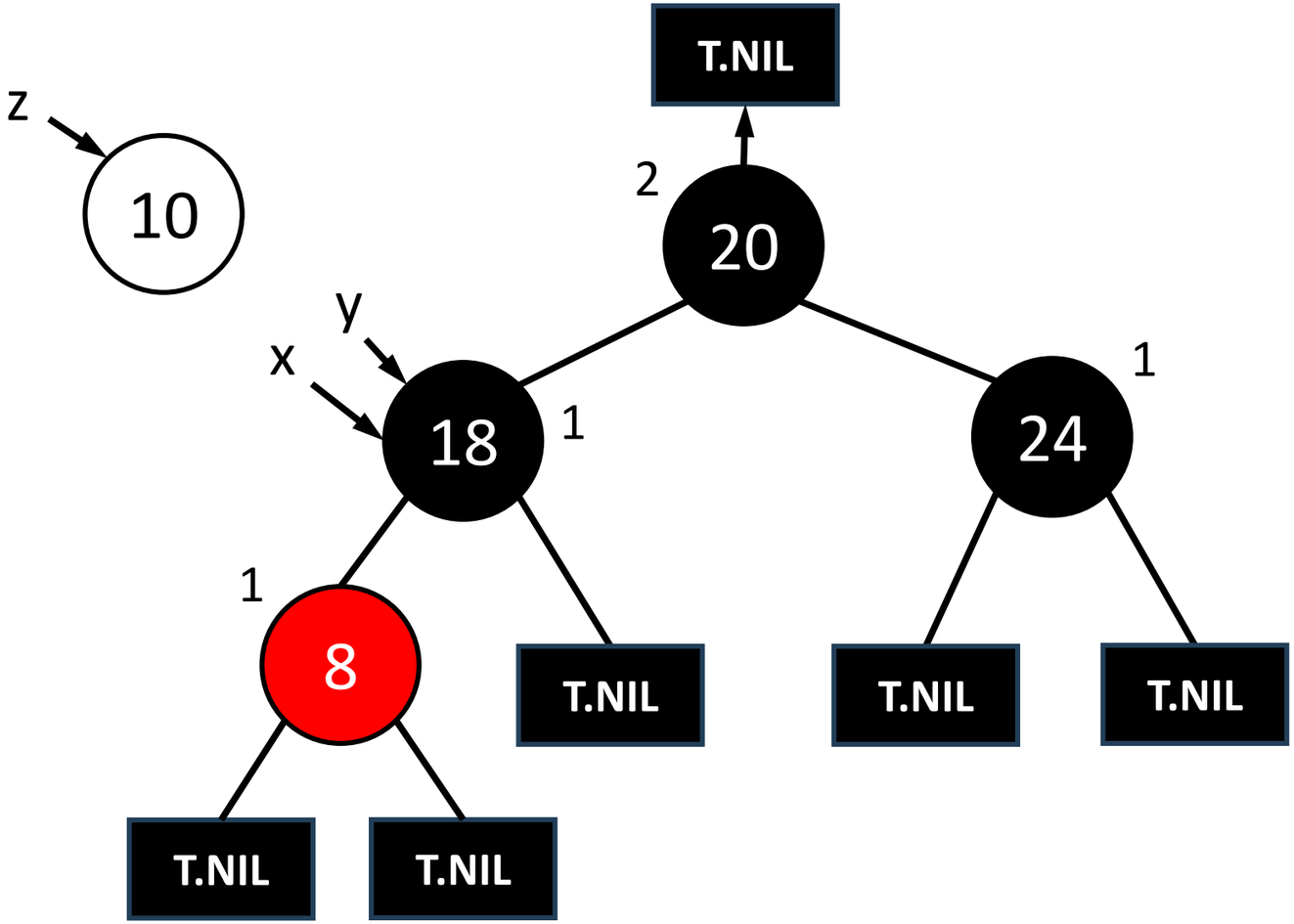
Inserting a New Node into a Red-Black Tree (contd.)

```
while x != T.nil
    y = x
    if z.key < x.key
        x = x.left
    else
        x = x.right
```



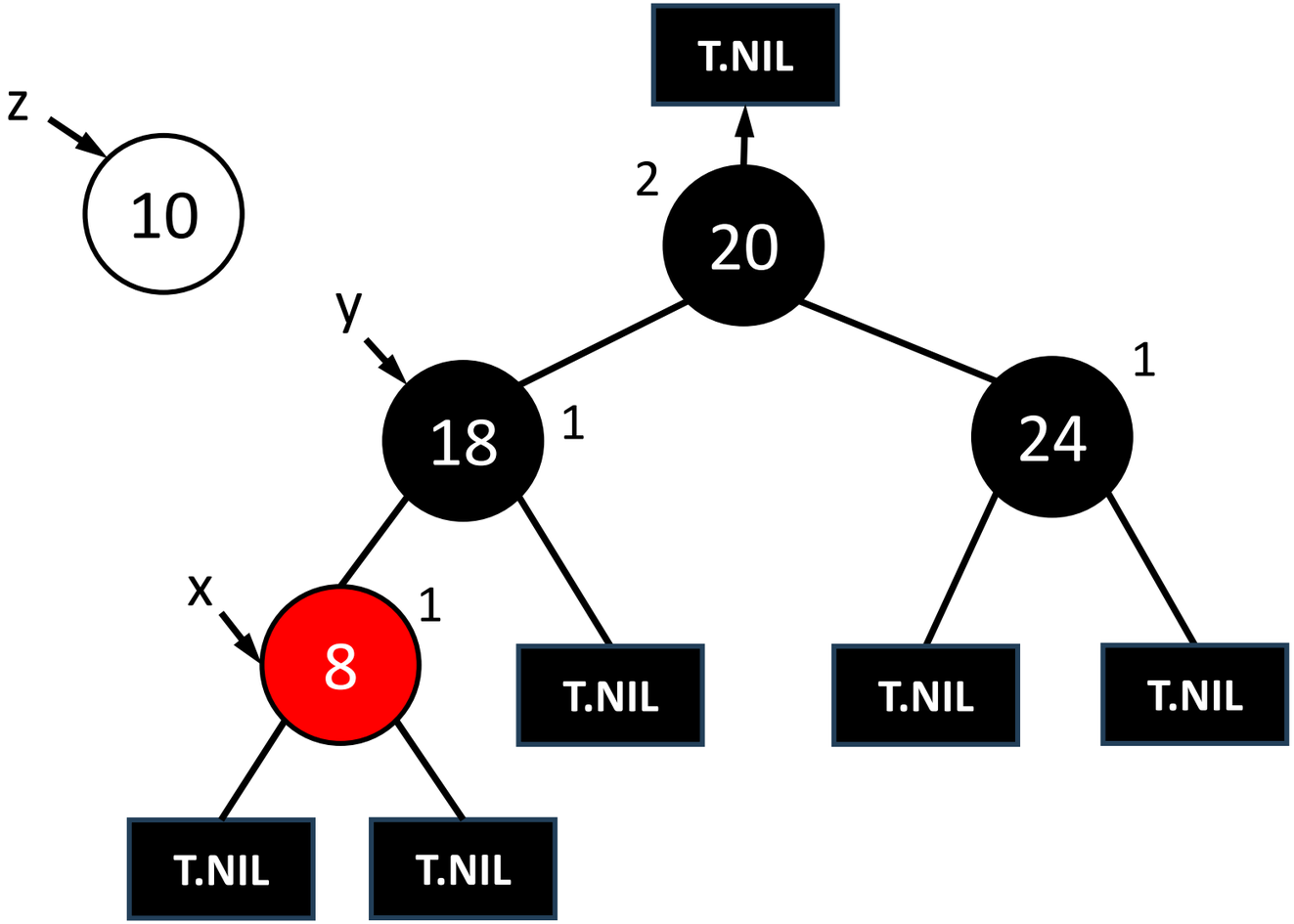
Inserting a New Node into a Red-Black Tree (contd.)

```
while x != T.nil
    y = x
    if z.key < x.key
      x = x.left
    else
      x = x.right
```



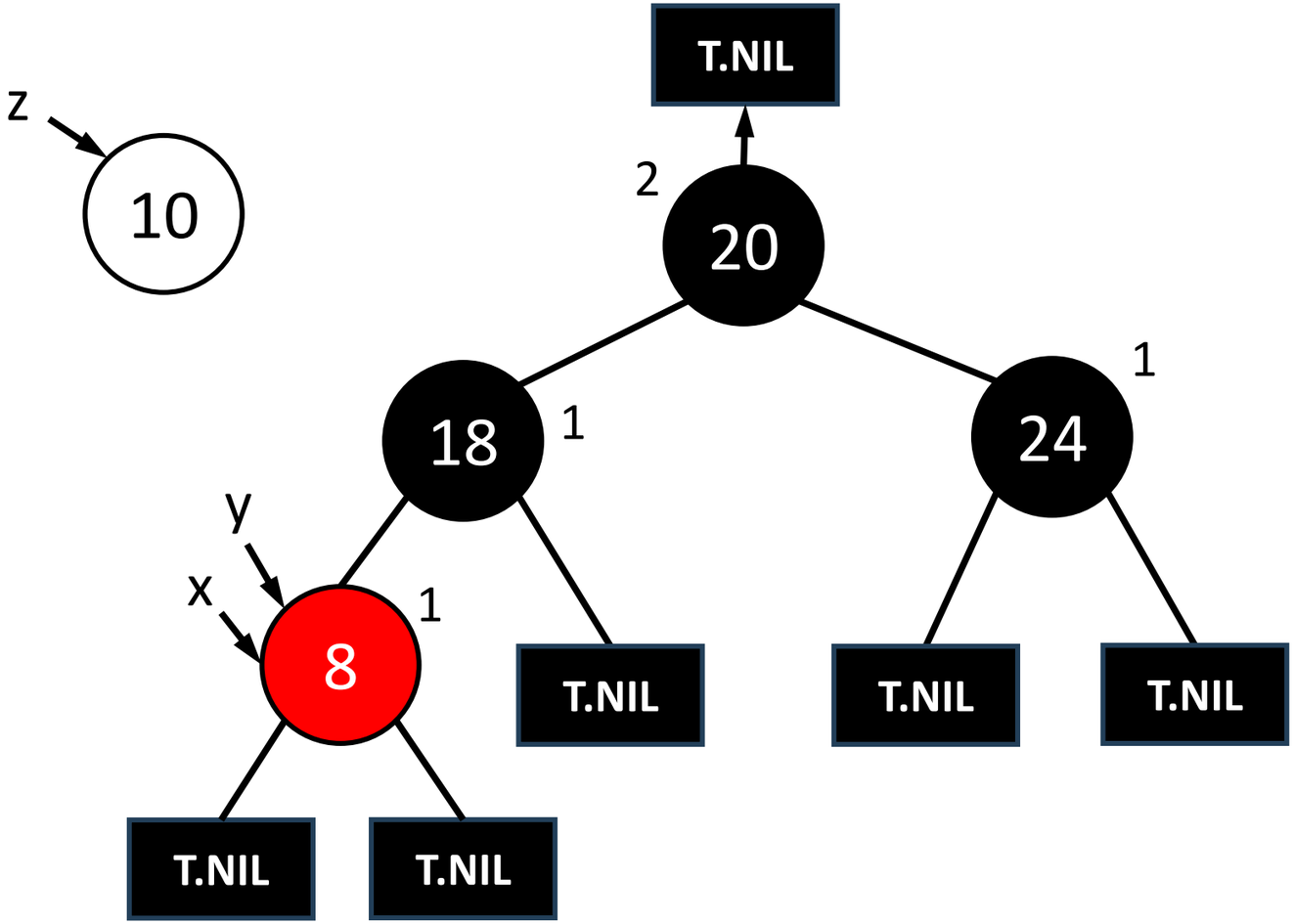
Inserting a New Node into a Red-Black Tree (contd.)

```
while x != T.nil
    y = x
    if z.key < x.key
        x = x.left
    else
        x = x.right
```



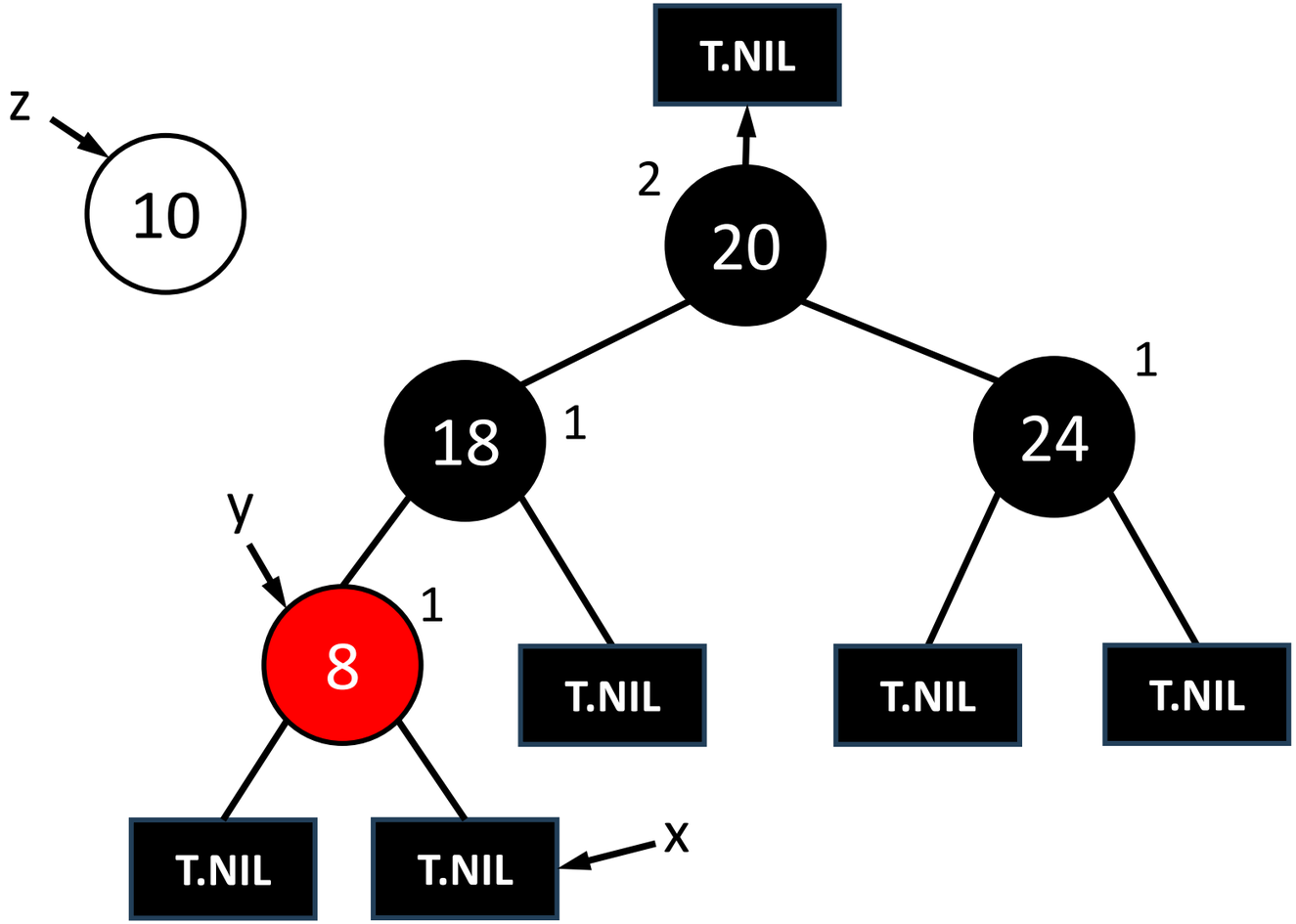
Inserting a New Node into a Red-Black Tree (contd.)

```
while x != T.nil
    y = x
    if z.key < x.key
      x = x.left
    else
      x = x.right
```



Inserting a New Node into a Red-Black Tree (contd.)

```
while x != T.nil
    y = x
    if z.key < x.key
      x = x.left
    else
      x = x.right
```

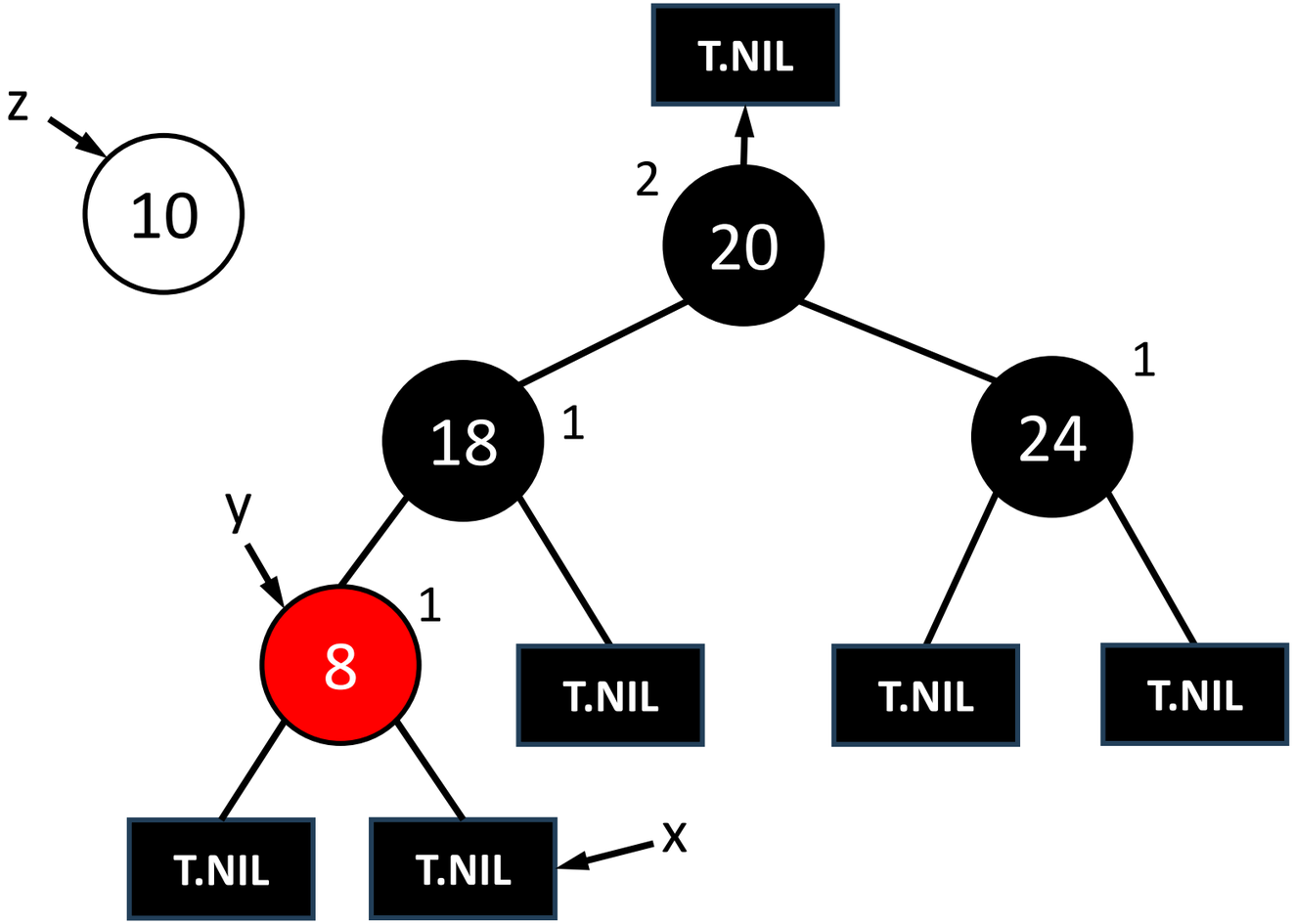


Inserting a New Node into a Red-Black Tree (contd.)

```
while x != T.nil // Does not satisfy
    y = x
    if z.key < x.key
      x = x.left
    else
      x = x.right
```

The program control exits the while loop.

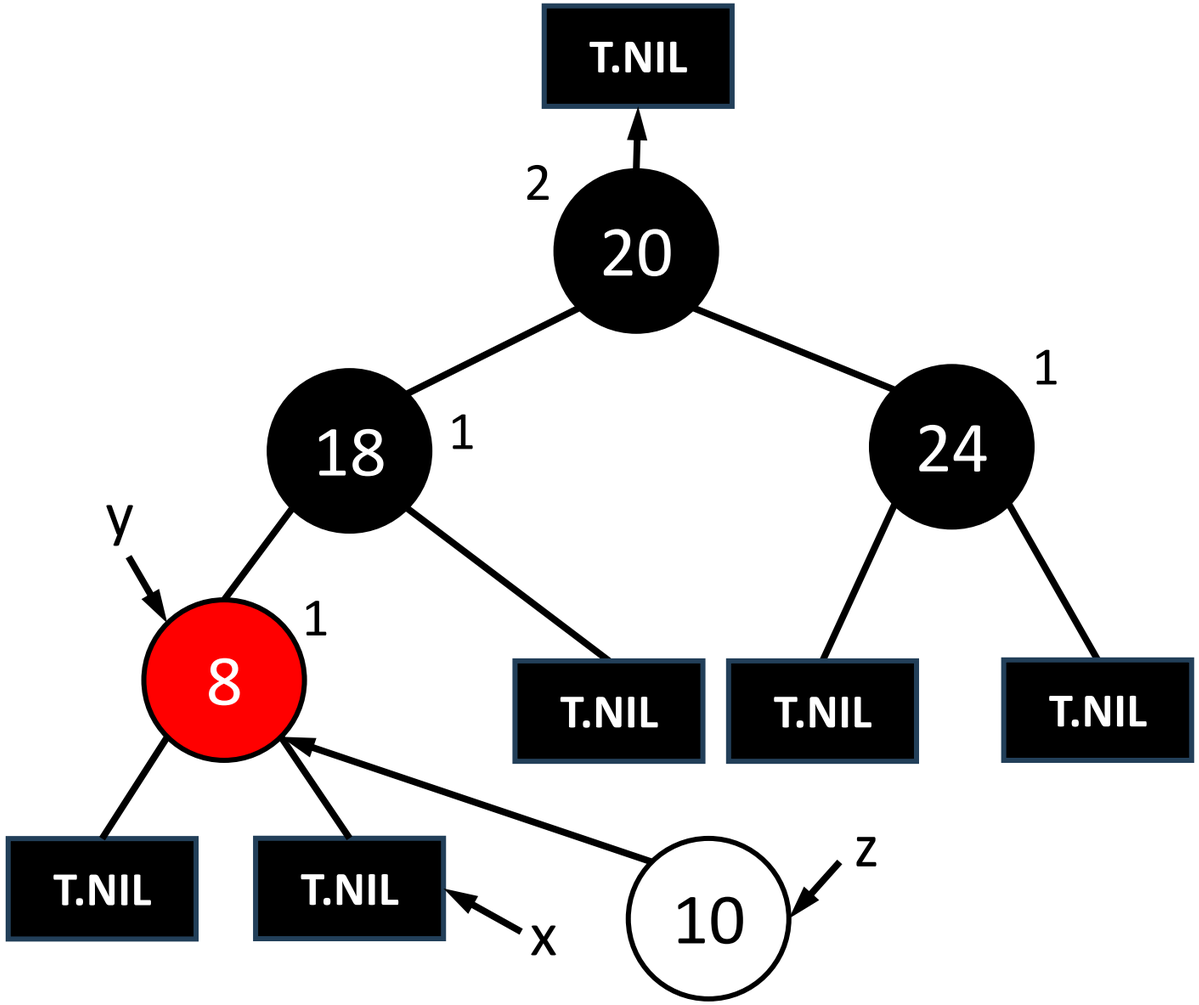
Therefore, the job of the while loop was to position x where z should be inserted and position y at x's parent.



Inserting a New Node into a Red-Black Tree (contd.)

z.p = y

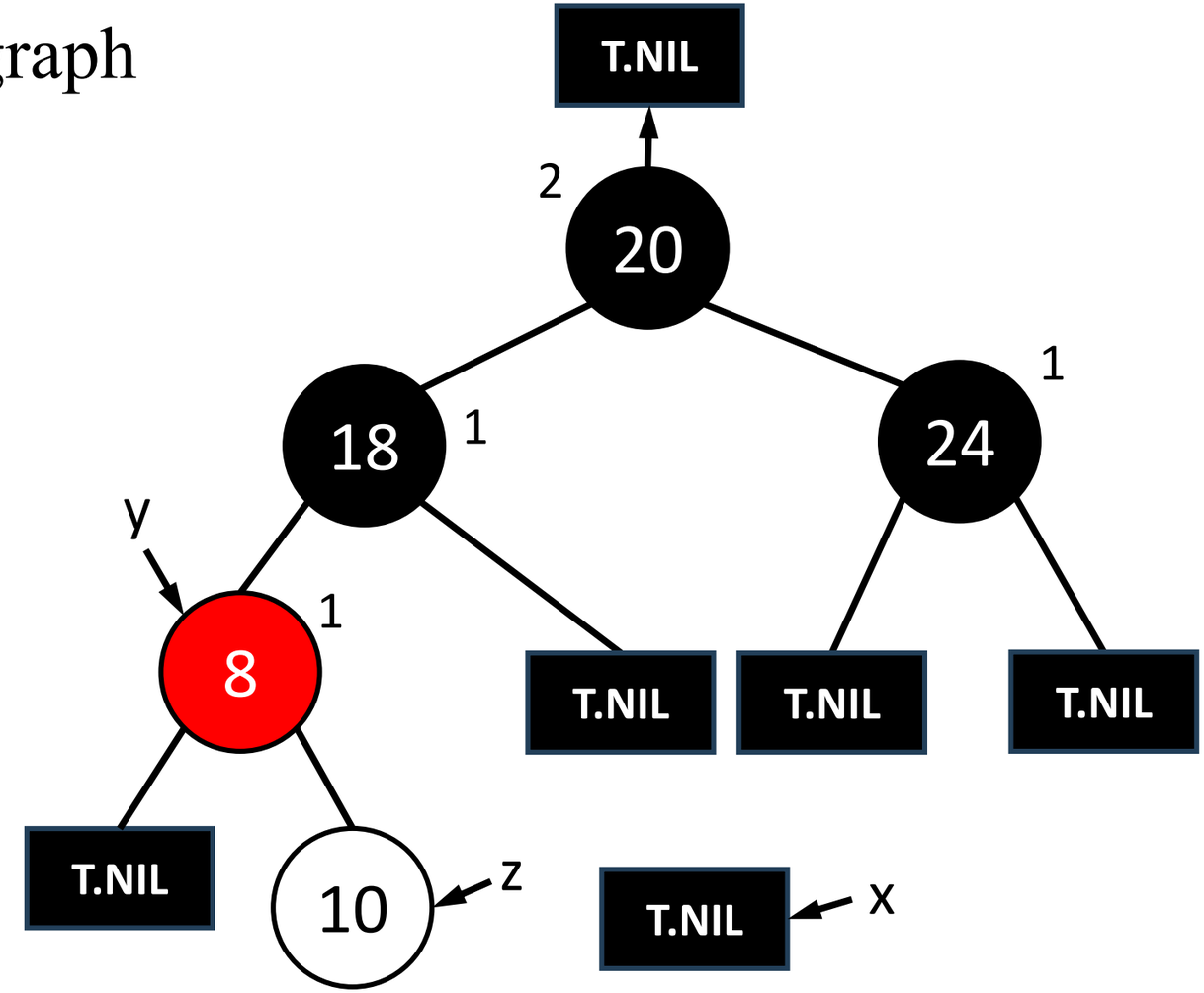
Make y the parent of z.



Inserting a New Node into a Red-Black Tree (contd.)

```
if y == T.nil // If T is an empty graph
    T.root = z
elseif z.key < y.key
    y.left = z
else
    y.right = z
```

Make z the right child of y.

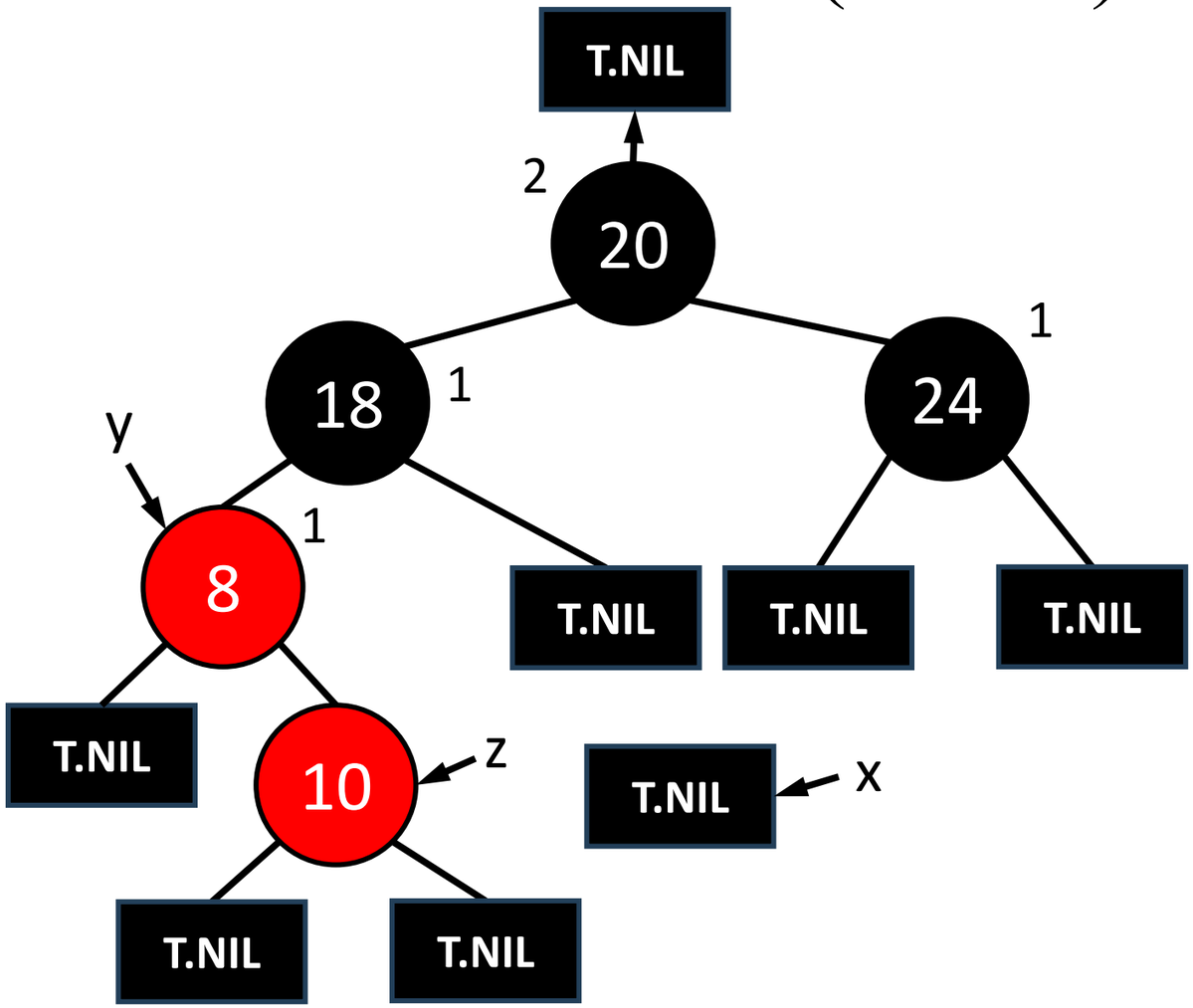


Inserting a New Node into a Red-Black Tree (contd.)

z.left = T.nil
z.right = T.nil
z.color = RED

Add NIL children to z and colour it red.

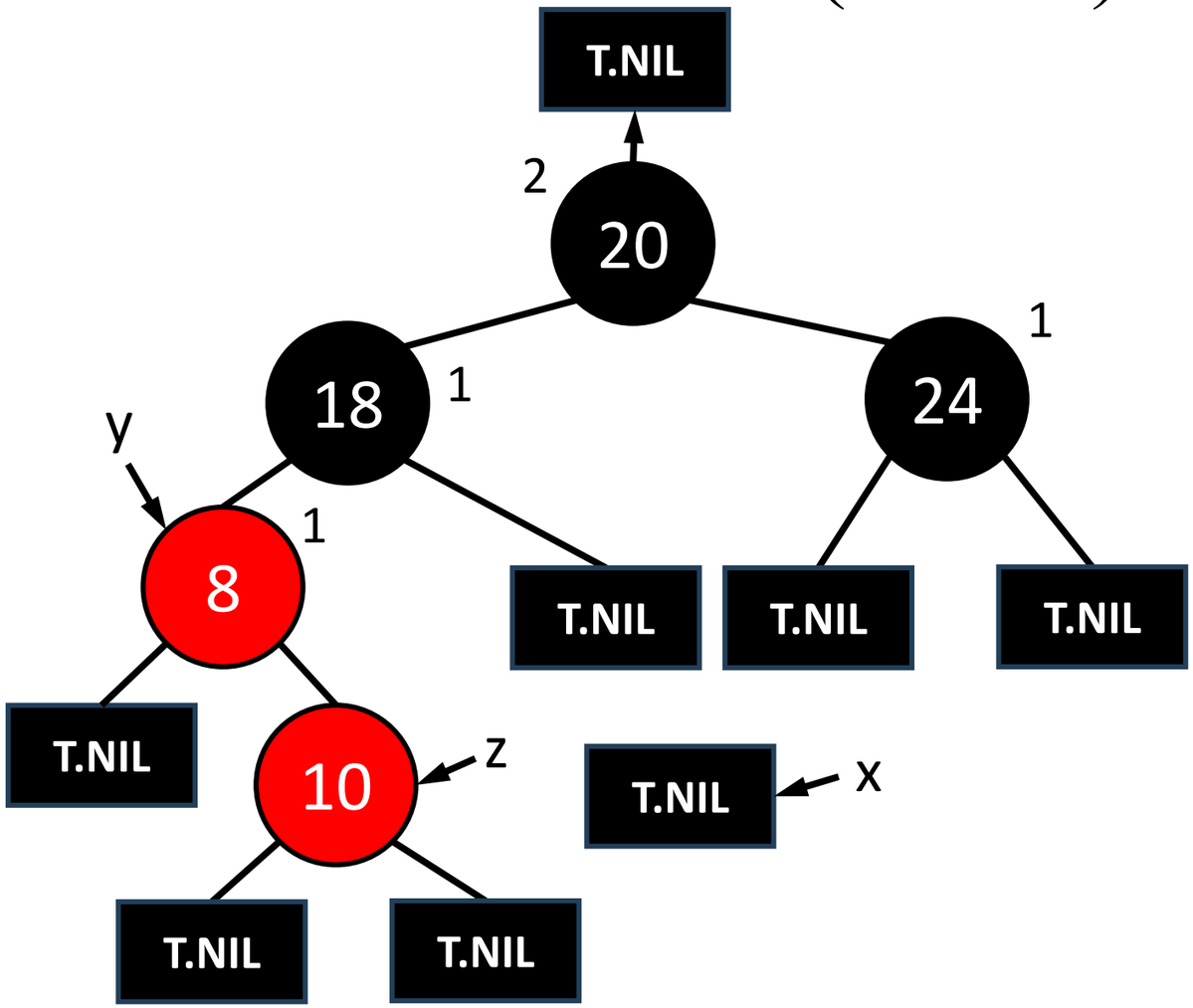
However, it may disturb the red-black properties of the tree.



Inserting a New Node into a Red-Black Tree (contd.)

RB-INSERT-FIXUP(T, z)

Fix the disturbance, if any, created by inserting node z into tree T by calling the RB-INSERT-FIXUP() subroutine.

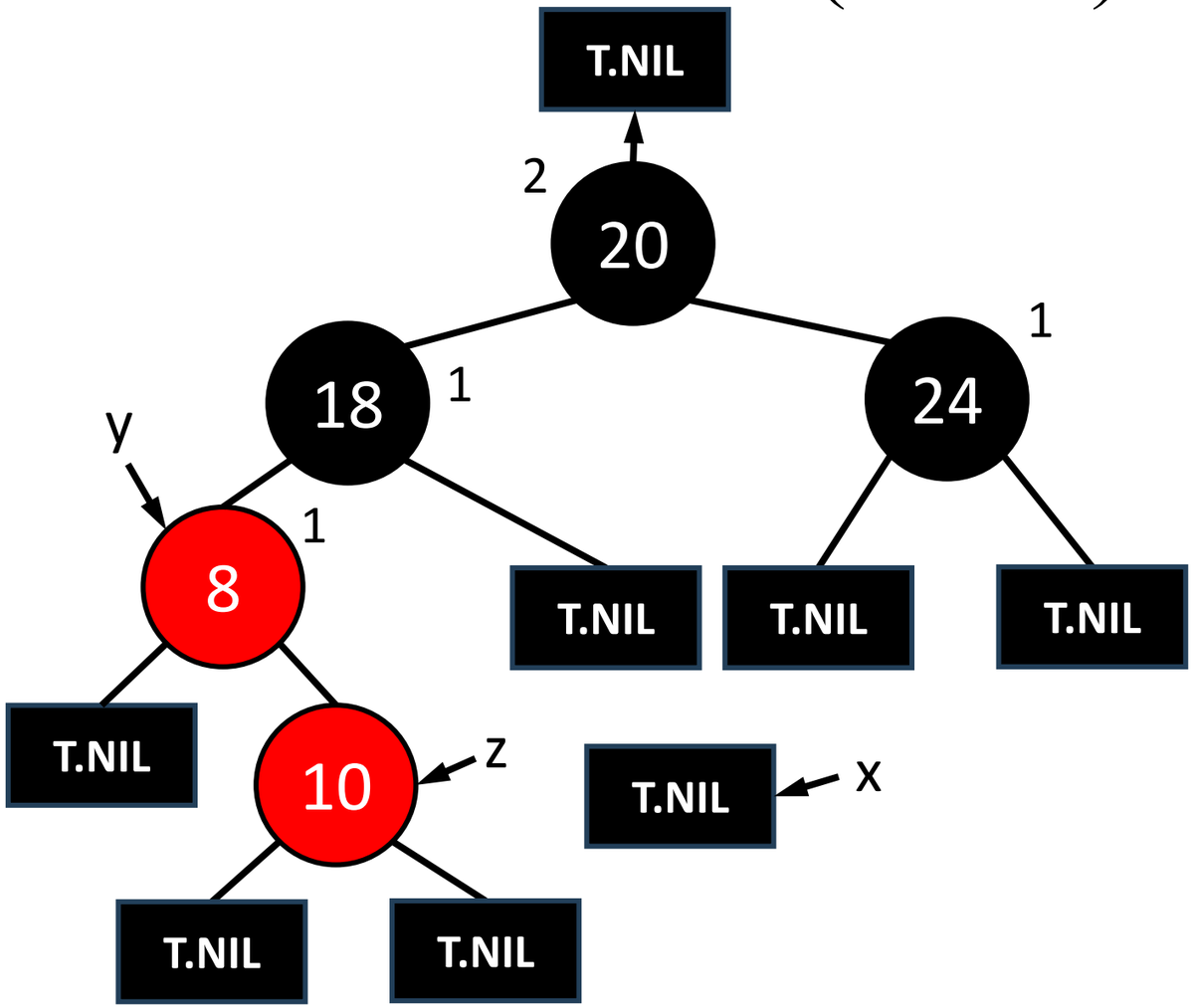


Inserting a New Node into a Red-Black Tree (contd.)

RB-INSERT-FIXUP(T, z)

```
while z.p.color == RED
    if z.p == z.p.p.left
        …
    elif z.p == z.p.p.right
        …
```

This subroutine uses a big while loop.

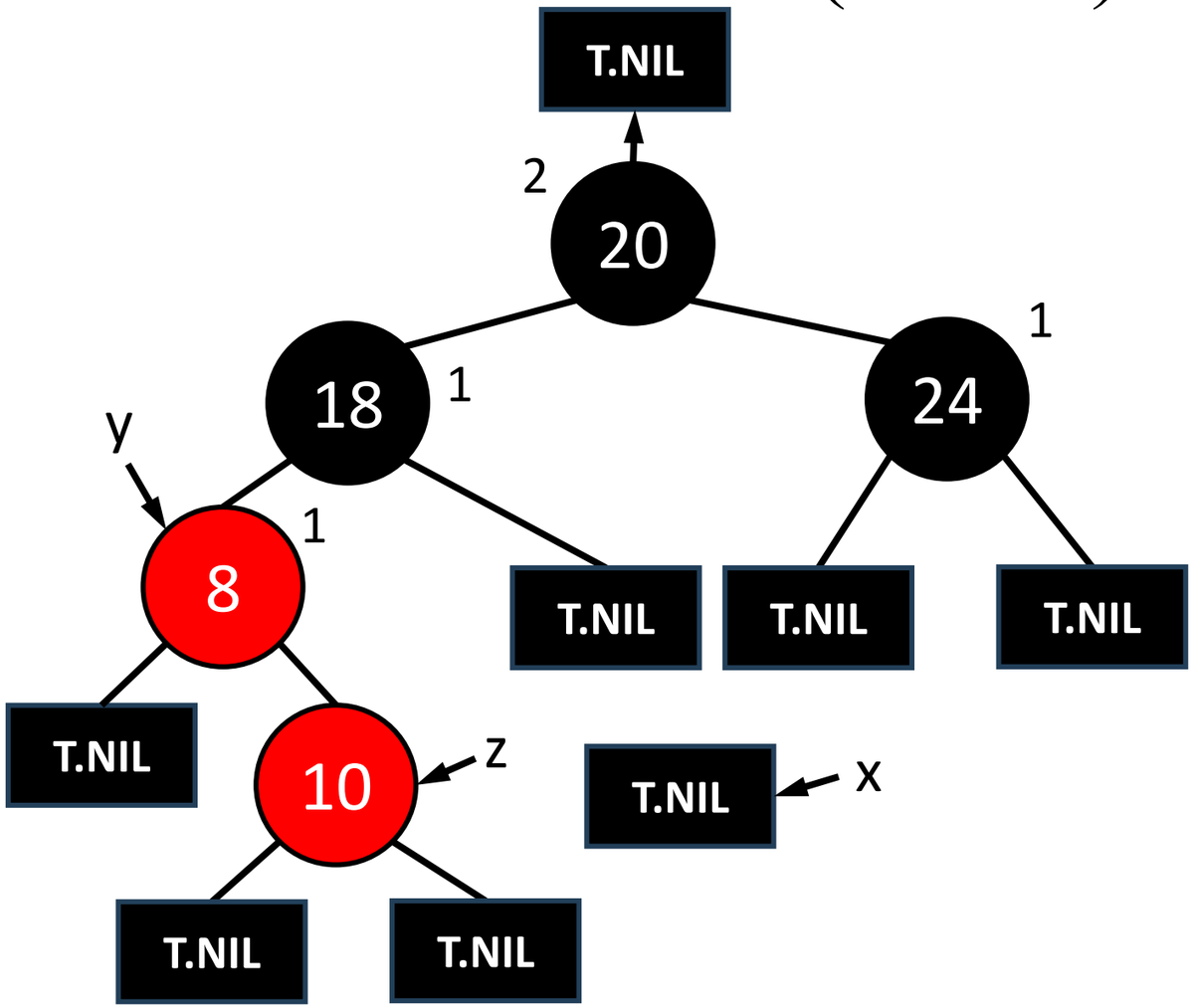


Inserting a New Node into a Red-Black Tree (contd.)

RB-INSERT-FIXUP(T, z)

```
while z.p.color == RED
    if z.p == z.p.p.left
        …
    elif z.p == z.p.p.right
        …
```

The big while loop checks whether z's parent is also red. If so, it violates the property that states "If a node is red, then both its children are black".



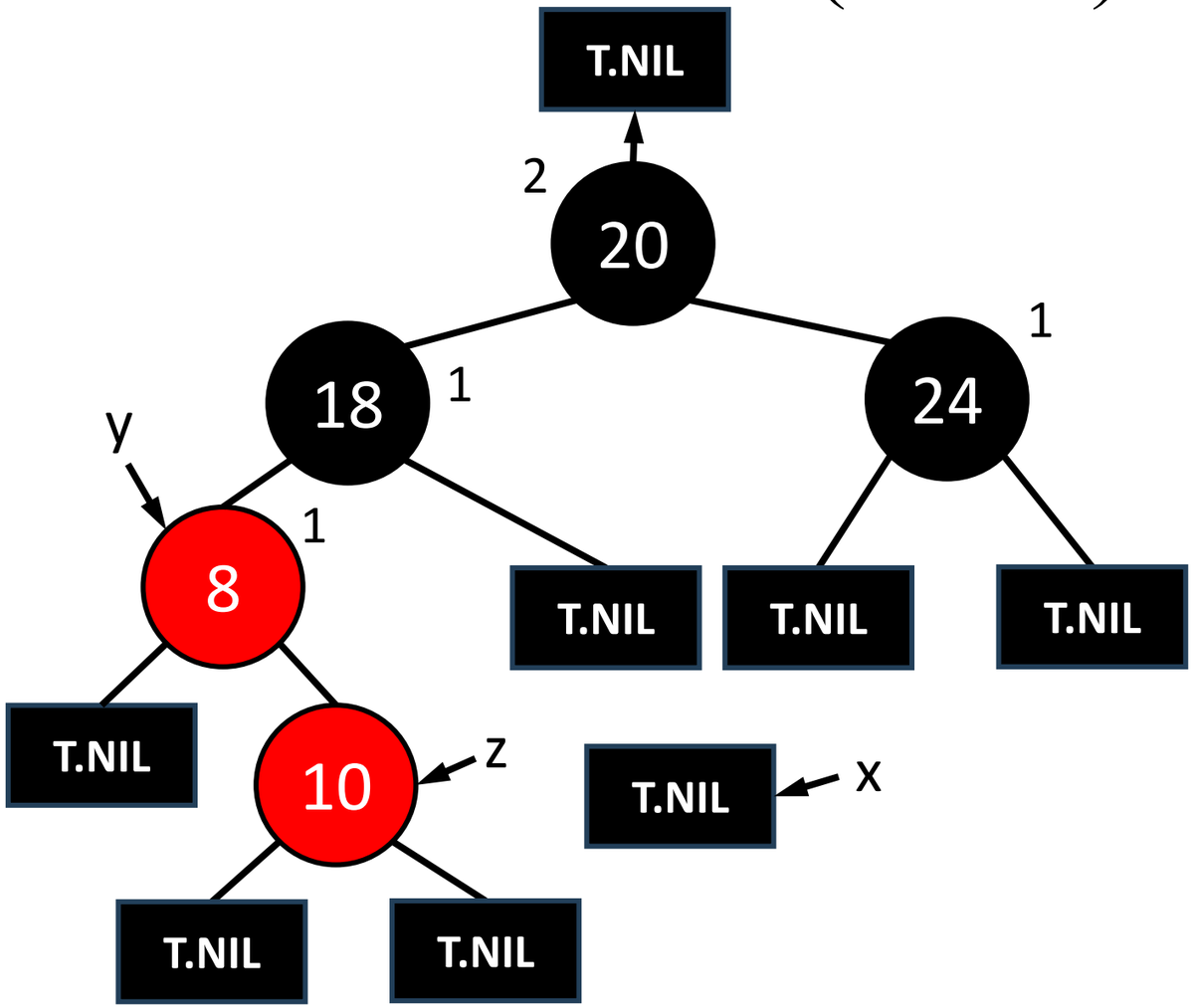
Inserting a New Node into a Red-Black Tree (contd.)

RB-INSERT-FIXUP(T, z)

```
while z.p.color == RED
    if z.p == z.p.p.left
        …
    elif z.p == z.p.p.right
        …
```

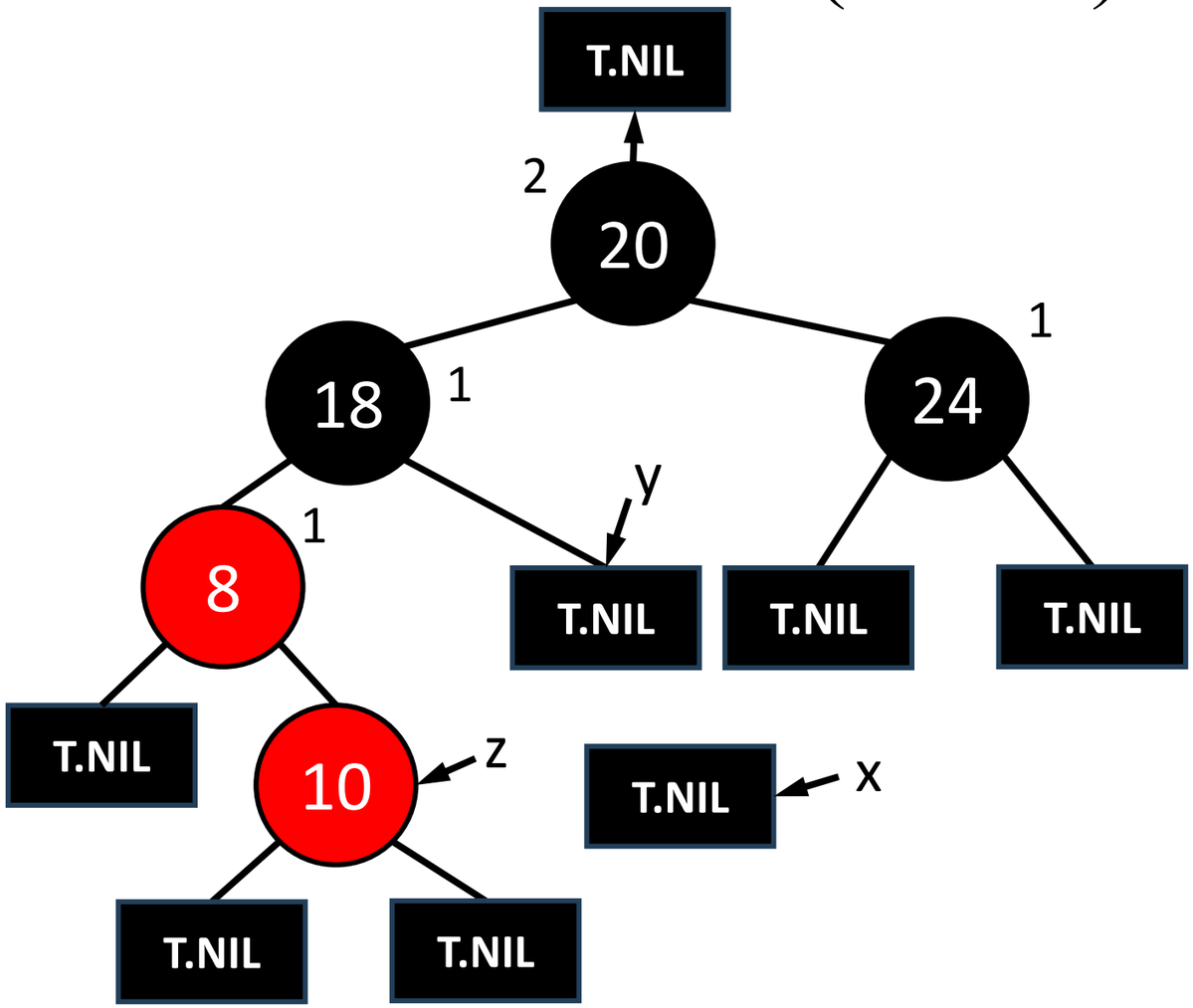
If violated, it checks whether z's parent is a left child of z's grandparent or a right child of z's grandparent.

In this example, z's parent is a left child of z's grandparent.



Inserting a New Node into a Red-Black Tree (contd.)

```
if z.p == z.p.p.left
    y = z.p.p.right // Point y at z's uncle
    if y.color == RED
        z.p.color = BLACK // case 1
        y.color = BLACK // case 1
        z.p.p.color = RED // case 1
        z = z.p.p // case 1
    else if y.color ==  BLACK
        if z == z.p.right
            z = z.p // case 2
            LEFT-ROTATE(T, z) // case 2
        z.p.color = BLACK // cases 2+3
        z.p.p.color = RED // cases 2+3
        RIGHT-ROTATE(T, z.p.p) // cases 2+3
```

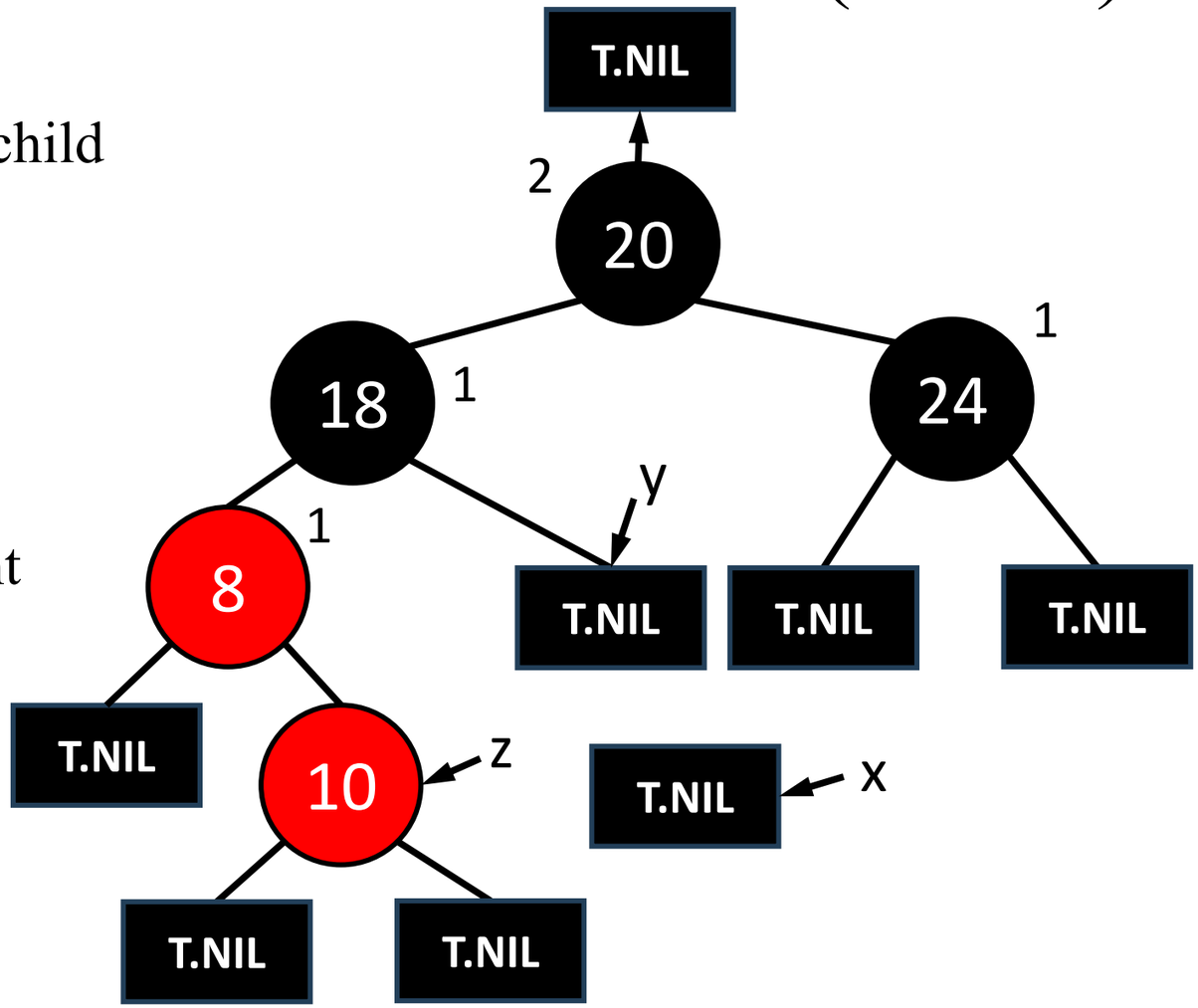


Inserting a New Node into a Red-Black Tree (contd.)

if z.p == z.p.p.left // If z's parent is a left child
    y = z.p.p.right // Point y at z's uncle

Now, one of the three cases can happen.
Case 1. z's uncle y is red.

Case 2. z's uncle y is black and z is a right child.
Case 3. z's uncle y is black and z is a left child.

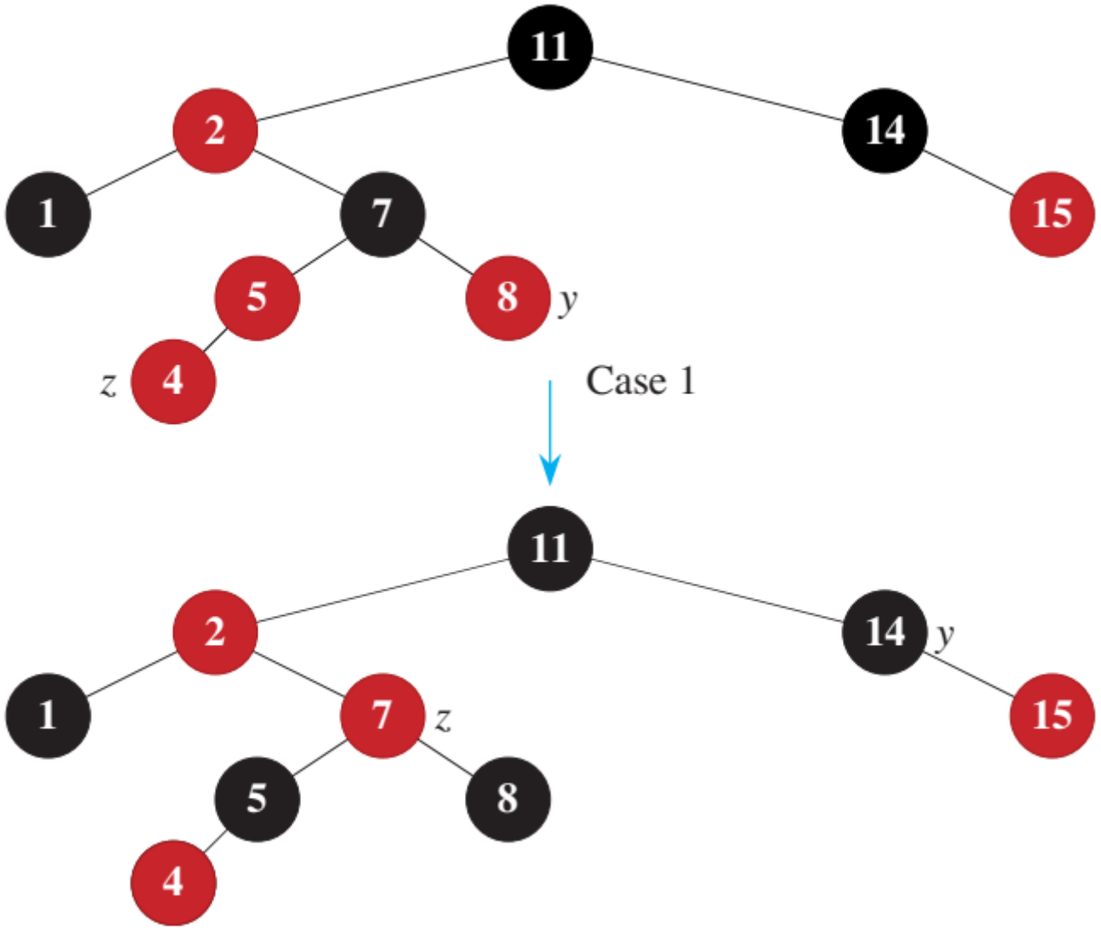


Inserting a New Node into a Red-Black Tree (contd.)

Case 1. z's uncle y is red.

```
if y.color == RED
    z.p.color = BLACK // case 1
    y.color = BLACK // case 1
    z.p.p.color = RED // case 1
    z = z.p.p // case 1
```

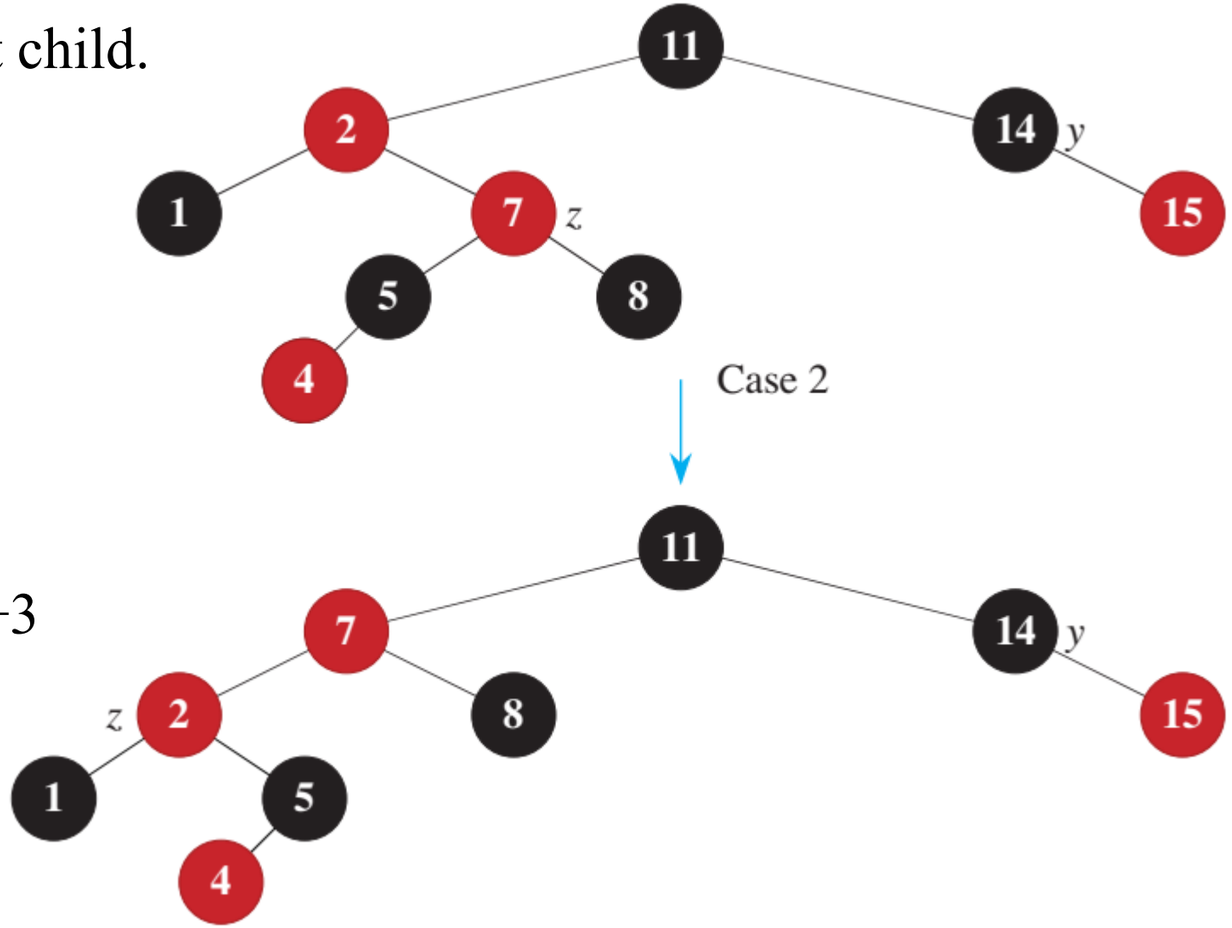
Recolour z's parent, z's uncle, and z's grandparent.
Move the z pointer to its grandparent.



Inserting a New Node into a Red-Black Tree (contd.)

Case 2. z's uncle y is black and z is a right child.

```
else if y.color ==  BLACK
    if z == z.p.right
        z = z.p // case 2
        LEFT-ROTATE(T, z) // case 2
    z.p.color = BLACK // cases 2+3
    z.p.p.color = RED // cases 2+3
    RIGHT-ROTATE(T, z.p.p) // cases 2+3
```



The left rotation in Case 2 always transforms the situation into Case 3.

The Pseudocode of the Left Rotation

The pseudocode of the right rotation would be similar

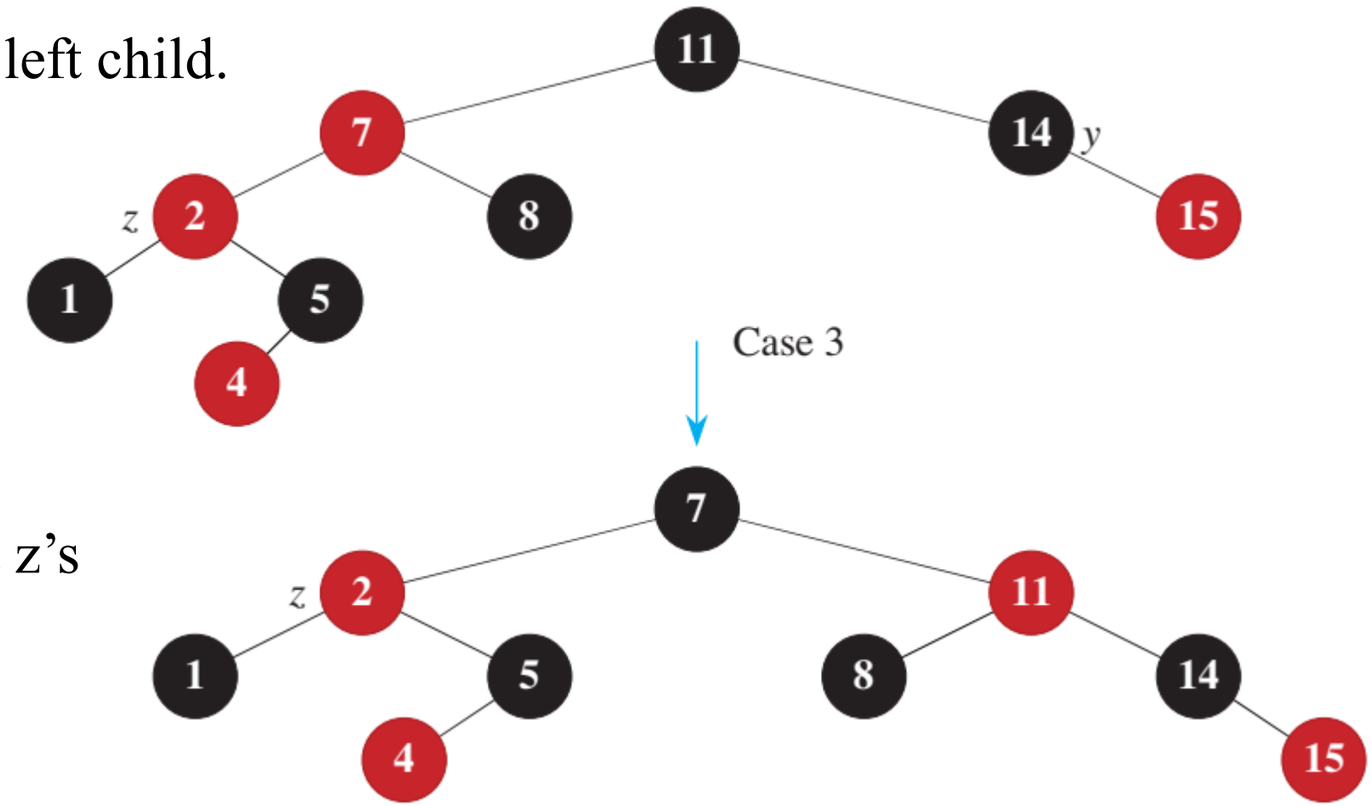
```
LEFT-ROTATE(T, x)
y = x.right
x.right = y.left          // turn y's left subtree into x's right subtree
if y.left ≠ T.nil         // if y's left subtree is not empty ...
    y.left.p = x          // ... then x becomes the parent of the subtree's root
y.p = x.p                 // x's parent becomes y's parent
if x.p == T.nil           // if x was the root ...
    T.root = y            // ... then y becomes the root
elseif x == x.p.left      // otherwise, if x was a left child ...
    x.p.left = y          // ... then y becomes a left child
else x.p.right = y        // otherwise, x was a right child, and now y is
y.left = x                // make x become y's left child
x.p = y
```

Inserting a New Node into a Red-Black Tree (contd.)

Case 3. z's uncle y is black and z is a left child.

z.p.color = BLACK // case 3
z.p.p.color = RED // case 3
RIGHT-ROTATE(T, z.p.p) // case 3

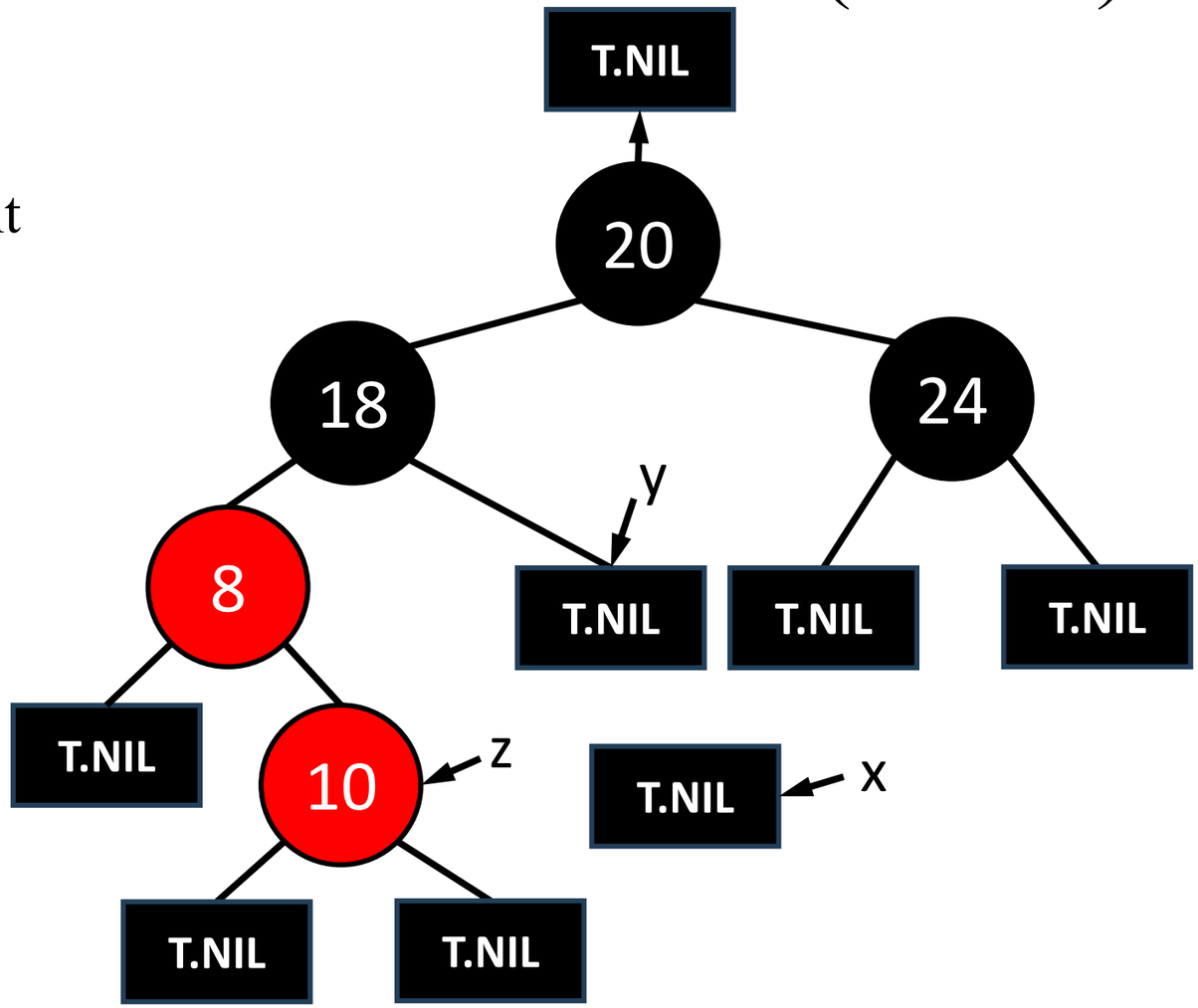
Recolour z's parent and grandparent.
Then right rotate the subtree rooted at z's grandparent.



Inserting a New Node into a Red-Black Tree (contd.)

Our example follows:
Case 2. z's uncle y is black and z is a right child.

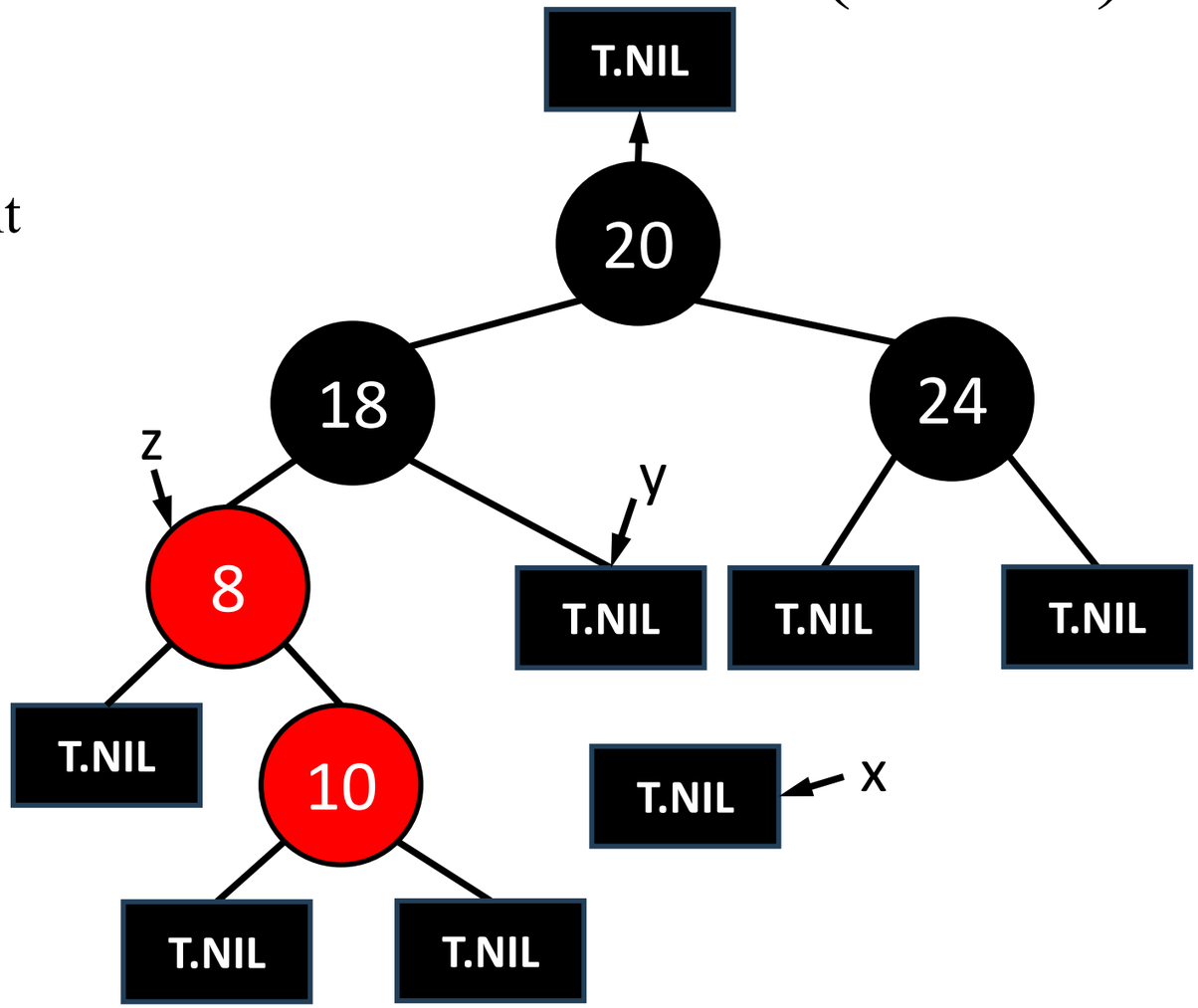
```
if z == z.p.right
    z = z.p // case 2
    LEFT-ROTATE(T, z) // case 2
z.p.color = BLACK // cases 2+3
z.p.p.color = RED // cases 2+3
RIGHT-ROTATE(T, z.p.p) // cases 2+3
```



Inserting a New Node into a Red-Black Tree (contd.)

Our example follows:
Case 2. z's uncle y is black and z is a right child.

```
if z == z.p.right
    z = z.p // case 2
    LEFT-ROTATE(T, z) // case 2
z.p.color = BLACK // cases 2+3
z.p.p.color = RED // cases 2+3
RIGHT-ROTATE(T, z.p.p) // cases 2+3
```



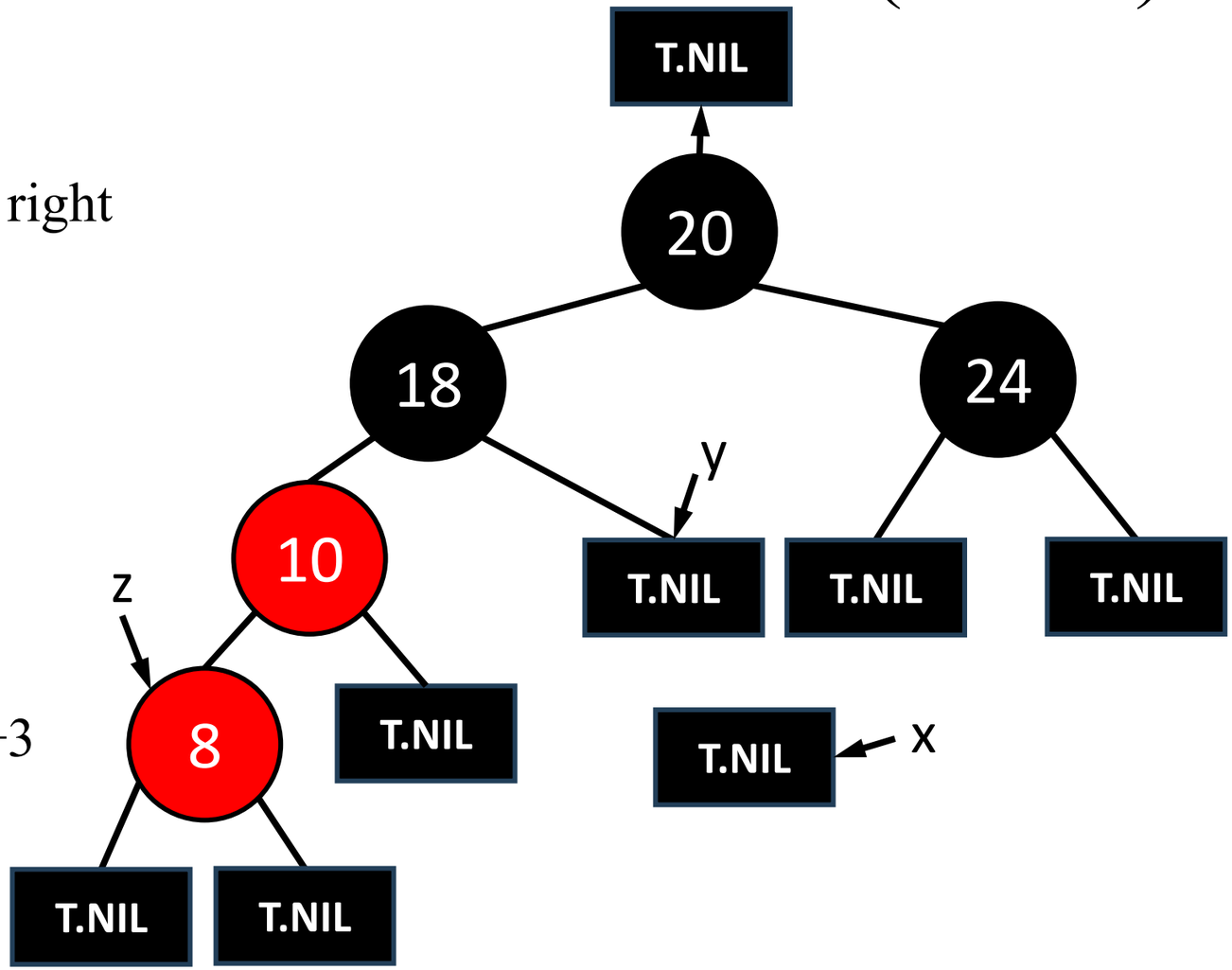
Move the z pointer to its parent.

Inserting a New Node into a Red-Black Tree (contd.)

Our example follows:
Case 2. z's uncle y is black and z is a right child.

```
if z == z.p.right
    z = z.p // case 2
    LEFT-ROTATE(T, z) // case 2
z.p.color = BLACK // cases 2+3
z.p.p.color = RED // cases 2+3
RIGHT-ROTATE(T, z.p.p) // cases 2+3
```

Then left rotate the subtree rooted at z.

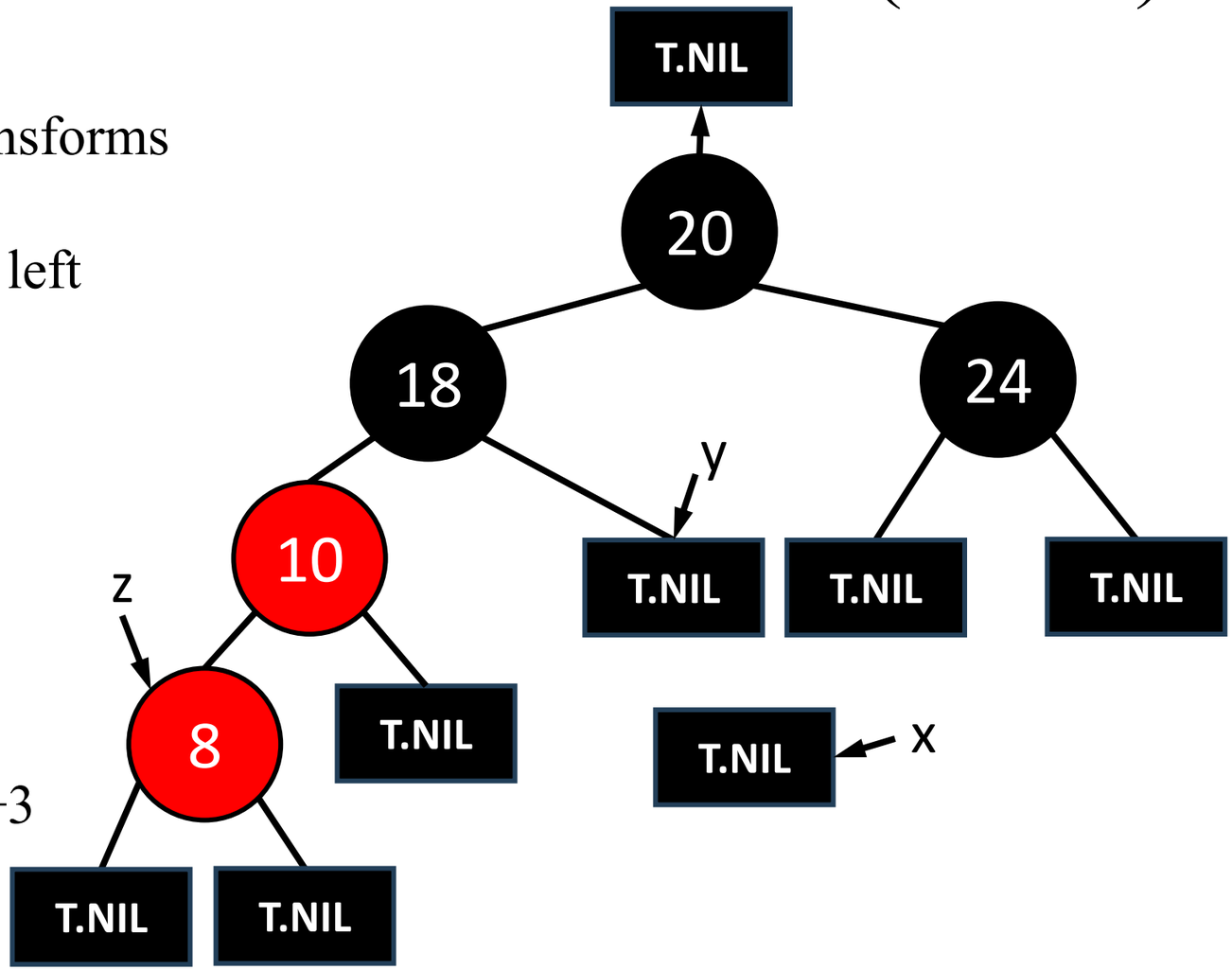


Inserting a New Node into a Red-Black Tree (contd.)

The left rotation of Case 2 always transforms the situation into Case 3.

Case 3. z's uncle y is black and z is a left child.

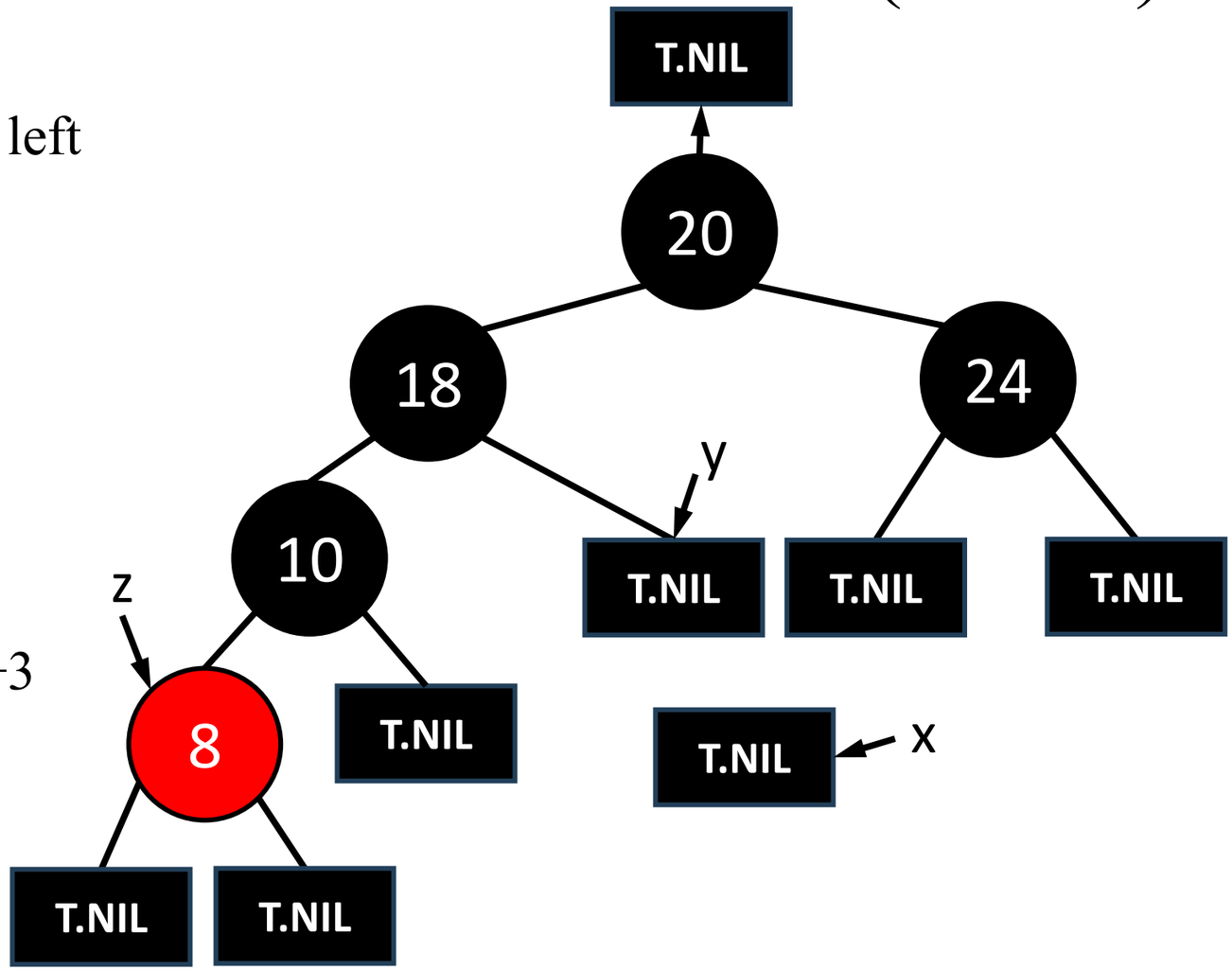
```
if z == z.p.right
    z = z.p // case 2
    LEFT-ROTATE(T, z) // case 2
z.p.color = BLACK // cases 2+3
z.p.p.color = RED // cases 2+3
RIGHT-ROTATE(T, z.p.p) // cases 2+3
```



Inserting a New Node into a Red-Black Tree (contd.)

Case 3. z's uncle y is black and z is a left child.

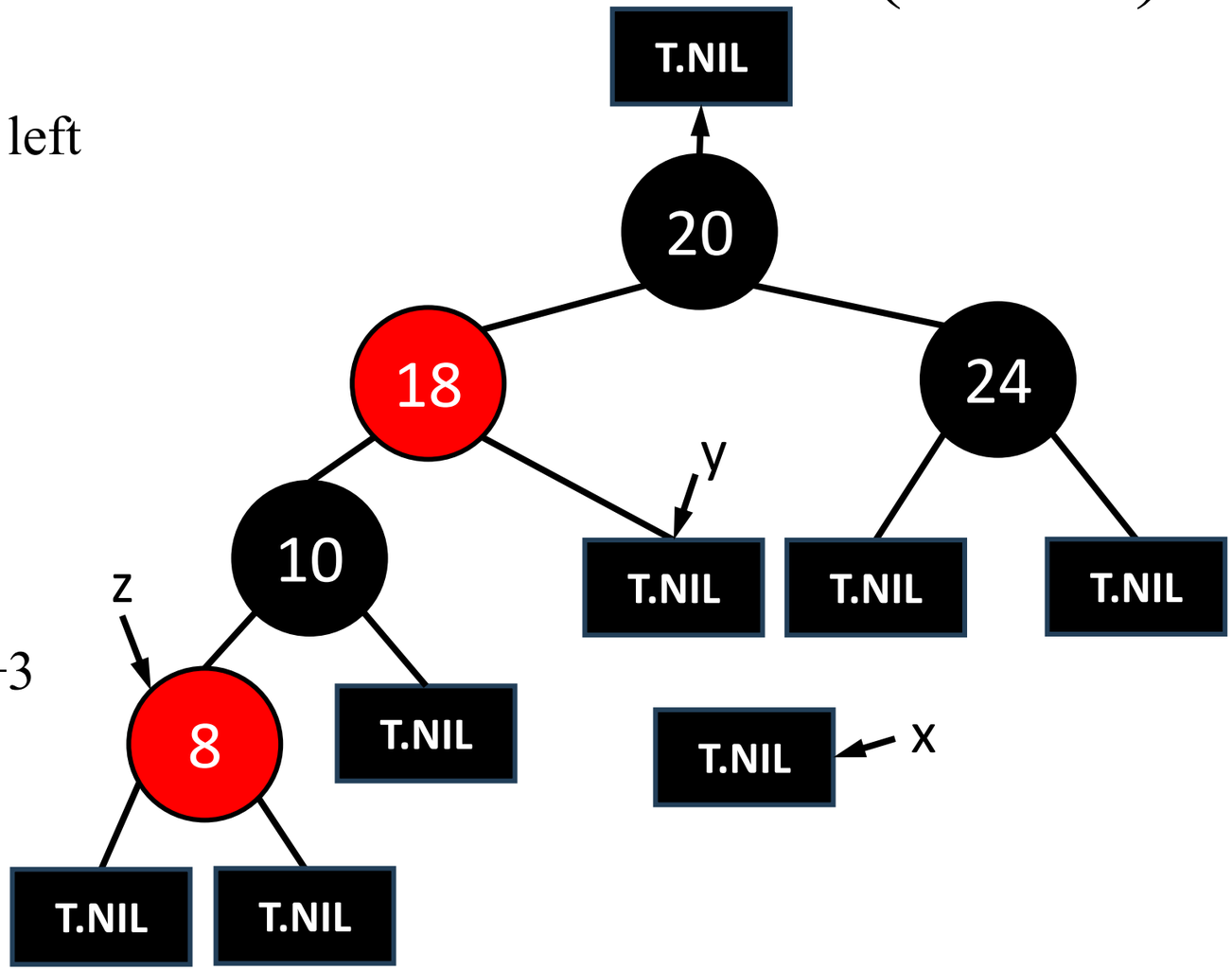
```
if z == z.p.right
    z = z.p // case 2
    LEFT-ROTATE(T, z) // case 2
z.p.color = BLACK // cases 2+3
z.p.p.color = RED // cases 2+3
RIGHT-ROTATE(T, z.p.p) // cases 2+3
```



Inserting a New Node into a Red-Black Tree (contd.)

Case 3. z's uncle y is black and z is a left child.

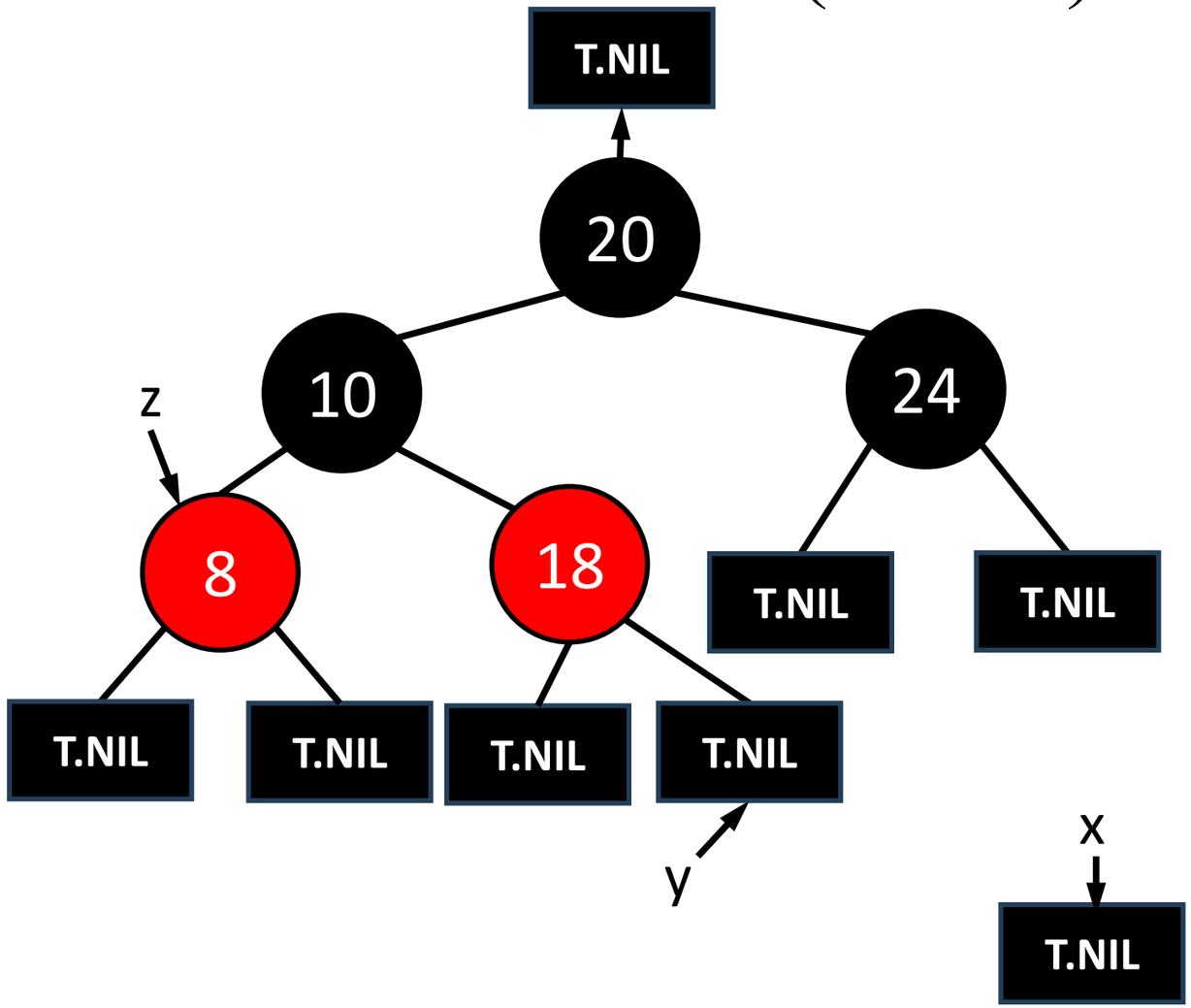
```
if z == z.p.right
    z = z.p // case 2
    LEFT-ROTATE(T, z) // case 2
z.p.color = BLACK // cases 2+3
z.p.p.color = RED // cases 2+3
RIGHT-ROTATE(T, z.p.p) // cases 2+3
```



Inserting a New Node into a Red-Black Tree (contd.)

Case 3. z's uncle y is black and z is a left child.

```
if z == z.p.right
    z = z.p // case 2
    LEFT-ROTATE(T, z) // case 2
z.p.color = BLACK // cases 2+3
z.p.p.color = RED // cases 2+3
RIGHT-ROTATE(T, z.p.p) // cases 2+3
```



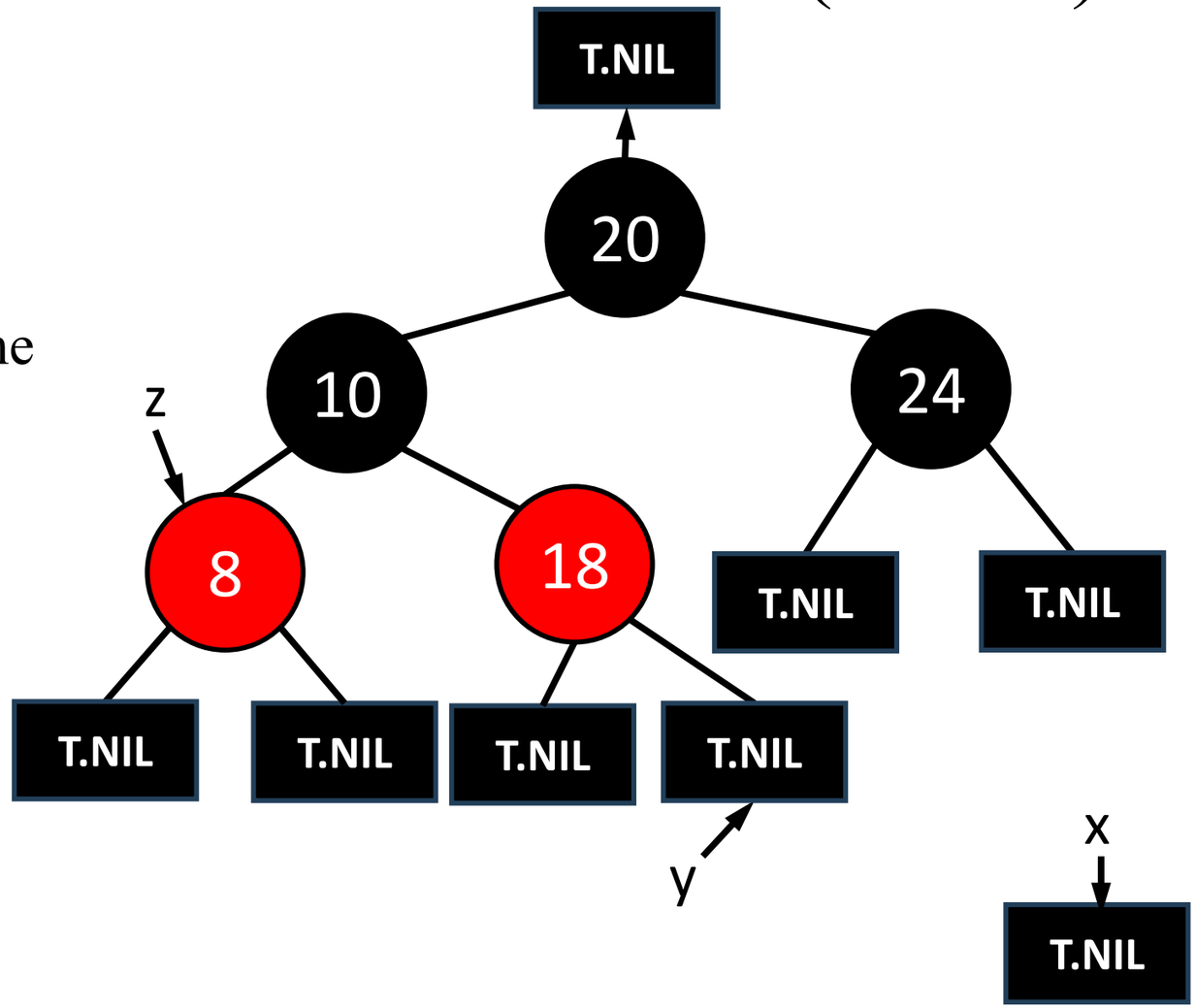
Inserting a New Node into a Red-Black Tree (contd.)

z's parent is no longer red, hence,
we exit the while loop.

There is only the following line outside the while loop.

T.root.color = BLACK

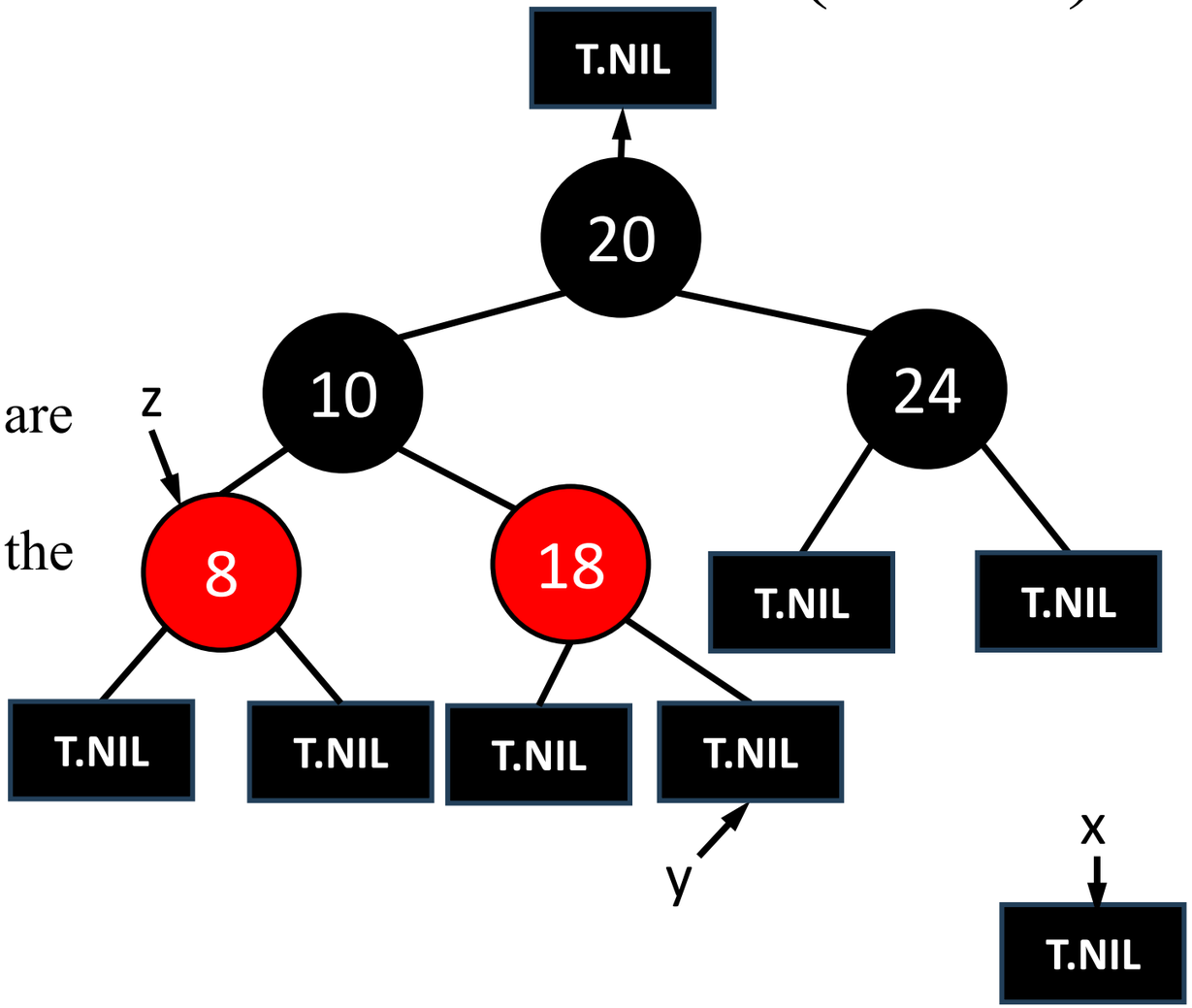
This is to colour the root black either because the tree T was empty and z is the only node now or
in case the root became red during the execution of the while loop.



Inserting a New Node into a Red-Black Tree (contd.)

The output is a valid red-black tree.

1. Every node is either red or black.
2. The root node is black.
3. Every **leaf node (NIL)** is black.
4. If a node is red, then both its children are black.
5. For each node, all **simple paths** from the node to descendant leaves contain the same number of black nodes.



Inserting a New Node into a Red-Black Tree: Recap

Insert the new node z into tree T as we would do for any binary search tree.

Colour z red and add two NIL children.

Call the fixup routine to **verify and restore** the red-black properties.

The fixup routine makes y the uncle of z if z's parent is red (violation of Property 4).
Then it iteratively checks for the following three cases until Property 4 is no longer violated.
Case 1. z's uncle y is **red**.
Case 2. z's uncle y is **black** and z is a right child.
Case 3. z's uncle y is **black** and z is a left child.
Finally, the fixup routine colours the root black.

Deleting a Node from a Red-Black Tree

Please study Section 13.4 ‘Deletion’, CLRS **4th Edition, not the 3rd Edition**.

It might seem a little complex at first.

However, if you study it step by step, just as we did for insertion, it will become easy and straight-forward.

Summary

We discussed two **height-balanced binary search trees —** the AVL tree and the red-black tree.

Next Class

Can we have search trees that are **not binary**?

Why not. We can have **n-ary search trees** where each node can have any number of children.

The **B-tree** is one such search tree. We will discuss them in the next class.

Thank You